Accelerating the Inference Era with AI-Driven, Globally Optimized HW/SW Co-Design

Miria Feng
Electrical Engineering
Stanford University
miria00@stanford.edu

Fangzhao Zhang
Electrical Engineering
Stanford University
zfzhao@stanford.edu

Adrian Lafuente
Computer Science
Stanford University
agamarra@stanford.edu

Mert Pilanci
Electrical Engineering
Stanford University
pilanci@stanford.edu

Azalia Mirhoseini
Computer Science
Stanford University
azalia@stanford.edu

Abstract

Deep learning inference demand is projected to grow $10{,}000\times$ over the next five years, a trajectory that general-purpose accelerators cannot match. Custom accelerators offer a path forward, but designing them requires co-optimizing hardware datapaths, software schedules, and compiler transforms across a search space exceeding $\mathcal{O}(10^{2300})$. Existing methodologies rely on decoupled, sequential pipelines that miss critical cross-stage trade-offs. We introduce CHEETAH: a globally optimized, AI-driven framework for inference accelerator co-design, and the first to combine *joint* tiling-fusion, dynamic placement, and rematerialization in a single linear programming (LP) formulation. Key contributions include: (1) time-indexed variables for dynamic memory management, (2) rematerialization variables that trade cheap recomputation for costly DRAM reloads, and (3) joint tiling-fusion optimization via McCormick linearization. An LLM-guided outer loop steers this inner loop through causal interventions based on deployment profiles and Lagrangian sensitivity signals. In under 6 hours, CHEETAH discovers Pareto-frontier designs across 12 workloads spanning CNNs, diffusion, and dense and mixture-of-experts LLMs, reaching up to $\sim 11.85\times$ speedup on vision, $\sim 24.20\times$ on LLM prefill, and $\sim 17.93\times$ on LLM decode over a simulated TPU-v3 baseline, with a certified 0.07% optimality gap on the largest instance, unlocking efficiency inaccessible to sequential pipelines.

1 Introduction

AI inference demand is expected to grow by $10{,}000\times$ over the next five years [Computing Research Association, 2026], and general-purpose accelerators leave much of the attainable efficiency unrealized, which motivates accelerators specialized to their workloads. A specialized accelerator is not one decision but many coupled ones: datapath and memory sizing, per-operation tiling, which tensors stay resident and when, what is fused, and what is recomputed rather than reloaded. These choices interact through shared capacity and bandwidth, and jointly they span a design space exceeding $\mathcal{O}(10^{2300})$ configurations [Zhang et al., 2022], far too large to enumerate, simulate, or materialize as one explicit program. Existing flows therefore decompose the problem: they fix a hardware configuration, map each operation in isolation, and then optimize reuse and fusion on what remains. Every prior co-design flow shares this sequential pipeline, and it is why the joint problem has stayed open: each stage commits before the global memory consequences of its choice are known, and the

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

commitment is irreversible downstream. The failure is concrete. A mapper that exports its fastest isolated tile can consume exactly the on-chip memory needed to keep a neighboring tensor resident, so the locally optimal tile forces a DRAM round trip and the complete schedule is slower. A slightly slower tile would have produced the faster program. No stage of the pipeline can see this trade-off, because it exists only in the joint space.

In this paper we introduce **CHEETAH**: **C**o-designed **H**ardware **E**xploration via **E**fficient **T**hroughput and **H**ybrid-search, a mathematically principled hardware–software co-design framework with LLM-guided search. CHEETAH is a new paradigm for co-design. Rather than adding one more stage to the sequential pipeline (Figure 1), it unifies in a single CHEETAH master linear program (LP) the decision spaces that prior hardware and software methods each cover only in part. Table 1 positions it against nine such systems, and Theorem 1 in Appendix B.2 proves that the lifted feasible set of FAST's fusion search embeds in CHEETAH's relaxation at the same objective, so the formulation subsumes that search rather than resembling it. Implementation choices determine work, traffic and memory. Time-indexed states track residency and reuse. Shared hardware variables allocate compute, vector, memory and bandwidth under area and power budgets, and McCormick envelopes and disaggregated epigraphs couple them. The program acts on a graph of logical invocations and typed storage objects, so convolutions, dense and expert GEMMs, attention, KV prefixes, expert weights and long skip connections enter one capacity and readiness accounting, and prefill and decode are the same program over different event sequences. Decode is where this matters most: under independent top-k routing with $n = 256$, $k = 8$, the expected number of experts touched grows from 8 at batch one to about 163 at batch 32, so batching alone does not make decode compute-bound and provisioning, batching and temporal residency must be chosen together.

Three properties of this problem defeat off-the-shelf solving. It does not fit in memory: Llama-1 13B's root alone is 10.1M columns by 18.3M rows, its full execution adds 520M rows, and stored densely it would occupy 51 PB, while the design space exceeds 10^{2300} branches, so the program must be generated on demand rather than built. Its structure is extreme: almost every row is a one-hot cover, an epigraph or a McCormick envelope, with 3.4 nonzeros per row at every scale and density below 0.01%, and rows outnumber columns three to one, so the optimal face is massively degenerate and cost tracks degeneracy rather than size (three times the rows costs six times the time). Its conditioning is hostile: coefficients span 10^{-6} to 10^{15}, a single reciprocal rate variable spans $80\times$ inside its own box, and standard first-order methods plateau at a 0.999 primal violation after 2,000 iterations and never descend, even with the chip pinned. A certificate therefore cannot be read off floating-point iterates. It has to be built in exact rational arithmetic. What makes the joint space tractable is that it never has to be materialized. CHEETAH solves the master LP by branch and bound over a declared family of whole-program branches using OCS [Feng and Boyd, 2026], our in-house operator solver. Each branch is a complete LP whose constraint map is applied rather than stored. Compact fits discover prices and primal candidates, and symbolic checks over the original domain certify a lower bound L_b for the branch. Branches that are infeasible or cannot beat the replayed incumbent are pruned, and $L = \min_b L_b$ with the incumbent U brackets the optimum. The 1,557-branch DeepSeek-V3 master LP closes to a certified 0.07% gap. Integral recovery and replay then turn the relaxation into a chip and a schedule, which are outputs distinct from the bound. An LLM architect reads the resulting resource prices and trial diagnostics to propose implementations and hardware neighborhoods in place of brute-force outer search. The hardware–software interactions themselves are captured in the inner formulation, not in the LLM.

Contributions. CHEETAH combines a unified HW/SW design relaxation, operator-based optimization and certified bounds with an LLM architect that proposes implementations and hardware search neighborhoods.

1. **CHEETAH formulation.** We introduce time-indexed LP variables $p^k_{i,t}$ and rematerialization variables $r_{i,t}$ to enable cheap recompute operations instead of paying DRAM reload costs, supporting skip connections and multi-fanout nodes. We unify tiling and fusion through **McCormick envelopes** and resource epigraphs. Binary tile–residency products are exact at integral choices; continuous hardware/service products retain their explicit relaxation. These couplings expose tiling–placement trade-offs that sequential pipelines can discard.

2. **Certified optimization and executable validation.** Compact numerical fits, rational cover reconstruction and original-domain checks certify the complete master LP rather than a

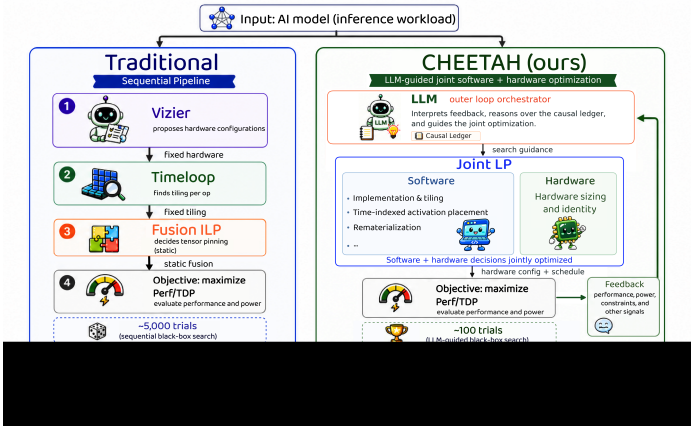

Figure 1: Heuristics based sequential design versus our unified optimization paradigm. (Left) Baseline methods decouple hardware search, operation-level tiling, and static fusion, precluding cross-stage trade-offs. (Right) CHEETAH presents a unified master linear program that jointly optimizes tiling, placement, and more while learning from a causal ledger.

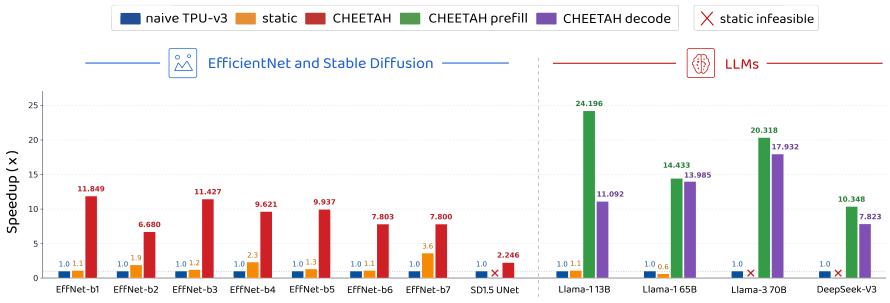

Figure 2: Speedup over the simulated naive TPU-v3 baseline, to be consistent with prior work [Zhang et al., 2022], for EfficientNet, SD1.5 UNet and LLM workloads. Vision workloads compare naive TPU-v3, static and CHEETAH. LLMs add separate CHEETAH prefill and decode results. Red crosses mark static infeasibility.

restricted support, and analytical replay validates a separately recovered design (Section 5). Global discrete optimality remains open.

3. **LLM Architect Outer Loop.** The architect interprets resource prices and a causal ledger of earlier trials to propose hardware and implementation changes. Deterministic checks admit proposals; the LLM itself supplies no certificate. This couples architecture exploration with the optimizer's diagnostics rather than relying only on black-box scores.
4. **Inference speedup.** Across 12 workloads, CHEETAH co-designs chips that outperform the naive TPU-v3 baseline by an order of magnitude on vision, LLM prefill and LLM decode (Figure 2, Section 6). These are analytical results under the TPU-v3 timing proxy, not measured serving gains, and the prefill and decode designs are different chips.

The following paper is organized as follows. Section 2 discusses related work, Section 3 summarizes preliminary background, Section 4 presents the master LP and its features, and Section 5 describes the OCS solver framework, its certificate and hardware search. Section 6 reports experimental results and ablations, and the appendix provides containment theorems, physical-validation details, co-design blueprints, and reproduction evidence.

2 Related Work

Accelerator mappers such as Timeloop, CoSA, and ZigZag explore tiling and mapping; FAST connects hardware search, mapping, and fusion in a staged flow [Parashar et al., 2019, Huang et al.,

2021, Mei et al., 2021, Zhang et al., 2022]. DOSA jointly differentiates hardware and mapping choices, while Gemini explores chiplet architectures and layer pipelines [Hong et al., 2023, Cai et al., 2024]. TileFlow and FADiff address fusion-aware dataflows and joint mapping–fusion optimization [Zheng et al., 2023, Jia et al., 2025]. CHEETAH couples these choices to temporal storage and shared hardware in an LP relaxation; this is a distinction in formulation, not a claim that prior work lacks joint optimization altogether.

First-order solver methods [Applegate et al., 2022] and McCormick envelopes [McCormick, 1976] supply the optimization building blocks. LLM proposal systems such as FunSearch and AlphaEvolve [Romera-Paredes et al., 2024, Novikov et al., 2025] inform the outer-loop role: propose changes, then evaluate them independently. Further comparisons appear in Appendix A.

3 Preliminaries

3.1 Linear Relaxation and Recovery

Discrete selections are relaxed to $[0, 1]$ and bounded products to convex envelopes (Appendix B), so the LP optimum lower-bounds only the domain the formulation and its certificate cover. Recovery then chooses integral hardware and implementations and checks dependencies, storage, transfers and predicted latency (Appendix C: C.1, C.3).

3.2 Capability Comparison

Table 1 compares decision spaces, rather than claiming that every listed capability has a measured gain. CHEETAH's row describes the master LP. Section 6.4 identifies the subset exercised by the full-inference execution. The table distinguishes searched decisions from restricted or user-specified support. DeFiNES chooses caching versus overlap recomputation, and Fast-and-Fusiest searches dependency-respecting orders within its fused mapspace. Gemini's fusion and order support concerns layer pipelines. Pluto optimizes affine schedules, and Tiramisu also exposes explicit storage, copies and redundant computation through its scheduling language. These differ from unrestricted cost-driven temporal placement. CoSA's guarantee concerns a per-layer proxy objective and Fast-and-Fusiest's its fixed-hardware mapspace, but neither is a bound over a joint hardware–schedule domain. The last column denotes one joint mathematical program rather than staged co-search, and DOSA's qualifying program is nonconvex. The comparison is not only descriptive: Theorem 1 in Appendix B.2 exhibits an injective, objective-preserving embedding of the lifted FAST fusion set into our relaxation, so by Corollary 2 our bound is never worse than FAST's matched objective and any fixed-hardware gap must come from the extra degrees of freedom, from rounding, or from cost-model and baseline conventions rather than from FAST points lying outside the search space.

4 CHEETAH Formulation for HW/SW Co-Design

CHEETAH minimizes predicted workload inference latency under a declared hardware budget. Its inputs are independent logical invocations, a finite implementation library, and a numerical contract. Each whole-program branch specifies compatible hardware and execution semantics; hardware is shared across all invocations within that branch. The same master LP template admits the retained joint tiling–placement model and causal execution branches. Variable-hardware branches retain their declared physical rows and resource envelopes; fixed-chip wrappers additionally impose exact installed-rate ceilings. The certified DeepSeek instance is the frozen 1,557-branch expanded family (Appendix M.3); changing that family or its coefficients requires a new certificate.

4.1 Pruning, Variables, and the Encoded Features

Pareto pruning pre-pass. For each operation we discard infeasible tiles and configurations dominated in every cost, traffic, workspace and interface dimension within the same implementation contract and hardware class. Timeloop or a native analytical recipe supplies the retained coefficients once (Figure 7), so the solver never calls the mapper during the search. Appendix B.4 states when the dominance relation survives a change of hardware.

Table 1: Decision spaces in the cited systems. ✓: formulated; ○: restricted or user-specified; ×: not in the cited method. Guarantees retain their own objective and domain. CHEETAH's row demonstrates our unified formulation scope.

	Schedule decisions					Search		Guarantee	
System	Per-op tiling	Fusion	Temporal placement	Remat.	Op. order	Mapping	HW sizing	Opt. guarantee	Joint HW/SW program
Per-op mappers									
Timeloop [Parashar et al., 2019]	✓	×	×	×	×	✓	×	×	×
CoSA [Huang et al., 2021]	✓	×	×	×	×	✓	×	○	×
Fusion-aware mappers									
DeFiNES [Mei et al., 2023]	✓	✓	○	○	○	✓	×	×	×
Fast-and-Fusiest [Andrulis et al., 2026]	✓	✓	○	×	✓	✓	×	○	×
Hardware design-space exploration									
ZigZag [Mei et al., 2021]	✓	×	×	×	×	✓	✓	×	×
DOSA [Hong et al., 2023]	✓	×	×	×	×	✓	✓	×	✓
Gemini [Cai et al., 2024]	✓	○	○	×	○	✓	✓	×	×
CODEBench [Tuli et al., 2023]	○	×	×	×	×	×	✓	×	×
FAST [Zhang et al., 2022]	✓	✓	○	×	×	✓	✓	×	×
Loop compilers									
Pluto [Bondhugula et al., 2008]	○	✓	×	×	✓	○	×	×	×
Tiramisu [Baghdadi et al., 2019]	○	○	○	○	○	○	×	×	×
CHEETAH (our framework)	✓	✓	✓	✓	✓	✓	✓	○	✓

Table 2: Master variables. Unscaled selection/state masses lie in $[0, 1]$; branch coordinates are scaled by λ_b.

Variable	Indices	Meaning
h_b	shared chip	Compute/vector resources, capacity and service rates; fixed to $\lambda_b \bar{h}_b$ only in fixed-chip wrappers
λ_b, x_b	branch b	Whole-program weight and scaled branch coordinates
y_k	implementation k	Tile, mapping, batch or fusion selection
$s_{ke}, w_{ke}, f_{ke}, a_{ke}$	implementation, event	Start, work, finish and active mass
D_{ie}, β_{je}	invocation/object, event	Logical completion and generated data
$p_{j\tau e}, v_{j\tau e}$	object, tier, event	Resident state and eviction
$q_{j,\tau\to\tau',e}$	object, link, event	Paid transfer and destination reservation
$\eta_{je}, \rho_{je}, \delta_{je}$	parameter, event	Staging occupancy, receive and drain
A_e, χ_e	event	Total active mass and array-workspace product
Q^r_e, u_r, Z^r_e, d_e	resource, event	Demand, inverse rate, service and nonnegative event duration
$T_i, p_{et}, p^W_{it}, r_{it}, w_{ick}$	retained schedule block	Latency, residency, recomputation and tile–residency products

Let $i \in \mathcal{N}$ index logical nodes, $j \in \mathcal{J}$ storage objects, $e \in \mathcal{E}_b = \{0, \dots, E_b - 1\}$ events in branch b, and τ memory tiers. An implementation k covers members S_k and specifies inputs, outputs, workspace $W_{k\tau}$ and resource demands. $\mathcal{K}_b$ is the finite implementation set of branch b, generated on demand. Table 2 defines the coordinates.

The family $\mathcal{B} = \mathcal{B}_C \dot\cup \mathcal{B}_P$ holds causal branches and, when included, retained joint tiling–placement branches. The objective charges causal event durations and retained operation latencies plus rematerialization, with κ_b, α_b converting to ms. Each x_b holds the branch coordinates already scaled by λ_b, and right-hand sides and bounds scale likewise. Below we suppress branch indices and write $E = E_b$. $\mathcal{R}_k$ lists required object/tier pairs, $\mathcal{D}_k$ completed controls, D^r_e linear resource demand, and $\mathsf{Cap}_{b,\tau e}$ live objects, incoming copies, active outputs, workspace and reservations. B_b, J_b hold the remaining state, backing and implementation rows (Appendix L: L.1, L.2, L.3, L.4, L.5, L.6 and Table 37). For $b \in \mathcal{B}_{\text{phys}} \subseteq \mathcal{B}_C$ the installed chip $\bar{h}_b$ fixes rates $\bar{R}_{br}$ and $\mathsf{D}^{r,\text{phys}}_{be}$ is the issued work or traffic on resource r at event e, while A^λ_b and P^λ_b denote affine area and peak power.

The cover, lifecycle, service and B_b, J_b rows apply to causal branches, and A^P_b, G^P_b to retained schedule branches. $\mathcal{W}, \mathcal{R}, \mathcal{M}$ are linear workspace-product, reciprocal and service-product envelopes.

$$
\begin{aligned}
&\min_{\lambda,\{x_b\}} && \text{objective} && \sum_{b\in\mathcal{B}_C}\kappa_b\sum_{e\in\mathcal{E}_b}d_{be}+\sum_{b\in\mathcal{B}_P}\alpha_b\left(\sum_i T_{bi}+\sum_{i,t}c^{\text{remat}}_{bi}r_{bit}\right)\\
&\text{s.t.} && \text{constraint 1} && \lambda_b\ge 0,\quad \sum_b\lambda_b=1,\\
& && \text{constraint 2} && \sum_{k\in\mathcal{K}_b:i\in S_k} y_k=\lambda_b,\\
& && \text{constraint 3} && \sum_e s_{ke}=\sum_e w_{ke}=\sum_e f_{ke}=y_k,\\
& && \text{constraint 4} && a_{ke}=a_{k,e-1}+s_{ke}-f_{k,e-1},\quad w_{ke}=f_{ke},\quad 0\le w_{ke},f_{ke}\le a_{ke}\le y_k,\\
& && \text{constraint 5} && a_{ke}\le p_{j\tau e}-v_{j\tau e}\ ((j,\tau)\in\mathcal{R}_k),\qquad a_{ke}\le D_{ie}\ (i\in\mathcal{D}_k),\\
& && \text{constraint 6} && D_{i,e+1}=D_{ie}+\sum_{k:i\in S_k} f_{ke},\quad D_{i0}=0,\ D_{iE}=\lambda_b,\\
& && \text{constraint 7} && 0\le v_{j\tau e}\le p_{j\tau e}\le\beta_{je},\qquad 0\le q_{j,\tau\to\tau',e}\le p_{j\tau e}-v_{j\tau e},\\
& && \text{constraint 8} && A_e=\sum_k a_{ke}\le\lambda_b,\qquad (A_e,h_{b,\text{arrays}},\chi_e)\in\mathcal{W}_b^{\lambda_b},\\
& && \text{constraint 9} && \mathsf{Cap}_{b,\tau e}(p,v,q,a,\chi,\eta,\rho,\delta)\le C_\tau^{\lambda}(h_b),\\
& && \text{constraint 10} && Q_e^r=\mathsf{D}_e^r(q,w,\rho,\delta),\quad (h_{br},u_r)\in\mathcal{R}_b^{\lambda_b},\quad (Q_e^r,u_r,Z_e^r)\in\mathcal{M}_{br}^{\lambda_b},\\
& && \text{constraint 11} && 0\le Z_e^r\le d_e,\qquad \mathsf{S}_{be}^v(q,w,\rho,\delta)\le d_e\quad (\text{fixed-rate services } v),\\
& && \text{constraint 12} && h_b=\lambda_b\bar{h}_b,\quad \mathsf{D}_{be}^{r,\text{phys}}\le\bar{R}_{br}d_e\quad (b\in\mathcal{B}_{\text{phys}},\ r\in\mathcal{R}_{\text{phys}}),\\
& && \text{constraint 13} && \mathsf{A}_b^{\lambda}\le 600\lambda_b,\quad \mathsf{P}_b^{\lambda}\le 300\lambda_b,\quad \mathsf{P}_b^{\lambda}\le 2\mathsf{A}_b^{\lambda}\quad (b\in\mathcal{B}_{\text{phys}}),\\
& && \text{constraint 14} && F_bh_b\le g_b\lambda_b,\quad B_bx_b=b_b\lambda_b,\quad J_bx_b\le j_b\lambda_b,\\
& && \text{constraint 15} && A_b^Px_b=a_b^P\lambda_b,\quad G_b^Px_b\le g_b^P\lambda_b\quad (\text{retained schedule branches}),\\
& && \text{constraint 16} && \ell_{b\nu}\lambda_b\le x_{b\nu}\le u_{b\nu}\lambda_b\quad (\text{each finite original bound}).
\end{aligned}
$$

(CHEETAH master LP)

Copy, conversion, KV restoration and permitted repeat costs enter the shared service demands and hence d_e, so the event epigraph already represents their overlap. The objective minimizes latency over the declared task boundary, not area, energy or Perf/TDP. Setting $\lambda_b = 1$ recovers that branch exactly (Corollary 3), fractional weights describe the LP relaxation, and recovery selects one program.

4.2 Features and Their Mathematical Encoding

Joint tiling and fusion [objective; constraints 2, 15]**.** For an isolated operation, the cover row reduces to $\sum_c y_{i,c} = 1$. A tile's traffic savings depend on residency. In the retained schedule block, w_{ick} represents $y_{ic}p_{\text{assoc}(i,k),o(i)}$ through

$$0 \le w_{ick}, \quad w_{ick} \le y_{ic}, \quad w_{ick} \le p, \quad w_{ick} \ge y_{ic} + p - 1. \tag{2}$$

By Proposition 1 these McCormick inequalities give the convex hull of one bounded bilinear graph and fix the product when either binary-domain factor is integral [McCormick, 1976]; they do not make every fractional solution executable, and Appendix B.1 gives a fractional point satisfying each separate envelope while violating the exact-product identity. Tile-dependent footprints use the same construction. The displayed unit-bound rows are for a branch at weight one; homogenization replaces 1 by λ_b. Native fused atoms carry their complete input, output, workspace and service recipe, so fusion cannot silently remove work. Appendix I shows two solved instances (I.1, I.2) in which a locally slower tile yields the faster schedule.

Shared hardware and resource epigraphs [objective; constraints 10, 11, 14, 16]**.** Compute, vector lanes, SRAM, and delivered memory service are shared decisions under linear area/power proxies $F_bh_b \le g_b$. Finite implementation or technology menus use disaggregated copies with linking equalities and scaled bounds. Resource epigraphs charge the maximum of concurrent service demands, while demands on the same port add; Corollary 1 shows the disaggregated perspective

form dominates the aggregated maximum-of-sums alternative. Native rows aggregate work and transfer demands per event before applying the service envelopes. Arbitrary executable overlap is not inferred from fractional sharing; native replay uses its declared inter-kernel ordering and internal panel pipelines.

Hard rooflines and physical budgets [constraints 12, 13, 14]**.** The strengthened fixed-chip branches enforce resource ceilings *inside the LP*: $\sum_k F_k^{\mathrm{issued}} w_{ke} \le \bar{C}_b d_e$ for matrix work, and $\mathsf{D}_{be}^{r,\mathrm{phys}} \le \bar{R}_{br} d_e$ for shared HBM, host, vector, DMA-staging and SRAM-bank-port service. Padded issued work bounds useful work; simultaneous kernels and copies share the same supply. Summing over events therefore gives MFU and accounted MBU at most one. Affine installed-component costs enforce area $\le 600\,\mathrm{mm}^2$, peak power $\le 300\,\mathrm{W}$ and chip-average density $\le 2\,\mathrm{W/mm}^2$, without discounting peak power by MFU. The same identified template supplies combined host/HBM pin, controller, whole-bank capacity/count/aspect, logic-depth and NoC-size constraints in $F_b h_b \le g_b \lambda_b$. Fixed rates keep these ceilings linear; a variable-rate McCormick envelope alone cannot enforce the exact physical product. These are hard limits of the declared analytical model, with the same predicates used in recovery. Aggregate bank and NoC proxies do not prove conflict-free arbitrary placement. Appendix L.5 gives the rows and distinguishes their current implementation from the frozen certificate.

Reciprocal and product envelopes [constraints 8, 10]**.** For a rate $r \in [\ell, h]$, $\mathcal{R}$ bounds $u \approx 1/r$ by tangents $u \ge 2/a - r/a^2$ and the global secant $u \le (\ell + h - r)/(\ell h)$. $\mathcal{M}$ bounds $Z \approx Qu$ by the four McCormick inequalities on $[0, \bar{Q}] \times [1/h, 1/\ell]$. All coefficients are constants, so the program is linear. Outward coefficient rounding preserves graph containment. These are relaxations of physical service; replay reinstates the exact rates and products. Selecting a discrete bandwidth fixes that rate, but continuous-rate envelopes remain relaxations even when implementation selectors are integral.

Time-indexed placement and KV across steps [constraints 7, 9]**.** Residency $p_{j\tau e}$ changes with events rather than being fixed for the whole run. KV production creates immutable append objects; each decode attention operation depends on its required prefix. The state rows preserve earlier entries while the prefix grows. Capacity accounts for resident data, incoming copies, and active workspace simultaneously. Long skip connections and multi-fanout tensors use the same object identities and consumer readiness.

Spill, restoration, and rematerialization [objective; constraints 7, 14]**.** Copies retain a valid source, reserve their destination, consume service, and become readable only after completion. Boundary rows require the declared terminal backing. A permitted repeat executes the original producer again and pays its work, input and workspace demands; it restores storage without advancing primary logical completion twice. Keeping full KV history in a slower tier is therefore different from deleting history. The native results credit paid KV restoration and retention; no native recomputation gain or lossy KV budget is claimed.

Weight residency and expert supply [constraints 5, 9]**.** A weight tensor is an immutable object with its own residency trajectory $p_{j\tau e}$, so retaining it across uses and paying a reload are different points of the same rows rather than separate models. First loads and every kernel read stay charged; only repeated imports of a retained tensor disappear. Capacity couples this to activation placement through the shared $\mathsf{Cap}_{b,\tau e}$ row, so weight retention competes with KV and live activations for the same tier. Expert weights use the identical object identities under the declared routing model: which experts are resident versus fetched is a residency choice subject to storage capacity, and a retained expert still pays its first import and its per-use reads.

Implementation menus and recovery [constraints 1, 14]**.** A batch column covers several compatible invocations and loads shared parameters once where its recipe permits. Cover constraints retain every member; start/finish and readiness rows prevent using future tokens or expert selections before their router dependencies complete. Event placement can vary within the finite library. The current native execution uses a recovered serial schedule; a larger scheduling domain does not imply that its optimum has been found.

Implementation menus and recovery [constraints 1, 14]**.** Parameter staging has explicit receive, occupancy and drain states. Full-K panels and FP8-to-BF16 receiver implementations pay buffer, port and conversion costs. Selected-reader backing keeps the raw or converted representation needed by each consumer; it cannot discard both. These rows connect hardware provisioning to software reuse. Precision, sparsity, array aspect, memory technology, and validated overlap policies enter as implementation choices with their own numerical and resource contracts. Accuracy-changing cache eviction or approximate reuse needs separate quality evidence; it is disabled in the full-input result. Larger cross-GEMM fusion and unrestricted overlap have no credited gain there. Recovery pins hardware and mappings, schedules transfers, and replays the original obligations. The resulting chip and schedule have a predicted latency under the analytical model. Section 5 distinguishes hardware branch-and-bound, continuous LP certification, and integral recovery.

Serving and phase choice [constraint 6]**.** Prefill and decode are the same master over different event sequences and boundary rows, not two models. A decode step inherits the KV prefix its attention operations require, and the declared task boundary fixes which preparation and loading costs the objective charges. Phase latencies are therefore comparable only under one declared boundary, and the selected chip is shared across every phase represented in a branch.

4.3 LLM Architect Diagnostic Outer Search

The LLM architect reads a ledger of resource prices, slacks, invalid trials, and replay outcomes, then proposes candidate implementations or search neighborhoods. Deterministic validation checks each proposal before evaluation. The architect is conditioned on a *deployment profile*, a natural-language statement of what the deployment values, because the right chip depends on it: an overnight financial-analysis workload values sustained throughput and memory bandwidth and tolerates latency and power, whereas streaming speech inference values single-request latency at small batch and moderate power. The profile sets the objective mode and steers which implementations and hardware regions the architect proposes; Appendix H.1 runs both profiles and reports the resulting best chips (Table 19) and invalid-trial rates (Table 20). The outer loop uses Vizier [Golovin et al., 2017] to search hardware neighborhoods; the master also represents hardware choices directly. Dual prices guide search, but an LLM proposal is not a complete pricing oracle: a full-domain optimality claim requires checking all omitted alternatives. The controller and prompts are documented in Appendix H: the prompts in Appendix H.2 and the price and validation signals in Appendix H.3.

5 Certificate-Producing Optimization and Design Recovery

To solve the master LP we use OCS [Feng and Boyd, 2026], our in-house solver framework. OCS is a standalone contribution whose numerical design is described in a companion paper, with the implementation released at the cited repository and its kernel and closing stage in Appendix M.2. Here we give only what the results require. OCS never forms the event-expanded matrix. It applies the constraint map Kx and its adjoint $K^T y$ as operators, runs an anchored, restarted primal–dual scheme with a diagonal variable metric on working supports, and then replaces the converged iterates with compact fits whose rational reconstruction is verified against every original row. The resulting bound is a certificate rather than a solver tolerance: the primal satisfies the complete original branch and the dual has valid signs and Lagrangian box support over all coordinates.

The master LP is solved by branch and bound over its declared family. Each branch b receives a certified lower bound L_b, branches with checked fixed-cost infeasibility or bounds that cannot improve the incumbent are pruned, and $L = \min_b L_b$ with the incumbent U brackets the optimum. For the frozen 1,557-branch DeepSeek master LP, composing the 1,787 per-branch certificates gives $L \approx 946{,}487.72$ ms and $U \approx 947{,}123.74$ ms, a relative gap of 0.07%, with 1,095 branches at the stopping threshold, 462 pruned and none unresolved (Appendix M.3).

Hardware branch and bound and recovery. Integral recovery searches a finite tree over array counts, vector lanes, DRAM bandwidth, host channels and local-service units; optimistic resource and capacity-priced retention bounds prune regions that are budget-infeasible or cannot improve a replayed incumbent, and leaves use actual implementation and capacity costs. Recovery then selects one branch, integral implementations, hardware and storage policies, and analytical replay checks dependency readiness, capacities and service at the selected rates. The retained execution

takes $2{,}718{,}970.508466$ ms; the continuous upper above is not its latency. Reciprocal/product relaxation and integral recovery gaps remain, so the continuous certificate does not establish discrete chip/schedule optimality (Appendix M.1).

6 Experiments

We evaluate predicted latency, search cost, and recovered schedules. Studies include prefill/decode and fixed-hardware mapping. For example, the full-input DeepSeek case adds autoregressive decode and phase-specific controls. TPU-v3 provides a publicly documented reference baseline consistent with prior work [Jouppi et al., 2021, Zhang et al., 2022]. All speedups below are analytical and modeled results.

What a new paradigm should be judged on. A sequential pipeline returns a design. CHEETAH returns a design *and* a certified bound *and* a gate record: every reported chip comes with an LP lower bound on any design in its declared family, passes six pre-RTL feasibility gates whose constants carry an audited provenance trail, and sits in a decision space whose coverage is provable (Table 1, Theorem 1). Raw speedup over a fixed baseline is the old paradigm's criterion. The results below should be read against these three as well.

Software alone is physics-capped; co-design is the only route past it. Our first result is a ceiling, not a speedup. For a fixed chip, no scheduler of any kind can beat the ratio of the naive streaming time to the compute-or-memory roofline floor (Theorem 2, Appendix B.3). On fixed TPU-v3 that ceiling is $2.14\times$, $2.14\times$, $2.51\times$ and $2.22\times$ for Llama-1 13B, Llama-1 65B, Llama-3 70B and DeepSeek-V3, and CHEETAH's fixed-hardware solutions reach exactly 100% of it on all four (Table 5; three of the four also appear in Table 4). The modest fixed-hardware gains are therefore not a weakness of the method but the physics of the chip: software can recover at most about $2.5\times$ on TPU-v3, so the only route to the 10–$25\times$ reported below is to co-design the chip, which is what the unified formulation enables and what a sequential pipeline cannot do. The two-factor decomposition of Appendix F.1 makes the split explicit for the prefill-oriented designs: of Llama-3 70B's $40.1\times$, $1.48\times$ comes from scheduling at fixed TPU-v3 silicon and $26.85\times$ from MAC provisioning, and the reconstructed product matches the headline to within 1% on all four LLMs, so the gain is not an artifact of simply adding compute.

6.1 Experimental Setup

The suite comprises EfficientNet-B1–B7, Llama-1 13B/65B, Llama-3 70B, DeepSeek-V3 [Tan and Le, 2019, Grattafiori et al., 2024, DeepSeek-AI, 2024] and Stable Diffusion 1.5 UNet. DeepSeek-V3 is evaluated on thirty-two 4,096-token Wikitext-2 prompts, each decoded greedily for 512 tokens, with FP8 weights dequantized to BF16. We compare static pinning, fixed-tile temporal placement and joint tiling under one cost convention. These are restricted controls of CHEETAH, not separate systems. Small programs are solved directly with HiGHS [Huangfu and Hall, 2018], and the native programs with OCS (Section 5), whose closing fits use HiGHS as the inner LP engine. Hardware is shared across each recovered execution with independent implementation choices, and the full-input experiment binds actual checkpoint routes and preserves every logical invocation (Appendix M). All ratios use the naive TPU-v3 reference with its original per-operation mapper rescaling, streaming sum and timing/precision proxy. The recovered decoder preserves BF16 operands, ordered FP32 arithmetic, routing and paid entry obligations, and standalone nonmatrix service outside the phase boundary is not silently included.

Physical Design Validation. Since modeling simulations ultimately present a gap with real world physical performance, we validate all results through a five-level audit process (Appendix E.1): (i) Roofline Auditor, (ii) Campaign Validator, (iii) Causal Ledger, (iv) solver diagnostics, and (v) six pre-RTL gates. Each CHEETAH design must pass all five audit tiers before it is deemed feasible and reported. Appendix E.3 states the six gates and their thresholds, Appendix N records where every gate constant and cost coefficient comes from, and Appendix J gives the complete blueprints of the designs that pass them, so each reported chip can be audited from its headline speedup down to the provenance of each constant it was checked against.

Table 3: Six architectural feasibility gates.

Gate	Check	Threshold
Pin count	HBM3-class signal pins at 0.8 GB/s/pin	$\leq 4{,}096$
Reticle area	Linear area model at 7 nm	≤ 600 mm^2
Logic depth	MAC-tree combinational depth	≤ 35 levels
NoC routing	$n_{\text{PEs}}\, n_{\text{banks}}$	$\leq 5 \times 10^6$ cells
Power density	Dynamic power / die area	≤ 2.0 W/mm^2
SRAM macros	64 KiB banks, aspect ratio 1:4–4:1	$\leq 4{,}096$ banks

The roofline check rejects costs below compute or memory service floors. Table 3 lists the architectural screening thresholds. These are modeling checks, not fabricated-chip timing or packaging sign-off. RTL and synthesis evidence is in Appendix D: BF16/FP32 decoder tests (D.1), schedule-tape and INT8 datapath checks (D.2), post-synthesis area (D.3) and measured energy coefficients (D.4).

Reported efficiency quantities. MFU, per-resource MBU and Perf/TDP are read off the scheduled execution, never assumed; Appendix E.4 defines each, states the fixed-chip rows that guarantee $\text{MFU} \leq 1$ by construction, and gives the conditions under which Perf/TDP is a valid comparison.

6.2 Workload Results and Fixed-Hardware Controls

The prefill-oriented designs report an average $22.07\times$ predicted speedup over the simulated TPU-v3 baseline, reaching $40.1\times$ on Llama-3 70B, under their own workload and traffic conventions. Appendix J shows the concrete hardware blueprints.

Table 4: Fixed-hardware software-search speedups (joint LP versus streaming) on simulated TPU-v3 and a compact NPU.

Workload	TPU-v3	Compact NPU
EfficientNet-B7	$2.21\times$	$20.01\times$
SD1.5 UNet	$1.50\times$	$5.89\times$
Llama-1 13B	$2.14\times$	$31.39\times$
Llama-3 70B	$2.51\times$	$7.66\times$
DeepSeek-V3	$2.22\times$	$7.76\times$

Table 4 reports the fixed-hardware software-search comparisons for five workloads on the simulated TPU-v3 and compact NPU. Each entry keeps its matrix graph, full implementation menu, memory policy, timing library and streaming-reference formula. The TPU LLM producer and the vision/compact producer use different LP conventions, including the capacity treatment noted in the table. The stored precision proxies are retained; these results do not constitute matched native INT8/BF16 experiments or recovered full-decode executions on either chip.

6.3 Search Cost and LLM Guidance

Cached implementation coefficients remove repeated mapper calls from the inner search. In the 50-trial comparison, joint tiling reduces Llama-70B search time from 58.42 to 1.17 hours and DeepSeek-V3 from 69.39 to 2.17 hours relative to the fixed-tile temporal pipeline (Figure 4; Appendix G gives the hardware search space in G.1 and Appendix C.2 the LP sizes). These are search times, not inference speedups. LLM guidance reduces mean invalid trials from 24.5% to 5.9% while reaching the same best observed Perf/TDP score as its full-space Vizier control (Figure 6); Appendix H.1 repeats the search under a throughput-oriented and a latency-oriented deployment profile. This tests proposal efficiency, not proof of an optimal architecture or causal validity of every dual suggestion. Whether the LLM outperforms a rule-based controller driven by the same duals and failure classes is not established here; we therefore present the architect as a search convenience and let the formulation and its certificates carry the contribution.

Performance characterization. Figure 3 shows the contribution of each CHEETAH component on the selected chips. We start from the naive TPU-v3 baseline and incrementally add components, so each slice is one cumulative increment and the four slices of a panel sum to the workload's total gain. For the Llamas the sequence first extends the compatible mapping menu, then restores active compute arrays, off-chip DRAM bandwidth and on-chip delivery bandwidth, each with weight and KV residency, paid first imports and installed bank/port reservations held fixed; all eight combinations of the three resource limits are repriced, and the six release orders move individual shares but not

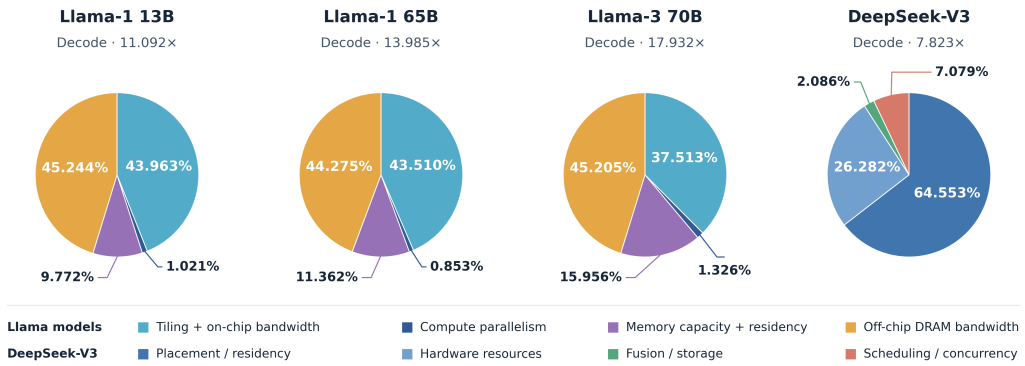

Figure 3: **Performance breakdown of each CHEETAH component**, relative to a simulated naive TPU-v3 baseline [Zhang et al., 2022]. Improvements are additive cumulative increments in $S = T_{\text{naive}}/T$ and sum to 100% of each workload's gain above $1\times$; attribution is order-dependent.

their combined contribution. Off-chip bandwidth is the dominant component at 44–45% on every Llama, with on-chip delivery at 38–44%; we group delivery with tiling because a wider mapping menu is only usable once arrays and ports can serve it, and the isolated mapping increments alone account for just 0.18–0.51%, since a better tiling buys nothing while the execution is service-bound. Memory capacity with weight/KV residency contributes 10–16% and compute parallelism about 1%: decode on these chips is bandwidth-bound, not compute-bound, so adding arrays without delivery yields almost nothing. DeepSeek-V3 reverses the picture. Placement and residency contribute 64.6% and hardware refinement 26.3%, while producer-layout storage fusion (2.1%) and scheduling, paid cross-wave prefetch with a second FP32 accumulator context (7.1%), matter only after the memory bottleneck is removed. As in FAST [Zhang et al., 2022], the components act in synergy: residency and bandwidth remove the memory wall, and only then do tiling, fusion and scheduling realize their utilization gains, which is why co-optimizing the hardware with the software that uses it recovers gains no single stage exposes. These are analytical service-cap controls on the selected designs: a capped control claims no smaller chip or power saving, and the fixed-hardware ratios of Table 4 are not subtracted from these shares.

Hardware–software interaction. Hardware and software gains are not independent. Holding either fixed and releasing the other recovers most of its benefit, but the combined saving is slightly less than the sum: a wider delivery path removes some of the traffic that better placement would otherwise have saved, so the two must be co-selected rather than optimized in sequence. Adding on-chip delivery and SRAM is what lets storage fusion keep intermediates on chip and lets a wider mapping menu be used at all; a larger menu buys nothing while the execution is service-bound. The two model families pull the chip in different directions. DeepSeek-V3's gains come overwhelmingly from placement and residency, so its design favors a deep memory hierarchy, with a large backing tier and capacity for resident experts and KV, over raw delivery; the dense Llamas are bandwidth-bound, so their designs favor off-chip and on-chip delivery and gain least from added arrays. Within the Llama family the balance shifts with scale: as parameters grow, keeping weights and KV resident accounts for a growing share of the gain, so larger models favor capacity over delivery. Table 38 (Appendix L.6) records which decision families each decoder design actually exercises. These finite-family controls do not certify unrestricted placement, rematerialization or scheduling optima.

6.4 Full-Inference DeepSeek Comparison

We evaluate DeepSeek-V3 on 32 independent requests of 4,096 prompt and 512 output tokens: prefill produces the first output and 511 decode forwards follow, with captured expert IDs and routing weights defining all $61{,}985{,}244$ invocations and the complete KV history preserved. The recovered chip has 320 16×16 MAC arrays, 5,248 vector lanes, 1,024 FP8 receiver lanes, 78.25 MiB SRAM, 64 GiB DRAM and 2 TiB backing, delivering 1,630 GB/s DRAM, 640 GB/s external and 20,480 GB/s local service at a 1 ns analytical clock; it passes the area, peak-power, density, pin, bank, depth and NoC gates at 321.69 mm^2 and 298.69 W. The reference is a vanilla simulated TPU-v3 with

published parameters, ordinary compiler behaviour and operands supplied from HBM at 900 GB/s; its interface assumptions are explicit parts of the prediction, not measurements of a calibrated serving stack.

Phase times add and speedup is the ratio of total predicted latencies. The recovered design predicts $\mathbf{5.25}\times$ on the complete task (15,546 s to 2,963 s), from $3.57\times$ on prefill, $6.08\times$ on decode and $2.01\times$ on conversion; decode contributes 80.45% of the time saved. Data supply is 14.22% of the reference total, so the result is largely insensitive to the offload assumption. The expanded master behind this design is certified to 0.067154% (Appendix M.3); neither that interval nor the replayed latency is a discrete-design optimality guarantee.

6.5 Feature Contributions on the Same Chip

The controls retain the same chip and remove one mechanism at a time on the same numerical workload. These are matched execution controls, not globally optimized feature-off designs. Interacting effects are not an additive attribution of the headline speedup.

Batching and storage fusion. The no-batch control executes every original member separately and reselects its tile. Resident weights remain resident; each streamed member pays its own receive. The control expands to exactly 61,985,244 kernels and takes 2,999.86 seconds, versus 2,100.62 with batching. Thus grouped execution and shared streamed-weight delivery give $1.61\times$ decode and $1.06\times$ prefill gains. SwiGLU and elementwise storage choices remove primitive spills while retaining arithmetic, BF16 rounding, and local/vector service. Full-K panels reuse weights across output-row tiles, paying SRAM and serialized panel waves. Holding one tile fixed per parameter family adds 0.87 seconds to decode; panel boundaries, buffers and residency stay fixed. This tests weighted decode tile adaptation, rather than every temporal choice in the master.

KV and time-indexed placement. Every arm retains complete cached decoding, growth, host backing and mandatory writes. Optional DRAM residency selects 246 request/layer/K-or-V prefix chains and pays 125,952 copies. A fixed whole-task partition fits 193 chains; releasing prefill temporaries before decode permits 246. Placement's 3.70-second gain is included in the 15.73-second KV gain. These modest marginal gains reflect limited DRAM capacity and the wide custom external interface. Independent checks replay changed copies and kernel traffic; Appendix M records exact inputs, costs, controls and their scope.

7 Conclusion

CHEETAH presents a new paradigm for hardware–software co-design: a single master LP relaxation that unifies hardware sizing, implementation and tiling, fusion, time-indexed placement, weight and KV residency, rematerialization and scheduling, decision spaces that prior methods cover only in part (Table 1), with a containment theorem showing that it subsumes rather than approximates the fusion search it generalizes. Branch and bound over the declared family, compact numerical fits and complete-domain certificates connect the relaxation to concrete designs. The frozen 1,557-branch DeepSeek master LP is solved to a certified 0.07% gap, and reported chips are screened by six pre-RTL feasibility gates whose constants carry an audited provenance trail. We believe this is a promising framework, and much work remains (Appendix K). The reported speedups are predictions of an analytical model, not measured silicon, and the recovered chips are replayed designs rather than certified discrete optima. Cross-GEMM fusion, unrestricted overlap and the LLM architect's advantage over simpler controllers are not yet established, and calibrating the cost model against fabricated hardware and closing the gap to discrete optimality are open directions we hope this formulation makes tractable.

Acknowledgments. We gratefully acknowledge support from federal sources: NSF under No. 24-554 (AIMing) and DARPA under No. HR00112520038 (Fallingwater). We also gratefully acknowledge support from Stanford HAI, IBM (Code Generation), Lightspeed, Felicis, and Google. This work was also partially supported by IBM and Felicis as an affiliate member of Stanford Institute for Human-centered Artificial Intelligence (HAI)'s industry program.

This work was supported in part by the National Science Foundation (NSF) CAREER Award under Grant CCF-2236829, in part by the NSF under Grant CNS-2551282, in part by the National Institutes

of Health under Grant 1R01AG08950901A1, in part by the Office of Naval Research under Grant N00014-24-1-2164, and in part by the Defense Advanced Research Projects Agency under Grant HR00112490441.

Miria Feng was supported in part by the Stanford Graduate Fellowship. We also thank Lucy Woof for her continuous support throughout, Dan Zhang for his invaluable guidance, Kevin Nam for feedback on an early draft and analysis, and ZhongWei Dang on RTL and physical design conversations.

References

T. Ajayi, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Lee, U. Mallappa, M. Neseem, et al. Toward an open-source digital flow: First learnings from the OpenROAD project. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*, 2019.

T. Andrulis, M. Gilbert, V. Sze, and J. S. Emer. Fast and fusiest: An optimal fusion-aware mapper for accelerator modeling and evaluation. *arXiv preprint arXiv:2602.15166*, 2026.

D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue, and W. Schudy. Practical large-scale linear programming using primal-dual hybrid gradient, 2022. URL https://arxiv.org/abs/2106.04756.

R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–205, 2019.

U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 101–113, 2008.

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax.

J. Cai, Z. Wu, S. Peng, Y. Wei, Z. Tan, G. Shi, M. Gao, and K. Ma. Gemini: Mapping and architecture co-exploration for large-scale DNN chiplet accelerators. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024. arXiv:2312.16436.

A. Chambolle and T. Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of mathematical imaging and vision*, 40(1):120–145, 2011.

L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric. ASAP7: A 7-nm FinFET predictive process design kit. In *Microelectronics Journal*, volume 53, pages 105–115, 2016.

Computing Research Association. How AI will reshape computer systems by 2035: A Jeffersonian dinner in San Francisco about our 10,000× future. CRA Industry Blog, Apr. 2026. URL https://cra.org/industry/2026/04/27/how-ai-will-reshape-computer-systems-by-2035-a-jeffersonian-dinner-in-san-francisco-about-our-10000x-future/. Accessed: 2026.

T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

DeepSeek-AI. Deepseek-v3 technical report, 2024. URL https://arxiv.org/abs/2412.19437.

M. Feng and S. Boyd. OCS. https://github.com/miria00/OCS, 2026. Solver implementation; the method is described in a companion paper.

E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro, and D. Sculley. Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, pages 1487–1495. ACM, 2017. doi: 10.1145/3097983.3098043. URL https://doi.org/10.1145/3097983.3098043.

A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

B. Halpern. Fixed points of nonexpanding maps. 1967.

C. Hong, Q. Huang, G. Dinh, M. Subedar, and Y. S. Shao. DOSA: Differentiable model-based one-loop search for DNN accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023. doi: 10.1145/3613424.3623797.

C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao. Cosa: Scheduling by constrained optimization for spatial accelerators, 2021. URL https://arxiv.org/abs/2105.01898.

Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

P. Jain, A. Jain, A. Nrusimha, A. Gholami, P. Abbeel, K. Keutzer, I. Stoica, and J. E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization, 2020. URL https://arxiv.org/abs/1910.02653.

S. Jia, Z. Ling, C. Bai, K. Zhao, and J. Zhai. Fadiff: Fusion-aware differentiable optimization for dnn scheduling on tensor accelerators, 2025. URL https://arxiv.org/abs/2511.22348.

N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, et al. Ten lessons from three generations shaped google's tpuv4i: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–14. IEEE, 2021.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

H. Lu and J. Yang. cupdlp.jl: A gpu implementation of restarted primal-dual hybrid gradient for linear programming in julia, 2024. URL https://arxiv.org/abs/2311.12180.

H. Lu, J. Yang, H. Hu, Q. Huangfu, J. Liu, T. Liu, Y. Ye, C. Zhang, and D. Ge. cupdlp-c: A strengthened implementation of cupdlp for linear programming by c language, 2024. URL https://arxiv.org/abs/2312.14832.

H. Lu, Z. Peng, and J. Yang. cupdlpx: A further enhanced gpu-based first-order solver for linear programming, 2025. URL https://arxiv.org/abs/2507.14051.

G. P. McCormick. Computability of global solutions to factorable nonconvex programs: Part i: Convex underestimating problems. *Mathematical programming*, 10(1):147–175, 1976.

L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst. Zigzag: Enlarging joint architecture-mapping design space exploration for dnn accelerators. *IEEE Transactions on Computers*, 70(8): 1160–1174, 2021.

L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst. DeFiNES: Enabling fast exploration of the depth-first scheduling space for DNN accelerators through analytical modeling. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 570–583. IEEE, 2023. doi: 10.1109/HPCA56546.2023.10071098. arXiv:2212.05344.

A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*, pages 304–315, 2019.

S. G. Patil, P. Jain, P. Dutta, I. Stoica, and J. E. Gonzalez. Poet: Training neural networks on tiny devices with integrated rematerialization and paging, 2022. URL https://arxiv.org/abs/2207.07697.

B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. Emer, C. T. Gray, B. Khailany, and S. W. Keckler. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 14–27. ACM, 2019. doi: 10.1145/3352460.3358302. Network-on-package / network-on-chip provisioning (radix, link count, hierarchical crossbar+mesh) for multi-chiplet DNN inference.

W. Snyder and contributors. Verilator: Open-source SystemVerilog simulator and lint system. https://www.veripool.org/verilator/, 2025. Version 5.050, cycle-based two-state simulation.

B. F. Spector, S. Arora, A. Singhal, D. Y. Fu, and C. Ré. ThunderKittens: Simple, fast, and adorable AI kernels. *arXiv preprint arXiv:2410.20399*, 2024.

M. Tan and Q. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

S. Tuli, C.-H. Li, R. Sharma, and N. K. Jha. Codebench: A neural architecture and hardware accelerator co-design framework. *ACM Transactions on Embedded Computing Systems*, 22(3): 1–30, Apr. 2023. ISSN 1558-3465. doi: 10.1145/3575798. URL http://dx.doi.org/10.1145/3575798.

D. Zhang, S. Huda, E. Songhori, K. Prabhu, Q. Le, A. Goldie, and A. Mirhoseini. A full-stack search technique for domain optimized deep learning accelerators. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, page 27–42. ACM, Feb. 2022. doi: 10.1145/3503222.3507767. URL http://dx.doi.org/10.1145/3503222.3507767.

L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang. Tileflow: A framework for modeling fusion dataflow via tree-based analysis. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 1271–1288, 2023.

Appendix

Table of Contents

A Extended Related Work . . . 18

B Modeling Properties and Containment . . . 19

B.1 Bounded Products and Epigraphs . . . 19

B.2 Containment of Lifted FAST Fusion . . . 19

B.3 A Fixed-Hardware Ceiling on Any Scheduler . . . 20

B.4 Structure and Safe Pruning . . . 21

C Linear Program Formulation Details . . . 22

C.1 Relaxation and Integral Recovery . . . 22

C.2 LP Matrix Structure . . . 22

C.3 Current Solver Path . . . 22

D Implementation-Level Validation: Decoder RTL and Schedule-Tape Checks . . . 23

D.1 BF16/FP32 decoder experiments . . . 23

D.2 Schedule-tape and INT8 datapath checks . . . 24

D.3 Physical-design validation: post-synthesis area and the linearity of the cost model . . . 25

D.4 Measuring the compute-energy coefficients . . . 26

E Designing for Manufacturability and Feasibility Audits . . . 28

E.1 Five Levels of Validation . . . 28

E.2 Compute-Floor and Roofline Tightness Audits . . . 28

E.3 Six Physical-Design Feasibility Gates . . . 32

E.4 Reported Efficiency Quantities . . . 33

F Speedup Analysis: Baseline Deficit Attribution and Design-Parameter Sensitivity . . . 34

F.1 Two-Factor Speedup Decomposition . . . 34

F.2 One-Decision-at-a-Time Sensitivity Sweeps . . . 36

G Hardware Search and Search Cost . . . 38

G.1 Hardware and Vizier Search . . . 38

H LLM-Architect Outer Loop Details . . . 40

H.1 Profile-Conditioned LLM-Architect Application . . . 40

H.2 LLM in the Loop Prompts . . . 42

H.3 Resource Prices and Proposal Validation . . . 45

I Two Optimal Tiling Examples: joint-tile LP Solution at a Glance . . . 46

I.1 joint-tile LP Solution: Llama-3 70B on simulated H100 . . . 46

I.2 joint-tile LP Solution: DeepSeek-V3 on simulated RTX 5090 . . . 48

J Examples of CHEETAH Design Blueprints . . . 51

J.1 Variant 1: TSMC N7 + GDDR/LPDDR . . . 51

J.2 Variant 2: TSMC N5 + HBM3 . . . 56

K Limitations, Future Work, and Broader Impact . . . 60

L Complete Rows of the Master Blocks 60
L.1 The Retained Joint Tiling–Placement Block 60
L.2 Native Cover, Lifecycle and Storage 61
L.3 Capacities, Staging and Installed Hardware 62
L.4 Native Resource Demands and Linear Service Envelopes 62
L.5 Direct Roofline and Physical Constraints 63
L.6 Feature Scope 64
M Native Full-Inference Provenance and Reproduction 66
M.1 End-to-End Solution and Recovery Workflow 66
M.2 The OCS Solver 67
M.3 Certified Solution of the Frozen Expanded Master 68
N Coefficient Audit Trail: Where Every Constant Comes From 70

A Extended Related Work

Architecture and mapping search. Timeloop searches mappings for a supplied architecture [Parashar et al., 2019]. CoSA uses mixed-integer optimization for a constrained mapping objective; its guarantee does not make the resulting latency globally optimal under an arbitrary simulator [Huang et al., 2021]. ZigZag expands the architecture–mapping space [Mei et al., 2021], while DOSA places architecture and mapping variables in one differentiable program [Hong et al., 2023]. Gemini couples spatial mappings and architectural choices for multi-chiplet accelerators [Cai et al., 2024]. CODEBench uses multi-objective exploration over accelerator configurations [Tuli et al., 2023]. FAST combines hardware, mapping, and compiler search, including a fusion ILP [Zhang et al., 2022]. These methods motivate our restricted controls. CHEETAH retains alternatives until shared storage, implementation cost, and hardware provisioning can be evaluated together.

Fusion, placement, and rematerialization. DeFiNES includes depth-first fused execution, caching, and overlap recomputation; its restricted placement and recomputation cells in Table 1 are therefore not marked absent [Mei et al., 2023]. Fast-and-Fusiest searches legal operation orders as well as fusion and mappings under its stated cost model [Andrulis et al., 2026]. These are substantial joint compiler capabilities, even though they do not provide CHEETAH's complete temporal state model and variable hardware domain. Checkmate formulates training checkpointing as an integer program [Jain et al., 2020]; POET combines rematerialization and paging [Patil et al., 2022]. CHEETAH uses paid recomputation, transfer completion, and readiness to represent these tradeoffs for inference. Representability alone is not a measured gain; our native controls exercise only the features identified in Section 6.5.

Polyhedral transformations. Pluto constructs affine schedules for parallelism and locality [Bondhugula et al., 2008]. Tiramisu exposes a richer programmable scheduling language, including storage and recomputation transformations [Baghdadi et al., 2019]. Their user-programmed capabilities should not be confused with an automatic joint search over every table axis. Conversely, CHEETAH's causal event choices should not be described as arbitrary loop transformation coverage: its implementation library and dependency contract bound the represented schedules.

Relaxations and first-order LP solvers. McCormick envelopes bound bilinear products over boxes [McCormick, 1976]; exactness for a single integral-factor product does not prove exactness of an entire fractional co-design program. PDLP demonstrates practical large-scale linear programming with first-order methods [Applegate et al., 2022]. GPU variants include cuPDLP.jl, cuPDLP-C, and cuPDLPx [Lu and Yang, 2024, Lu et al., 2024, 2025]. OCS, our in-house solver framework, uses an anchored restarted PDHG kernel with a diagonal variable metric and structured forward/adjoint operators for discovery, and compact fits with original-domain verification as its certified closing stage. A certified primal–dual objective interval establishes the frozen LP accuracy independently of any single iterate's tolerance; it does not establish a general automatic solver or discrete-design optimality.

LLM-guided search. FunSearch and AlphaEvolve combine model proposals with deterministic evaluation [Romera-Paredes et al., 2024, Novikov et al., 2025]. CHEETAH similarly uses an LLM to propose search interventions, with resource prices and failed-trial diagnostics as context. The outer loop edits Vizier search neighborhoods; current master experiments can also propose implementations and hardware alternatives directly. A proposal is admitted only after its numerical, dependency, and resource checks. The proposal mechanism is not an exhaustive separation oracle.

B Modeling Properties and Containment

The following statements concern represented feasible sets. They do not imply that a numerical solver has optimized the complete master or that arbitrary rounding preserves feasibility.

B.1 Bounded Products and Epigraphs

Proposition 1. *For $0 \le y, p \le 1$, the inequalities $0 \le w$, $w \le y$, $w \le p$, and $w \ge y + p - 1$ describe the convex hull of the graph $w = yp$. If either factor is integral, these inequalities force $w = yp$.*

Proof. The lower envelope is $\max(0, y + p - 1)$ and the upper envelope is $\min(y, p)$ [McCormick, 1976]. For $y = 0$ they both force $w = 0$; for $y = 1$ they force $w = p$. The cases for integral p are symmetric. □

This is an individual-product property, not a convex-hull theorem for the complete program. For example,

$$y_1 = y_2 = p = \tfrac{1}{2}, \quad w_1 = w_2 = \tfrac{1}{2} \tag{3}$$

satisfies the separate envelopes and $y_1 + y_2 = 1$, but violates the identity $w_1 + w_2 = p$ that holds for exact products. Linking rows

$$\sum_c w_{ick} = p_{\mathrm{assoc}(i,k)} \tag{4}$$

can strengthen a branch when its intended integral semantics justify them. They must not be added to an immutable compatibility branch while claiming containment of every original fractional point.

Corollary 1 (Disaggregated service epigraphs)**.** *For fixed resource costs a_c, b_c, the one-choice epigraph has the perspective form $\sum_c y_c = 1$, $y_c \ge 0$, $t_c \ge a_c y_c$, $t_c \ge b_c y_c$, $T \ge \sum_c t_c$. It gives $T \ge \sum_c y_c \max(a_c, b_c)$ and dominates $T \ge \max(\sum_c a_c y_c, \sum_c b_c y_c)$.*

Proof. For each c, minimizing t_c gives $y_c \max(a_c, b_c)$. Summing maxima is at least either sum separately. With a one-hot choice the two formulations coincide. For shared variable hardware, every additional coupling must also be disaggregated consistently; separate product envelopes alone do not prove the convex hull of the entire HW/SW problem. □

B.2 Containment of Lifted FAST Fusion

This subsection concerns the retained joint tiling–placement block of Section 4.1, whose coordinates are the tile selection y_{ic}, activation residency p_{et}, weight residency p^W_{it}, rematerialization r_{it}, the one-hot global-buffer selector $h_{\mathrm{GM}} \in [0,1]^K$ over a discrete grid $g_1, \ldots, g_K$, and the per-operation latencies T_i that the objective sums. Write that block's feasible set at a fixed chip H and workload graph G as

$$\mathcal{P}_V(H, G) = \{x = (y, p, p^W, r, h_{\mathrm{GM}}, T) : A_V x \le b_V(H, G)\}, \tag{5}$$

with $\sum_c y_{ic} = 1$, $\sum_k h_{\mathrm{GM},k} = 1$, all selection and residency coordinates in $[0,1]$, $T \in \mathbb{R}^L_{\ge 0}$, and Timeloop-derived constants $T_i^{\min}(c), T_i^{\max}(c), t_i^k(c)$ at C Pareto tile configurations per operation.

The lifted FAST fusion set. FAST runs the mapper first, commits to one tile choice c_i^{FAST} per operation, and only then solves its fusion problem over tensor pinning. To compare the two formulations in one variable space we lift FAST's feasible set by encoding its committed mapper choice as a singleton one-hot,

$$\mathcal{F}_F^{\mathrm{lift}}(H, G) = \Big\{(y, p, p^W, T) : y_{i, c_i^{\mathrm{FAST}}} = 1,\ y_{ic} = 0\ \forall c \ne c_i^{\mathrm{FAST}},\ p \text{ in FAST's consecutive fusion pattern},\ A_F x \le b_F(H, G)\Big\} \tag{6}$$

where A_F, b_F encode FAST's buffer capacity at its hard-coded g, its bandwidth, and its per-stage compute floor under the fixed-y projection. FAST commits to $r = 0$ and to that fixed g by construction. The first $LC + ET + LT$ columns of A_V coincide with A_F, and both use the same bandwidth b and the same mapper cost vectors.

Theorem 1 (Containment of lifted FAST fusion)**.** *Fix H, G, a grid value $g \in \{g_1, \dots, g_K\}$ and the mapper-derived constants, and let $k^\star$ satisfy $g_{k^\star} = g$. The map*

$$\iota : \mathcal{F}_F^{\mathrm{lift}}(H,G) \hookrightarrow \mathcal{P}_V(H,G), \qquad \iota(y,p,p^W,T) = \big(y,\tilde{p},p^W,r=0,h_{\mathrm{GM}} = e_{k^\star},T\big), \tag{7}$$

with $\tilde{p}_{et} = 1$ exactly on the timesteps covered by FAST's consecutive fusion region for the edge supplying e and zero elsewhere, is well defined and injective, and it preserves the objective: $\mathbf{c}^\top \iota(x) = \mathbf{c}^\top x$ for every $x \in \mathcal{F}_F^{\mathrm{lift}}(H,G)$.

Proof. Feasibility. The integer cube embeds into the LP cube, $\{0,1\}^d \subset [0,1]^d$, so any integer-feasible (y,p,p^W) lies in $\mathcal{P}_V$ along the shared coordinates. The lifted $\tilde{p}$ is zero off FAST's consecutive fusion region by construction, so the activation-persistence and capacity rows of $\mathcal{P}_V$ reduce to FAST's. Setting $r = 0$ disables rematerialization and $h_{\mathrm{GM}} = e_{k^\star}$ pins the buffer-capacity row to FAST's g. The per-stage compute floor evaluates identically because both sets consume the same mapper cost vectors under the singleton-y projection. Hence $\iota(x) \in \mathcal{P}_V$. *Injectivity.* ι is the identity on the shared coordinates and constant on the new ones, so $\iota(x) = \iota(x')$ implies $x = x'$. *Objective preservation.* The objective $\sum_i T_i$ depends only on the shared coordinate T, so $\mathbf{c}^\top \iota(x) = \mathbf{c}^\top x$. If the objective additionally carries an ε-tiebreaker $\varepsilon \sum_k k\, h_{\mathrm{GM},k}$, that term equals the constant $\varepsilon k^\star$ on $\iota(x)$ and changes neither the optimum nor its minimizer at fixed g. □

Corollary 2 (The relaxation dominates the lifted FAST objective)**.** *For every chip H, workload G and matched mapper constants,*

$$\min_{x \in \mathcal{P}_V(H,G)} \mathbf{c}^\top x \;\le\; \min_{x \in \mathcal{F}_F^{\mathrm{lift}}(H,G)} \mathbf{c}^\top x. \tag{8}$$

Proof. Let $x_F^\star$ attain the lifted FAST minimum. By Theorem 1, $\iota(x_F^\star) \in \mathcal{P}_V$ and $\mathbf{c}^\top \iota(x_F^\star) = \mathbf{c}^\top x_F^\star$, so the minimum over $\mathcal{P}_V$ is at most that value. □

Consequently a fixed-hardware gap between FAST-style fusion and CHEETAH cannot be attributed to FAST solutions lying outside the search space: it must come from the extra continuous degrees of freedom y, r and h_{GM}, from relaxation versus integral rounding, or from differences in cost model or baseline normalization.

Remark (Scope). Theorem 1 is a modeling-containment result and bounds the relaxation, not the rounded executable schedule, which is reported separately. It does not predict numerical equivalence of reported speedups: those also move with baseline normalization, simulator overheads, depthwise mapping on 128×128 arrays, and batch and core conventions. Containment is also not automatic reproduction of a published chip, since the branch and its complete contract must be imported, and unbounded branches can retain recession directions at zero weight.

Corollary 3 (Branch containment in the full master)**.** *Write an included branch as $P_b = \{x : E_b x = e_b,\ G_b x \le g_b,\ \ell_b \le x \le u_b\}$; the master homogenizes its rows and finite boxes with λ_b. For every included branch p and every $x \in P_p$, the assignment $\lambda_p = 1$, $x_p = x$, and $\lambda_b = 0, x_b = 0$ for $b \neq p$ is master-feasible with the same objective and hardware readout, so $\inf_{z \in P_M} c_M^\top z \le \min_b \inf_{x \in P_b} c_b^\top x$.*

Proof. Every active-branch row and bound becomes its original constraint, and every inactive-branch row has zero on both sides, which satisfies each homogenized finite box. The weights sum to one, active hardware coordinates are unchanged, and $\sum_b c_b^\top x_b = c_p^\top x$. The argument covers fractional points as well as integral schedules. □

B.3 A Fixed-Hardware Ceiling on Any Scheduler

For a workload G on a fixed chip H with DRAM bandwidth b_H, define the *baseline-deficiency ratio*

$$\mathcal{D}(H,G) = \frac{T_{\mathrm{naive}}(H,G)}{\max\big(T^{\mathrm{LB}}_{\mathrm{compute}}(H,G),\, T_{\mathrm{memory}}(H,G)\big)}, \tag{9}$$

where $T_{\mathrm{naive}} = \sum_i T_i^{\max}$ is the naive streaming time, $T^{\mathrm{LB}}_{\mathrm{compute}} = \sum_i \min_c T_i^{\min}(c)$ is the schedule-independent per-operation compute floor, and $T_{\mathrm{memory}} = \sum_i \mathrm{bytes}_i / b_H$ is the bandwidth floor. The denominator is the optimistic compute-plus-DRAM roofline bound on the cycles any scheduler needs to execute G on H; adding NoC, SRAM, vector or control floors can only raise it and lower $\mathcal{D}$.

Theorem 2 (Fixed-hardware deficiency ceiling)**.** *For every chip H, workload G and feasible point $x \in \mathcal{P}_V(H,G)$,* $\text{speedup}(x) := T_{\text{naive}}(H,G)/\mathbf{c}^\top x \le \mathcal{D}(H,G)$*; in particular* $\min_{x\in\mathcal{P}_V} \mathbf{c}^\top x \ge \max(T^{\text{LB}}_{\text{compute}}, T_{\text{memory}})$.

Proof. The compute floor $T_i \ge \sum_c T_i^{\min}(c)\, y_{ic}$ and the bandwidth floor $\sum_i T_i \ge T_{\text{memory}}$ are rows of $\mathcal{P}_V$. Since $\sum_c y_{ic} = 1$ and $y \ge 0$, $T_i \ge \min_c T_i^{\min}(c)$ for every feasible y; summing over i gives $\sum_i T_i \ge T^{\text{LB}}_{\text{compute}}$, and with the bandwidth floor $\mathbf{c}^\top x \ge \max(T^{\text{LB}}_{\text{compute}}, T_{\text{memory}})$. Dividing T_{naive} by both sides gives the claim. □

The bound is physical rather than algorithmic: it limits every scheduler, not only CHEETAH or FAST, so $\mathcal{D}$ measures how much any software can extract on a given chip–workload pair. Table 5 evaluates it on fixed TPU-v3 (65,536 MACs, 900 GB/s, 32 MiB, BF16). On every LLM the fixed-hardware

Table 5: Fixed-TPU-v3 deficiency ceiling and the fixed-hardware LP solution. CHEETAH saturates the physical ceiling exactly on every LLM.

Workload	T_{naive} (cyc)	$\max(T_c, T_m)$ (cyc)	$\mathcal{D}(H,G)$	CHEETAH, fixed HW
Llama-1 13B	2.90×10^{10}	1.35×10^{10}	$2.14\times$	$2.14\times$ (100%)
Llama-1 65B	1.45×10^{11}	6.74×10^{10}	$2.14\times$	$2.14\times$ (100%)
Llama-3 70B	1.85×10^{11}	7.38×10^{10}	$2.51\times$	$2.51\times$ (100%)
DeepSeek-V3	5.88×10^{10}	2.65×10^{10}	$2.22\times$	$2.22\times$ (100%)

solution reaches the ceiling, so the modest fixed-TPU-v3 gains of Table 4 measure the chip, not the formulation: TPU-v3 has only about $2.5\times$ of extractable inefficiency on these workloads, and the larger gains of Section 6 require co-designing the chip itself.

B.4 Structure and Safe Pruning

Proposition 2. *For fixed graph incidence, variable domains, and candidate library, changes only to numerical costs and bounds preserve the structural row/column pattern.*

Proof. The same indexed row templates address the same coordinates. Hardware-dependent coefficients can change their values; changing the implementation library or introducing different lifetimes need not preserve the pattern. □

This permits reuse of indexing and warm starts. A new scaling or norm bound must still be checked when coefficients change. Pareto pruning similarly needs componentwise dominance in every resource and interface used by the branch; a two-axis latency plot alone is insufficient when workspace, traffic tiers, or numerical formats differ.

C Linear Program Formulation Details

C.1 Relaxation and Integral Recovery

The master is a linear relaxation of a finite implementation and hardware library. Its canonical form is

$$\min_x c^T x \qquad \text{subject to} \qquad \ell_c \le Ax \le u_c, \quad \ell_v \le x \le u_v. \tag{10}$$

McCormick envelopes and epigraphs retain linearity, but do not guarantee a small integrality gap. Integral recovery selects a concrete hardware and implementation assignment and checks the resulting schedule. Nearest-integer rounding alone is not a feasibility proof. Appendix L gives the explicit retained joint schedule rows and the expanded causal rows.

C.2 LP Matrix Structure

Table 6 records the two controls' sparse patterns. Fixed incidence and menus permit index reuse. Numerical coefficient changes can still require new scaling and norm checks, and a changed library or dependency graph can change the pattern itself.

Table 6: Structural comparison of the original fixed-tile and joint-tile controls. For Llama3-70b: $L = 720, T = 720, E \approx L$.

Property	fixed-tile control	CHEETAH
Variables	$T_i\,(L) + p^I, p^O, p^W, r\,(4 \cdot L \cdot T) + T_{\text{final}}\,(1)$ $+ FT_i\,(L) \;\approx\; 4L^2 + 2L + 1$	$T_i\,(L) + y_{i,c}\,(LC) + p_{e,t}\,(ET)$ $+ p^W, r\,(2LT) + w_{i,c,k}\,(3LC) \;\approx\; 3L^2 + 4LC + L$
Llama-3 70B actual	L=720, T=720 $\rightarrow$ 2.07M vars, 2.33M constraints, 4.67M nnz	Similar order of magnitude; $p_{e,t}$ is edge-indexed (ET) instead of layer-pair-indexed, which saves $\approx L^2$ vars when DAG is mostly linear
Density	$\text{nnz}/(m \cdot n) \approx 9.6 \times 10^{-7}$ ($\approx$1 nnz per million entries)	Similar; row group (5) is the most dense
Avg. nnz/row	≈ 2.0	≈ 2.0 with bursts up to $4L + L$ on capacity rows
Constant pattern across runs	Yes, parametric in L. Every workload of L layers gets the exact same matrix template; only coefficient values (`T_max`, `T_min`, `d_I/O/W`) change.	Yes, parametric in (L, C, E, T). DAG topology lives in the (i, e, t) index map, but row template is invariant. Sparsity pattern is workload-agnostic.
Banded	Most rows touch only (i, t) and $(i, t-1)$ (persistence chains, weight monotone, FT recurrences). Bandwidth $\approx T$ along the time axis.	McCormick triples (4a/b/c) are 1–2 nnz; persistence on edge (6) couples (e, t) to $(e, t-1)$. Same time-band structure.
Exception to band-ness	Rows from group (C): global memory capacity per timestep span all L weights at that timestep $\rightarrow$ wide $\approx L+2$ nnz rows, T of them.	Rows from group (5): span all E edges + all L weight footprints at that timestep $\rightarrow$ $\approx E+L \approx 2L$ nnz rows, T of them.
Why constant pattern works for solver	Per-row degree distribution is bimodal: $\mathcal{O}(L^2)$ rows with $\le$3 nnz + $\mathcal{O}(T)$ fat capacity rows. Our solver handles this after preconditioning.	Same bimodal pattern, plus a small $(3 \cdot L \cdot C)$ dense McCormick block. Still sparse in aggregate.

C.3 Current Solver Path

These matrix sizes describe the original restricted workloads, not the full native temporal expansion. Section 5 summarizes the OCS solver framework and Appendix M.2 gives its kernel and closing stage. These smaller cases do not by themselves establish universal solver superiority; the expanded continuous LP is certified at the stated accuracy by the primal–dual evidence in Appendix M.3.

D Implementation-Level Validation: Decoder RTL and Schedule-Tape Checks

Performance results remain analytical. This appendix covers BF16/FP32 decoder mechanism tests, schedule-tape and INT8 datapath experiments, and post-synthesis area and energy measurements. Verilator [Snyder and contributors, 2025] executes actual RTL and checks its outputs and control faults; simulation ticks do not establish achievable frequency, full-chip area or measured model latency. The decoder tests cover arithmetic, selected bank/delivery calendars and a KV transition slice. They do not supply a complete decoder or inference controller. The ASAP7 placement experiment in Appendix D.3 applies to its INT8 tile and does not calibrate the BF16 datapath, SRAM macros or external-memory package.

D.1 BF16/FP32 decoder experiments

We compiled and ran six RTL test groups; all twelve build/test commands passed (Table 7). These are CPU-hosted Verilator executions of the selected mechanisms, not GPU performance measurements. Sources, fixtures, executables, command lines, timestamps and outputs are preserved with the actual-run receipt.

Table 7: Functional RTL results. Every group passes its declared checks. Counts describe finite representative tests, not all model invocations.

RTL group	Executed coverage	Result
BF16/FP32 arithmetic	615,652 comparisons; all BF16 patterns times ± 1; special/random triples; 32 ordered $K = 7,168$ reductions	Pass
DeepSeek score tile	256 PEs (8×32), 5,227 source ticks, 49,800 received words, 22 positive output installs, 135,168 valid MAC updates	Pass
DeepSeek AV tile	256 PEs, 8,889 source ticks, 33,540 received words, 2 positive installs, 262,208 valid MAC updates	Pass
Dense decoder tiles	31 full-K cases (13B/65B/70B: 10/10/11); 30,399,392 MAC updates, 3,136 BF16 outputs, 7,400 explicit stalls	Pass
KV transition	4,096 prefix tokens, 511 N16 K-word read/modify/write appends, 4,607 terminal tokens bit-identical; 1,533 backpressure cycles	Pass
PE context isolation	258 updates; completed output stable during 193 other-context updates; simultaneous installation and independent update	Pass

Arithmetic is executed in the DUT. Integer SystemVerilog logic implements BF16 multiplication, ordered fused FP32 accumulation and final BF16 round-to-nearest-even. The C++ `std::fma` oracle is confined to the testbench; no DPI, `real` or `shortreal` arithmetic supplies DUT outputs. Coverage includes 1,152 subnormal results, 184 signed-zero results, 32,585 infinities and 5,234 canonical NaNs. Overflow-cancellation and half-min-subnormal cases distinguish fused accumulation from separately rounded multiplication and addition. Canonical NaNs use `0x7fc00000`/`0x7fc0`; NaN payload/sign preservation is not claimed. This tests the declared ordered arithmetic contract, not universal reduction semantics of every upstream matrix library. The 577-bit exact intermediate is a functional construction with no synthesized-area or 1-GHz timing qualification.

Banks, delivery and output installation. The DeepSeek fixtures bind original request/layer/context owners and selected source calendars. An actual 256-PE outer-product tile executes both FP32 contexts. The delivery top contains ping-pong storage, addressed reads/writes, Ready/live state, a 64-byte A register file with paid bank preloads, registered fanout, matched B delay and K/wave/context tags. A shared 32-byte packing buffer arbitrates receiving and completed-context extraction. Score/AV tests include 264/516 preload ticks and 704/64 extraction ticks; 590/30 ticks overlap MAC updates with extraction from a different context. Output conversion and service remain paid, and reuse waits for installation. Both calendars reject four injected faults: port conflict, read-before-Ready, live-buffer overwrite and missing output installation.

The fixture drives the controller pins from the source calendar. One tile instantiates all arithmetic PEs; the rest of the gang is checked through bank/route coordinates and demand, not replicated as a whole arithmetic chip. The row/column outer-product interface is not a routed nearest-neighbour

systolic mesh. Route delays and positive output-service intervals remain stimulus obligations; there is no physical DRAM/HOST controller.

Fusion and KV boundaries. The AV test consumes an already aligned BF16 A buffer. The FP32 softmax/cast producer and its head-strided padding, zero-fill and store routine remain source-checked, rather than implemented by this RTL test. Thus the producer-layout fusion ablation in Table 38 has analytical source/replay evidence, while final output rounding and selected delivery/context mechanisms have RTL evidence. The KV controller checks one K head/component against functional external backing: initialized padding does not confer logical readiness, and every append waits for its write. Full V-cache behavior, terminal HOST DMA and the complete inference sequencer remain outside this test.

What the experiments establish. All injected numerical values are synthetic; task shapes, coordinates, reduction order and output boundaries are source-bound. Dense tests cover selected mappings for all three Llamas, but not their ordinary-DRAM delivery controllers. These experiments support the tested arithmetic and control mechanisms. They do not establish trained-weight model quality, full-model bit equivalence, physical bank macros, routed timing, complete decoder RTL or the predicted accelerator runtimes. The eight N7/N5 architectural views retain unknown library/clock/area/power fields and apply no process-node scaling.

D.2 Schedule-tape and INT8 datapath checks

The following experiments use their own workloads and hardware and are not BF16 decoder validation.

Experiment 1: schedule tape replay (realizability and memory legality). We compile the full committed DeepSeek-V3 [DeepSeek-AI, 2024] schedule at a TPU-v3-class configuration ($C_{\text{GM}} = 8$ MiB) into a 1,383-instruction event tape over the vocabulary {EXECUTE, LOAD, FREE, EVICT, RELOAD, REMATERIALIZE}, and execute it on a cycle-accurate SystemVerilog event engine (a Global-Buffer allocator, a bandwidth-limited DMA model, and a compute model) under Verilator. Each instruction is cross-checked against the analytical residency model layer-by-layer, and a deliberately over-committed tape confirms the capacity checker is live (it flags exactly one violation). Three pre-stated criteria pass (Table 9), and Table 8 reports, decision by decision, that every scheduled event executed. *Memory legality:* the tape runs to completion with **zero** capacity violations, and the RTL peak on-chip occupancy equals the analytical prediction *exactly* (7,602,176 B $= 7.25/8$ MiB, 9.4% headroom). *Cycles:* the total RTL cycle count lands within $+0.000001\%$ of the rounded LP objective, the entire gap being the 16,431 decoder/DMA sequencing cycles (10^{-8} of the total); i.e., the rounded schedule realizes with *no hidden serialization or stall* beyond the model's accounting. *Two-tape ratio:* against a stream-everything realization of the same operation order, the RTL-measured latency ratio is $1.0723\times$, matching the analytical ratio to 8 significant digits. This ratio is scoped to the pin/reload channel alone (both tapes hold the per-operation-optimal tiling fixed, so joint tile selection, the dominant channel elsewhere, is deliberately held equal). Together these establish that the LP's time-indexed residency model (Section 4.2) is not merely roofline-feasible but executes byte- and cycle-faithfully on hardware that enforces the capacity constraint at every step.

Experiment 2: datapath utilization mechanism. We next build one weight-stationary INT8 systolic PE cluster in synthesizable SystemVerilog (4×4 MAC array, INT8×INT8→INT32, with L1 buffers and a tile-loop FSM; plus a 16×16 variant), golden-verified against a plain-C++ reference convolution ($8/8$ runs bit-exact). Simulated on EfficientNet-B7 [Tan and Le, 2019] layers, the RTL reproduces the two utilization mechanisms the cost model charges for. First, the *depthwise structural ceiling*: on a broadcast systolic array a depthwise convolution can engage only 4 of 16 MAC lanes, and the RTL measures a steady-state depthwise utilization of 24.9% against the 25% analytic ceiling (6.13% vs. 6.25% on the 16×16 variant), while dense 1×1 layers hold 97–99% on the identical RTL. Second, the predicted *geometry contrasts*: the measured lane law reproduces the $4.00\times$ systolic-vs-grid contrast to $4.06\times$, and extends the cache's $18\times$ depthwise contrast to within $1.22\times$. Crucially, the same runs show the analytical cache cycle *floors* to be $192\times$ to $2475\times$ conservative (the RTL executes the identical work that much faster), which is precisely why RTL-derived claims are scoped to these utilization *contrasts* and to the schedule replay above, never to absolute cycle agreement.

Table 8: Did each decision fire? Per-decision schedule-versus-execution counters from the DeepSeek-V3 tape replay ($C_{\text{GM}} = 8\,\text{MiB}$). Every event the LP schedules is executed by the RTL engine exactly once, the capacity row is checked in hardware at every step, and the decisions the LP chose not to use at this cell are confirmed absent while their opcodes are proven live on a synthetic tape. The bound says how good the schedule is; the counters say the schedule is what ran.

Decision	LP schedule	RTL executed	Check
Tile choice (y)	903 EXECUTE	903	✓ ids match 903/903
Residency (p, p^W)	180 LOAD, 299 FREE, 2,709 pins	180, 299	✓ 2,709/2,709 pins
Fusion (operand kept resident)	418 of 903 ops	418	✓
Eviction / reload	0 / 0	0 / 0	✓ opcodes live
Rematerialization (r)	0	0	✓ opcode live
Capacity row	1,083 hardware checks	0 violations	✓ injected overflow flags 1
Peak occupancy	7,602,176 B	7,602,176 B	✓ byte-exact, trace within 4 B
Total cycles	1.6437×10^{12}	+16,431 cycles	✓ +0.000001%, accounted

Table 9: Analytical-model-to-RTL cycle validation. Synthesizable SystemVerilog simulated cycle-accurately with Verilator [Snyder and contributors, 2025] and checked against a C++ reference. The RTL confirms schedule realizability, byte-exact memory legality, and the LP's utilization mechanisms; it is *not* used as an absolute cycle oracle.

Quantity (RTL vs. analytical)	Measured	Verdict
Exp. 1: DeepSeek-V3 schedule replay (1,383 instr., $C_{GM} = 8$ MiB)		
Capacity violations	0	pass
Peak on-chip occupancy	7,602,176 B (exact)	byte-exact
Total cycles vs. rounded LP obj.	+0.000001%	no hidden serialization
Two-tape ratio (pin/reload only)	1.0723× (8-digit match)	pass
Exp. 2: EfficientNet-B7 INT8 systolic cluster (golden 8/8)		
Depthwise MAC-lane utilization	24.9% vs. 25% ceiling	mechanism confirmed
Systolic-vs-grid geometry contrast	4.06× vs. 4.00×	mechanism confirmed
Absolute cache cycle floors	192–2475× conservative	scope guard

The mechanism is what matters here: the utilization losses the LP models (Section 4) are the ones the real datapath exhibits.

D.3 Physical-design validation: post-synthesis area and the linearity of the cost model

Experiments 1 and 2 check the schedule tape and INT8 datapath slices. A second, independent question is whether the *cost model* that evaluates the hardware is itself faithful: the program sizes a chip through the coefficients k_{MAC} (area per MAC) and π_{MAC} (power per MAC), and every manufacturability gate and the area and power caps are expressed in them. If those coefficients are wrong, the certified designs are certified against the wrong physics. We therefore take the datapath the program sizes, write it as synthesizable RTL, and push it through a full open-source RTL-to-placement flow at an advanced predictive node.

Setup. We implement a parameterized weight-stationary INT8 systolic tile (an $N \times N$ array of processing elements, each holding a resident 8-bit weight, streaming 8-bit activations east and 32-bit partial sums south, one MAC per PE per cycle in steady state) and synthesize, floorplan, place, and timing-repair it with OpenROAD [Ajayi et al., 2019] against the ASAP7 7 nm predictive PDK [Clark et al., 2016] at a 1 GHz target, for $N \in \{16, 32, 64\}$, that is 256 to 4,096 MACs. We report post-detailed-placement extracted areas. We do not run clock-tree synthesis or routing, and we make no tape-out claim: the quantity under test is the area coefficient and its scaling law, both of which are fixed at placement.

Result 1: the linear area model is structurally correct. Table 10 reports extracted cell area at the three sizes. A linear fit gives $A_{\text{cell}} = 82.08\,\text{MACs} - 206.5\ \mu\text{m}^2$ with $R^2 = 0.999999$, and the extracted area *per MAC* varies by only 0.46% across a 16× range in MAC count. This is a stronger statement than agreement on a single number: the program assumes area is *linear* in the MAC count,

with the same slope at every chip size, and that assumption is exactly what a placed datapath exhibits. The linear area row in the program is therefore not a convenient approximation but a measured property of the datapath it sizes.

Result 2: the area coefficient is anchored and conservative. The fitted slope is 8.21×10^{-5} mm^2/MAC of standard-cell area. Placed designs occupy a core larger than their cell area; our floorplans sit at 41% placement utilization, giving 2.0×10^{-4} mm^2/MAC of core area. The value the program uses, $k_{\mathrm{MAC}} = 1.5 \times 10^{-4}$ mm^2/MAC, lies between these two and corresponds to the cell area at 54.7% placement utilization, a typical density for a datapath of this kind. The coefficient is thus bracketed by extracted silicon area from below and by our own conservative floorplan from above.

Result 3: the power coefficient, and why we anchor it to silicon rather than to this flow. Power at the same operating point is instructive in a different way. Reporting placement-level power on this tile with flat random switching activity yields 2.0 to 6.5 pJ per MAC operation as the assumed input activity rises from 5% to 50%, against the 0.40 pJ/MAC-op implied by π_{MAC}. Rather than adjudicate this within the flow, we test it against an external physical bound. Shipped accelerators deliver INT8 arithmetic at 0.35 pJ/op (H100, 4 nm), 0.64 pJ/op (A100, 7 nm), and 1.27 pJ/op (TPU-v4i, 7 nm), and these are *whole-chip* figures that include memory, cache, and control. Our number is for the multiplier array alone, at the same node, and exceeds an entire A100 per operation by 3.2–10.1×; a bare array cannot consume more energy per operation than the complete system that contains it. The measurement, not the coefficient, is therefore out of range: with a generic synthesized multiplier, no clock gating or operand isolation, and flat random activity in place of correlated INT8 data, this flow reports an upper bound rather than a representative figure. The program's $\pi_{\mathrm{MAC}} = 0.40$ pJ/MAC-op sits where an array-only coefficient should, at the same order as and slightly below whole-chip shipped silicon, and we anchor it there (Appendix N).

Synthesis of the three hardware verification results. Design-space-exploration frameworks are conventionally validated in three ways: against measured taped-out chips, against post-synthesis extraction, and against other frameworks [Mei et al., 2021]. The Timeloop-derived Pareto data places this work in the third category and the silicon-anchored coefficients in the first; this section supplies the second. Concretely, it provides an RTL-to-placement check of the INT8 array's area scaling at 7 nm. It does not convert every architectural gate into a physically validated constraint, and its coefficient cannot be transferred to the BF16/FP32 functional implementation. The decoder power entries remain architectural proxies, with external package costs and physical timing open.

Table 10: Post-placement extracted area of the INT8 systolic tile at ASAP7 7 nm. Area per MAC is constant to within 0.46% over a $16\times$ range in MAC count; the linear fit has $R^2 = 0.999999$.

N	MACs	Cell area (μm^2)	Cell area/MAC (mm^2)	Core area/MAC (mm^2)
16	256	20,981	8.20×10^{-5}	1.99×10^{-4}
32	1,024	83,629	8.17×10^{-5}	1.99×10^{-4}
64	4,096	336,049	8.20×10^{-5}	2.00×10^{-4}
Linear fit ($R^2 = 0.999999$)			8.21×10^{-5}	
Program value k_{MAC}			1.5×10^{-4} (cell area at 54.7% utilization)	

D.4 Measuring the compute-energy coefficients

The area result above validates k_{MAC}. The program also charges for compute *energy*, and one property of that dual-guided scoring deserves a measurement rather than an assumption. Reducing arithmetic precision is usually modelled as a reduction in bytes moved, so that precision affects memory traffic alone. That is incomplete: a narrower multiplier also switches less, so precision should reduce compute energy as well. We add that term, and because a multiplier's energy is often argued to follow its partial-product count, we could have set the ratio analytically. Instead we measured it.

Method. We synthesize and place six variants of the tile at ASAP7: three operand-width combinations (W8A8, W4A8, W4A4) crossed with two power-management levels (ungated, and clock-gated with operand isolation). Power is reported at a matched input activity so that the ratios are comparable.

As a check on the flow, the W8A8 ungated variant reproduces the independently synthesized tile of Table 10 to within 0.1% despite a different source file, module name and container.

Result: the analytic ratio is wrong in both directions. Table 11 reports the measurements. The partial-product model predicts energy ratios of 0.50 for W4A8 and 0.25 for W4A4 against the W8A8 baseline. The measured ratios are 0.38 and 0.36. Narrowing the *weight* saves more than the model predicts, because switching concentrates in the multiplier tree; narrowing the *activation* saves far less, because the 32-bit accumulator and the partial-sum pipeline registers do not narrow with the operands, so the second halving buys only a further 5%. We therefore charge compute energy with the measured ratios rather than the analytic ones. The $\mathrm{W4A16}$ entry is not directly measured and is held at a conservative 0.5 rather than extrapolated in our own favour.

A negative result on gating, and why we do not include it. The same matrix was intended to characterize a clock-clock-gating decision, and it cannot. Under a flat synthetic activity model the gated variants measure as *more* expensive than the ungated ones ($1.40\times$ the power at W8A8), which is not physical. The cause is that a flat activity assumption charges for the gating cells while never crediting the idle cycles they exist to skip; the benefit of gating is visible only under a realistic idle pattern, which requires switching activity captured from a simulation of the actual workload. The area cost of gating is measured and real ($+22\%$ at W4A8, $+20\%$ at W4A4), but without a credible power benefit to weigh against it we do not introduce the decision. We report the attempt because the failure is informative: it delimits what this class of measurement can and cannot establish.

Scope. These measurements characterize the synthesized tile; they are not substituted into the cost model used for the reported speedups. They therefore provide coefficient sensitivity rather than a whole-chip performance result. The best measured variant consumes 1.22 pJ per MAC operation against the 0.40 implied by π_{MAC}, a factor of three rather than the order of magnitude that an ungated tile driven by random data suggested, with the residual attributable to predictive-PDK pessimism, a generic rather than optimized multiplier, and synthetic activity. We continue to anchor π_{MAC} to shipped silicon.

Table 11: Measured compute energy and area of the tile at ASAP7 7 nm ($N = 16$, 1 GHz, matched input activity). Ratios are against the W8A8 ungated variant. The analytic partial-product model predicts 0.50 and 0.25 for the W4A8 and W4A4 energy ratios; both are wrong.

Variant	Power (W)	pJ/MAC-op	Energy ratio	Area ratio
W8A8, ungated	0.87	3.41	1.000	1.001
W4A8, ungated	0.33	1.28	**0.38**	0.73
W4A4, ungated	0.31	1.22	**0.36**	0.63
W8A8, gated	1.22	4.77	1.40	–
W4A8, gated	0.35	1.36	0.40	0.89
W4A4, gated	0.31	1.22	0.36	0.75

E Designing for Manufacturability and Feasibility Audits

This appendix preserves the hardware audits and validation examples from the original campaign. Its RTL tapes, Timeloop mappings, and physical proxy gates retain that campaign's workload and hardware scope. The native full-inference extension has separate execution and service contracts in Appendix M; these earlier audits do not establish its memory-interface timing or physical implementation.

E.1 Five Levels of Validation

The prefill study uses five levels of modeled feasibility validation.

1. Roofline Auditor: A deterministic Roofline Auditor checks physical feasibility by comparing each LP-predicted latency against the lower bound

$$\mathrm{LB} = \max(t_{\text{floor}},\ t_{\text{memory}})\,, \tag{11}$$

   where t_{floor} is the mapper-derived compute floor of Eq. (13) and t_{memory} is post-placement off-chip traffic divided by available memory bandwidth. The audit additionally reports the compute-floor tightness ρ_{comp} and memory utilization; values above 1 would indicate a physically impossible prediction. Neither ratio is a model FLOPs utilization, which Appendix E.2 distinguishes. The prefill study's audited results have zero roofline lower-bound violations; Appendix E.2 gives the detailed checks.
2. Campaign Validator: A deterministic Campaign Validator legalizes LLM-proposed architectural neighbourhoods by enforcing hardware ranges, power-of-two constraints, TDP and area caps, MAC caps, CGM feasibility floors, and compute/bandwidth coupling constraints before Vizier is permitted to sample candidates.
3. Causal Ledger: The Causal Ledger records each LLM hypothesis, intervention, and observed outcome, making the search loop fully auditable and preventing repeated exploration of empirically invalid regions.
4. Solver diagnostics: The LP solver reports mathematical solver diagnostics such as primal and dual residuals, duality gap, rounding gap, and fractionality. In our experiments LP solutions are not treated as trustworthy unless both the relaxation and the rounded design are numerically well behaved.
5. Pre-RTL Feasibility Gates: Original-campaign designs pass six first-order screens for pin count, power density, logic depth, and related resource costs. Appendix E.3 provides their definitions. Passing these proxies does not constitute physical-design sign-off.

E.2 Compute-Floor and Roofline Tightness Audits

What this audit reports, and what it is not. The prefill study's audit column (Tables 12 and 13) records a *tightness* ratio, not a model-FLOP utilization. Write T_{LP} for the LP-predicted latency of a cell: its modeled execution time, never the optimizer's runtime. The recorded quantity is

$$\rho_{\text{comp}} = \frac{t_{\text{floor}}}{T_{\text{LP}}}, \tag{12}$$

which we call *compute-floor tightness*. We retire that label and leave every recorded number unchanged: the audited values in Appendix J and Appendix F are ρ_{comp}.

The label has to go because of the numerator. The auditor does not count useful model arithmetic from the graph. It derives a FLOP total from the per-operator Timeloop tile minima, $F = 2\,M_{\text{ref}} \sum_c T_{\min}(c)$, and divides by the candidate chip's arithmetic rate, so that

$$t_{\text{floor}} = \sum_c T_{\min}(c) \cdot \frac{M_{\text{ref}}}{M}, \tag{13}$$

with M the candidate's MAC count and M_{ref} the reference geometry's. Now $\sum_c T_{\min}(c)$ is a *mapping* result: it already carries the reference datapath's tile padding and mapping inefficiency. So t_{floor} is not $F_{\text{model}}/C_{\text{peak}}$, and ρ_{comp} is not the model FLOPs utilization of Appendix E.4. The two

ratios would coincide only if the numerator were the useful-arithmetic count at the same peak rate and phase boundary, and here it is not. Accordingly $\rho_{\text{comp}} = 1$ says the LP objective has reached the mapper's per-operator floor at the selected geometry. That is a statement about scheduling slack, not about how much of the silicon's arithmetic the model keeps busy.

Roofline-bound tightness. The auditor separately checks the lower bound

$$T_{\text{LP}} \;\geq\; \text{LB} = \max(t_{\text{floor}},\; t_{\text{memory}})\,, \tag{14}$$

where t_{memory} is total scheduled bytes divided by peak bandwidth, and enforces `roof_ratio` $= \text{LB}/T_{\text{LP}} \leq 1.001$ as the pass criterion. Only the ratio built on this maximum, $\rho_{\text{roof}} = \text{LB}/T_{\text{LP}}$, deserves the name *roofline* tightness. The distinction matters in precisely the regime the compute-only ratio hides: a memory-bound cell can sit on its roofline bound, $\rho_{\text{roof}} \to 1$, while its compute-floor ratio stays small.

What the audit establishes. A value $\rho_{\text{comp}} > 1$, or an LP latency below $\max(t_{\text{floor}}, t_{\text{memory}})$, would indicate a physically impossible prediction or a cost-model violation; the prefill study's audited cells show no such violation. That is a necessary consistency condition, and it is all this audit supplies. It does not establish realizability, silicon timing accuracy, or optimality, and a real implementation would generally fall below either bound through pipeline bubbles, NoC contention, SRAM bank conflicts, control overhead, imperfect scheduling and compiler effects.

Predicted versus measured utilization. Model FLOPs utilization is defined in Appendix E.4 as $F_{\text{model}}/(C_{\text{peak}}T)$. Predicted and measured variants are the same metric under different timing evidence,

$$\text{MFU}_{\text{pred}} = \frac{F_{\text{model}}}{C_{\text{peak}}T_{\text{pred}}}, \qquad \text{MFU}_{\text{meas}} = \frac{F_{\text{model}}}{C_{\text{peak}}T_{\text{meas}}}, \tag{15}$$

and forming either needs only an analytical count of useful model arithmetic and one execution time. Hardware counters explain where utilization is lost but are not required to compute the ratio. A cycle-accurate simulator supplies a *simulated* T, which we would report as such and not as a silicon measurement. For the present designs no executable implementation of the selected hardware/schedule pair exists, so we report MFU_{pred} against the recovered execution's modeled latency and never a measured figure. Two conventions must travel with any such number. The FLOP-counting rule, because padding, rematerialized arithmetic and inactive experts are charged in time but are not credited as useful work; and the precision of C_{peak}, which must match the arithmetic actually modeled, so an FP8-stored but BF16-computed path takes the BF16 capability rather than an FP8 peak. Where one multiply–accumulate counts as two FLOPs, numerator and denominator follow that convention together.

Relation to FAST. FAST reports *compute utilization*, defined there as a fraction of peak throughput, and neither names nor specifies MFU; its simulated TPU-v3 entry is 0.14 on EfficientNet-B7 [Zhang et al., 2022]. We therefore make no claim to reproduce a FAST MFU definition. The tightness ratios ρ_{comp} and ρ_{roof} are not that quantity and are not comparable to it: a utilization comparison would require one useful-work count and one peak convention applied to both arms.

The below audit verifies that all reported fixed-tile/joint-tile results are roofline-feasible: the LP-predicted latency never falls below the compute or memory lower bound, so the speedups are not artifacts of violating basic physical throughput limits.

Table 12: Single Workloads of Vizier-discovered hardware configuration. $\rho_{\text{comp}} = t_{\text{floor}}/T_{\text{LP}}$ is a compute-floor tightness ratio (see Eq. 12), not a measured hardware utilization and not the model FLOPs utilization of Appendix E.4. $\rho_{\text{comp}} = 1.000$ means the LP hit the mapper-derived compute floor. *Compute-bound*: $\text{obj} \approx t_{\text{compute}}$. *Memory-bound*: $\text{obj} \approx t_{\text{memory}}$, $\rho_{\text{comp}} < 1$. *Above-roofline*: obj exceeds both floors; expected for Static on hard cells, unexpected for fixed-tile/joint-tile. Two cells (Llama-3 70B and DeepSeek-V3 under Static) had no physically valid trial and are marked infeasible.

Method	Model	MACs	BW	Speedup	Obj cycles	Comp. floor	Mem. floor	ρ_{comp}	obj/lb	Binding	Verdict
Static FAST ILP											
Static	EN-B1	32,768	64	1.12	2.76×10^7	2.76×10^7	4.24×10^6	1.000	1.000	compute-bound	OK
Static	EN-B2	65,536	64	1.93	2.58×10^7	2.58×10^7	5.47×10^6	1.000	1.000	compute-bound	OK
Static	EN-B3	32,768	64	1.25	7.91×10^7	7.91×10^7	7.55×10^6	1.000	1.000	compute-bound	OK
Static	EN-B4	65,536	64	2.25	1.05×10^8	1.05×10^8	1.37×10^7	1.000	1.000	compute-bound	OK
Static	EN-B5	32,768	64	1.31	4.94×10^8	4.94×10^8	2.28×10^7	1.000	1.000	compute-bound	OK
Static	EN-B6	32,768	64	1.10	9.95×10^8	9.95×10^8	3.51×10^7	1.000	1.000	compute-bound	OK
Static	EN-B7	131,072	64	3.60	5.15×10^8	5.15×10^8	5.28×10^7	1.000	1.000	compute-bound	OK
Static	Llama-1 13B	32,768	1024	1.07	2.71×10^{10}	2.71×10^{10}	2.98×10^7	1.000	1.000	compute-bound	OK
Static	Llama-1 65B	65,536	128	0.61	2.38×10^{11}	6.74×10^{10}	9.51×10^8	0.28	3.53	above-roofline	loose (expected)
Static	Llama-3 70B	–	–	–	–	–	–	–	–	–	infeas
Static	DeepSeek-V3	–	–	–	–	–	–	–	–	–	infeas
Dynamic fixed-tile LP (time-indexed + rematerialisation)											
fixed-tile	EN-B1	65,536	64	2.24	1.38×10^7	1.38×10^7	4.24×10^6	1.000	1.000	compute-bound	OK
fixed-tile	EN-B2	131,072	64	3.87	1.29×10^7	1.29×10^7	5.47×10^6	1.000	1.000	compute-bound	OK
fixed-tile	EN-B3	131,072	64	4.99	1.98×10^7	1.98×10^7	7.55×10^6	1.000	1.000	compute-bound	OK
fixed-tile	EN-B4	131,072	64	4.51	5.23×10^7	5.23×10^7	1.37×10^7	1.000	1.000	compute-bound	OK
fixed-tile	EN-B5	131,072	64	5.24	1.23×10^8	1.23×10^8	2.28×10^7	1.000	1.000	compute-bound	OK
fixed-tile	EN-B6	262,144	64	8.82	1.24×10^8	1.24×10^8	3.51×10^7	1.000	1.000	compute-bound	OK
fixed-tile	EN-B7	524,288	64	14.39	1.29×10^8	1.29×10^8	5.28×10^7	1.000	1.000	compute-bound	OK
fixed-tile	Llama-1 13B	262,144	64	8.57	3.38×10^9	3.38×10^9	4.77×10^8	1.000	1.000	compute-bound	OK
fixed-tile	Llama-1 65B	262,144	64	8.57	1.69×10^{10}	1.69×10^{10}	1.90×10^9	1.000	1.000	compute-bound	OK
fixed-tile	Llama-3 70B	524,288	64	20.07	9.23×10^9	9.23×10^9	1.97×10^9	1.000	1.000	compute-bound	OK
fixed-tile	DeepSeek-V3	262,144	128	8.87	6.62×10^9	6.62×10^9	7.48×10^8	1.000	1.000	compute-bound	OK
Joint joint-tile LP (joint tiling + fusion)											
joint-tile	EN-B1	131,072	64	4.47	6.89×10^6	6.89×10^6	4.24×10^6	1.000	1.000	compute-bound	OK
joint-tile	EN-B2	262,144	64	7.73	6.44×10^6	6.44×10^6	5.47×10^6	1.000	1.000	compute-bound	OK
joint-tile	EN-B3	262,144	64	9.99	9.88×10^6	9.88×10^6	7.55×10^6	1.000	1.000	compute-bound	OK
joint-tile	EN-B4	262,144	64	9.01	2.61×10^7	2.61×10^7	1.37×10^7	1.000	1.000	compute-bound	OK
joint-tile	EN-B5	524,288	64	20.98	3.09×10^7	3.09×10^7	2.28×10^7	1.000	1.000	compute-bound	OK
joint-tile	EN-B6	524,288	64	17.64	6.22×10^7	6.22×10^7	3.51×10^7	1.000	1.000	compute-bound	OK
joint-tile	EN-B7	1,048,576	64	28.78	6.44×10^7	6.44×10^7	5.28×10^7	1.000	1.000	compute-bound	OK
joint-tile	Llama-1 13B	1,048,576	64	34.27	8.45×10^8	8.45×10^8	4.77×10^8	1.000	1.000	compute-bound	OK
joint-tile	Llama-1 65B	1,048,576	64	34.28	4.22×10^9	4.22×10^9	1.90×10^9	1.000	1.000	compute-bound	OK
joint-tile	Llama-3 70B	1,048,576	64	40.14	4.61×10^9	4.61×10^9	1.97×10^9	1.000	1.000	compute-bound	OK
joint-tile	DeepSeek-V3	1,048,576	64	35.50	1.66×10^9	1.66×10^9	1.50×10^9	1.000	1.000	compute-bound	OK
Summary: 31 cells plotted ($\rho_{\text{comp}} \leq 1$, roofline-feasible) · 2 cells skipped (no physically valid trial) · 0 $\rho_{\text{comp}} > 1$ regressions · 0 fixed-tile/joint-tile loose rows.											

Table 13: Roofline lower-bound audit (compute floor + memory-BW floor). Lower bound: $\mathrm{LB} = \max\left(\frac{\text{total FLOPs}}{2\times\text{MACs}},\ \frac{\text{total bytes}}{\text{BW}}\right)$. Memory floor: bytes moved (HBM $\leftrightarrow$ PEs, all activations + weights) divided by BW (GB/s), converted to cycles at 1 GHz clock. Compute floor: total FLOPs $/(2 \times$ total MACs$)$ in cycles. The LP objective (obj cycles) must satisfy obj $\geq$ LB. `roof_ratio` $= \mathrm{LB}/\mathrm{obj} \leq 1.001$ is the pass criterion. Summary: 31 cells plotted · 2 skipped (no physically valid trial) · 0 violations.

Method	Model	MACs	BW	FLOPs	Bytes	Comp. LB	Mem. LB	Roofline LB	Obj	Roof ratio	Bottleneck	Verdict
Static FAST ILP												
Static	EN-B1	32,768	64	1.81×10^{12}	2.71×10^{8}	2.76×10^{7}	4.24×10^{6}	2.76×10^{7}	2.76×10^{7}	1.000	compute	OK
Static	EN-B2	65,536	64	3.38×10^{12}	3.50×10^{8}	2.58×10^{7}	5.47×10^{6}	2.58×10^{7}	2.58×10^{7}	1.000	compute	OK
Static	EN-B3	32,768	64	5.18×10^{12}	4.83×10^{8}	7.91×10^{7}	7.55×10^{6}	7.91×10^{7}	7.91×10^{7}	1.000	compute	OK
Static	EN-B4	65,536	64	1.37×10^{13}	8.78×10^{8}	1.04×10^{8}	1.37×10^{7}	1.04×10^{8}	1.04×10^{8}	1.000	compute	OK
Static	EN-B5	32,768	64	3.24×10^{13}	1.46×10^{9}	4.94×10^{8}	2.28×10^{7}	4.94×10^{8}	4.94×10^{8}	1.000	compute	OK
Static	EN-B6	32,768	64	6.52×10^{13}	2.24×10^{9}	9.95×10^{8}	3.50×10^{7}	9.95×10^{8}	9.95×10^{8}	1.000	compute	OK
Static	EN-B7	131,072	64	1.35×10^{14}	3.38×10^{9}	5.15×10^{8}	5.28×10^{7}	5.15×10^{8}	5.15×10^{8}	1.000	compute	OK
Static	Llama-1 13B	32,768	1024	1.77×10^{15}	3.05×10^{10}	2.71×10^{10}	2.98×10^{7}	2.71×10^{10}	2.71×10^{10}	1.000	compute	OK
Static	Llama-1 65B	65,536	128	8.84×10^{15}	1.22×10^{11}	6.74×10^{10}	9.51×10^{8}	6.74×10^{10}	2.38×10^{11}	0.28	compute	OK
Dynamic fixed-tile LP (time-indexed + rematerialisation)												
fixed-tile	EN-B1	65,536	64	1.81×10^{12}	2.71×10^{8}	1.38×10^{7}	4.24×10^{6}	1.38×10^{7}	1.38×10^{7}	1.000	compute	OK
fixed-tile	EN-B2	131,072	64	3.38×10^{12}	3.50×10^{8}	1.29×10^{7}	5.47×10^{6}	1.29×10^{7}	1.29×10^{7}	1.000	compute	OK
fixed-tile	EN-B3	131,072	64	5.18×10^{12}	4.83×10^{8}	1.98×10^{7}	7.55×10^{6}	1.98×10^{7}	1.98×10^{7}	1.000	compute	OK
fixed-tile	EN-B4	131,072	64	1.37×10^{13}	8.78×10^{8}	5.22×10^{7}	1.37×10^{7}	5.22×10^{7}	5.22×10^{7}	1.000	compute	OK
fixed-tile	EN-B5	131,072	64	3.24×10^{13}	1.46×10^{9}	1.23×10^{8}	2.28×10^{7}	1.23×10^{8}	1.23×10^{8}	1.000	compute	OK
fixed-tile	EN-B6	262,144	64	6.52×10^{13}	2.24×10^{9}	1.24×10^{8}	3.50×10^{7}	1.24×10^{8}	1.24×10^{8}	1.000	compute	OK
fixed-tile	EN-B7	524,288	64	1.35×10^{14}	3.38×10^{9}	1.29×10^{8}	5.28×10^{7}	1.29×10^{8}	1.29×10^{8}	1.000	compute	OK
fixed-tile	Llama-1 13B	262,144	64	1.77×10^{15}	3.05×10^{10}	3.38×10^{9}	4.77×10^{8}	3.38×10^{9}	3.38×10^{9}	1.000	compute	OK
fixed-tile	Llama-1 65B	262,144	64	8.84×10^{15}	1.22×10^{11}	1.69×10^{10}	1.90×10^{9}	1.69×10^{10}	1.69×10^{10}	1.000	compute	OK
fixed-tile	Llama-3 70B	524,288	64	9.68×10^{15}	1.26×10^{11}	9.23×10^{9}	1.97×10^{9}	9.23×10^{9}	9.23×10^{9}	1.000	compute	OK
fixed-tile	DeepSeek-V3	262,144	128	3.47×10^{15}	9.57×10^{10}	6.62×10^{9}	7.48×10^{8}	6.62×10^{9}	6.62×10^{9}	1.000	compute	OK
Joint joint-tile LP (joint tiling + fusion)												
joint-tile	EN-B1	131,072	64	1.81×10^{12}	2.71×10^{8}	6.89×10^{6}	4.24×10^{6}	6.89×10^{6}	6.89×10^{6}	1.000	compute	OK
joint-tile	EN-B2	262,144	64	3.38×10^{12}	3.50×10^{8}	6.44×10^{6}	5.47×10^{6}	6.44×10^{6}	6.44×10^{6}	1.000	compute	OK
joint-tile	EN-B3	262,144	64	5.18×10^{12}	4.83×10^{8}	9.88×10^{6}	7.55×10^{6}	9.88×10^{6}	9.88×10^{6}	1.000	compute	OK
joint-tile	EN-B4	262,144	64	1.37×10^{13}	8.78×10^{8}	2.61×10^{7}	1.37×10^{7}	2.61×10^{7}	2.61×10^{7}	1.000	compute	OK
joint-tile	EN-B5	524,288	64	3.24×10^{13}	1.46×10^{9}	3.09×10^{7}	2.28×10^{7}	3.09×10^{7}	3.09×10^{7}	1.000	compute	OK
joint-tile	EN-B6	524,288	64	6.52×10^{13}	2.24×10^{9}	6.22×10^{7}	3.50×10^{7}	6.22×10^{7}	6.22×10^{7}	1.000	compute	OK
joint-tile	EN-B7	1,048,576	64	1.35×10^{14}	3.38×10^{9}	6.44×10^{7}	5.28×10^{7}	6.44×10^{7}	6.44×10^{7}	1.000	compute	OK
joint-tile	Llama-1 13B	1,048,576	64	1.77×10^{15}	3.05×10^{10}	8.45×10^{8}	4.77×10^{8}	8.45×10^{8}	8.45×10^{8}	1.000	compute	OK
joint-tile	Llama-1 65B	1,048,576	64	8.84×10^{15}	1.22×10^{11}	4.22×10^{9}	1.90×10^{9}	4.22×10^{9}	4.22×10^{9}	1.000	compute	OK
joint-tile	Llama-3 70B	1,048,576	64	9.68×10^{15}	1.26×10^{11}	4.61×10^{9}	1.97×10^{9}	4.61×10^{9}	4.61×10^{9}	1.000	compute	OK
joint-tile	DeepSeek-V3	1,048,576	64	3.47×10^{15}	9.57×10^{10}	1.66×10^{9}	1.50×10^{9}	1.66×10^{9}	1.66×10^{9}	1.000	compute	OK

E.3 Six Physical-Design Feasibility Gates

The prefill study applies six physical-design proxy gates to its candidate accelerators. The gate thresholds are calibrated end-to-end to reproduce TPU-v3's published headline numbers ($700\,\mathrm{mm}^2$ die, $250\,\mathrm{W}$ TDP, $0.36\,\mathrm{W/mm}^2$ density at $65{,}536$ MACs / $900\,\mathrm{GB/s}$ / $32\,\mathrm{MiB}$ CGM) when fed TPU-v3 inputs. Therefore, they are not arbitrary but pinned to a known-quality silicon.

(1) Pin count. Converts off-chip bandwidth into HBM3-class signal wires at $6.4\,\mathrm{Gb/s/pin}$ ($0.8\,\mathrm{GB/s/pin}$) and rejects designs needing more than $4{,}096$ high-speed pins, which is the practical ceiling for four HBM3 stacks on a single package. This prevents "free-bandwidth" LP solutions that silently assume 8+ stacks.

(2) Reticle area. Evaluates the linear area model

$$A_{\mathrm{die}} \;=\; \mathrm{MACs} \times 1.5 \times 10^{-4} \;+\; \mathrm{CGM_MB} \times 1.5 \;+\; \mathrm{dram_channels} \times 8 \;+\; 50 \quad [\mathrm{mm}^2]$$

(calibrated to a 7 nm process) and rejects any die exceeding $600\,\mathrm{mm}^2$. TSMC's N7 hard reticle is $858\,\mathrm{mm}^2$, and our design choice of 600 leaves headroom for yield, redundancy rings, and IO. This prevents over-scaled compute that would otherwise require monolithic chiplet packaging or multi-die exposure.

(3) Logic depth. Estimates the combinational gate-level depth of a single MAC-tree as

$$D \;=\; \lceil \log_2(\mathrm{sys_x} \times \mathrm{sys_y} \times \mathrm{MAC_bits}) \rceil + 8$$

and rejects depths above 35 levels. Beyond $\sim$35 levels, the critical path cannot meet a $1\,\mathrm{GHz}$ clock at TSMC N7 nominal V_{dd} without aggressive pipelining we do not model. This prevents designs whose systolic array is too wide for one cycle.

(4) Crossbar / NoC routing. Computes the cell count of the PE-to-SRAM interconnect as

$$N_{\mathrm{NoC}} \;=\; n_{\mathrm{PEs}} \times n_{64\,\mathrm{KB\ banks}}$$

and rejects values above 5×10^6 cells. The cap is the empirical budget for a 2-level hierarchical NoC (cluster-local crossbar + inter-cluster mesh) at modern AI-accelerator scales: flat all-to-all crossbars cap around 10^6 cells before routing dominates the floorplan, and we use the hierarchical-NoC cap with industry precedent (TPU-v4 sparse-core mesh, H100 SM-cluster L2 partitions, MI300X chiplet-local + XGMI, Cerebras WSE-3 per-tile fabric).

(5) Power density. Computes dynamic power as

$$P_{\mathrm{dyn}} \;=\; \mathrm{total_MACs} \times 1\,\mathrm{GHz} \times 0.5_{\mathrm{util}} \times 0.4\,\mathrm{mW/MAC}$$

(the $0.4\,\mathrm{mW/MAC}$ constant is calibrated against TPU-v3's $250\,\mathrm{W}$ envelope, and bumps to $1.2\,\mathrm{mW}$ for bf16), divides by the same linear area model from gate (2), and rejects anything above $2.0\,\mathrm{W/mm}^2$. This is the realistic ceiling for an AIO-cooled package today (TPU-v3 0.36, TPU-v4 0.55, H100 0.7, MI300X 0.6, Cerebras WSE-3 1.5). This prevents "thin-but-hot" tiny-MAC chips that would melt without exotic cooling.

(6) SRAM macro feasibility. Requires the global memory to decompose into a banked array of standard $64\,\mathrm{KiB}$ SRAM macros (the workhorse macro at 7 nm), caps the bank count at $4{,}096$ (the practical NoC routing limit), and requires the bank-array aspect ratio to lie within $[1{:}4,\ 4{:}1]$ so the macro grid can actually be laid out. A $17\,\mathrm{MiB}$ CGM organized as a 1×272 strip fails this even though the byte-count is fine.

Outcome. Original-campaign designs pass all six declared screens before reporting. These bound reticle area, interface pins, logic depth, interconnect size, power density, and SRAM organization under the stated models. They do not prove routed timing, package feasibility, or achievable whole-chip power. The native extension's additional external-memory interface is a separately stated architectural assumption.

E.4 Reported Efficiency Quantities

Three quantities carry the efficiency claims below, and each is read off the scheduled execution rather than assumed. Let T be the duration of the interval being reported, C_{peak} the peak arithmetic throughput of the recovered chip, b_r the installed delivered bandwidth of resource r, and P_{peak} its modeled peak power.

$$\text{MFU} = \frac{F_{\text{useful}}}{C_{\text{peak}} T}, \qquad \text{MBU}_r = \frac{B_{r,\text{read}} + B_{r,\text{write}}}{b_r T}, \qquad \text{Perf/TDP} = \frac{Q/T}{P_{\text{peak}}}. \tag{16}$$

Model FLOP utilization. MFU is the fraction of the chip's arithmetic capability that the schedule converts into useful work, where F_{useful} counts only the arithmetic the model requires. It is distinct from the work the arrays actually issue: padding, masked edge tiles and replicated operands are charged to the machine but are not useful, so $F^{\text{useful}} \le F^{\text{issued}}$ always. Within a fixed-chip branch the per-event service rows enforce $\sum_i F^{\text{issued}}_{i,e} x_{i,e} \le C_{\text{peak}} \Delta t_e$, which sums every concurrent implementation sharing the arrays in event e; because the installed rate is a branch constant there, summing over the event intervals guarantees an aggregate $\text{MFU} \le 1$ by construction rather than by inspection. This guarantee is a property of the fixed-chip rows and is not inherited by the variable-hardware relaxation, where the corresponding product is bilinear and its McCormick envelope is necessary but not exact (Appendix L.5); the installed-rate condition is therefore re-checked exactly at execution admission.

Model bandwidth utilization. MBU_r is the same ratio for a memory resource, and it is computed from the *complete* scheduled traffic: operand rereads, KV transfers between timesteps, and spill and restore movement all count, since each occupies the interface whether or not it advances the model's arithmetic. Because the resources have different peak rates and different consequences for a schedule, HBM, the host link and on-chip SRAM are reported separately rather than aggregated into a single figure.

Performance per TDP. Perf/TDP divides delivered throughput, Q units of work per unit time, by modeled peak power. Three conditions govern its use as a comparison. The process, the power scope and the unit of work must match between the designs compared; a ratio across differing scopes is not a ratio of efficiencies. Average measured power is not a substitute for modeled peak TDP, and we do not interchange them. And because the objective minimizes latency rather than this ratio, a performance/power tradeoff is obtained by solving across power budgets and comparing recovered designs: for a fixed amount of work the ratio factors as $(\text{Perf/TDP})_{\text{design}}/(\text{Perf/TDP})_{\text{baseline}} = (T_{\text{baseline}}/T_{\text{design}}) \cdot (\text{TDP}_{\text{baseline}}/\text{TDP}_{\text{design}})$, so a latency objective does not by itself maximize it.

For the current decode designs, performance per TDP uses a chip-power proxy because complete package TDP is unavailable. Matched prefill and whole-task efficiency triples have not been re-established for these decode-only chips, so no such values or cross-phase averages are claimed. Fractional objective bounds, replayed executions and physical qualification remain distinct.

F Speedup Analysis: Baseline Deficit Attribution and Design-Parameter Sensitivity

A large end-to-end speedup number is only credible if two things are shown explicitly: (i) the naive baseline actually *carries* that deficit, and the deficit can be *localized* to a physical cause; and (ii) the reported gain can be *decomposed* into a small number of design decisions, each of which is separately sized. This appendix supplies both. We decompose the 34.6–$40.2\times$ co-design speedup of the prefill-oriented designs into exactly two orthogonal factors (Appendix F.1) and sweep each design decision one at a time, reporting the objective's sensitivity (Appendix F.2). This section is a companion to the pre-RTL feasibility gates of Appendix E: the gates certify that a design is *buildable*; this section certifies where its speedup *comes from*.

Reading conventions (read before any number below). Throughout, every quantity written **MFU** in this appendix is the **compute-floor tightness** ratio $\rho_{\text{comp}} = t_{\text{floor}}/T_{\text{LP}}$ of Appendix E.2 (the mapper-derived compute floor over the LP/analytic latency within one cost-model convention). It is *not* model-FLOP utilization in the sense of Appendix E.4, because its numerator is a Timeloop tile minimum carrying the reference geometry's padding rather than a useful-work count, and it is *never* a measured-silicon utilization; the searched datapaths are hypothetical, not fabricated. **Same-silicon** comparisons are equal-area and equal-TDP *by construction* (both sides share the PE grid, so the area/power model of Appendix E.3 returns identical budgets). The 97% pre-fusion memory-stall figures and the naive MFU values are *naive-baseline attributions*, not properties of the co-designed schedule. Every quantity is convention-tagged, and we never form a ratio across two conventions. The fixed-silicon algorithmic gain on EfficientNet-B7 is $2.21\times$ under this paper's naive convention (a $2.81\times$ value that appears in an earlier discussion note is a transcription error and is not used).

F.1 Two-Factor Speedup Decomposition

The headline speedup factors *exactly* into two orthogonal terms:

$$\underbrace{\frac{T_{\text{naive}}(\text{TPU-v3})}{T_{\text{joint}}(\text{chip})}}_{\text{headline}} = \underbrace{\frac{T_{\text{naive}}(\text{TPU-v3})}{T_{\text{joint}}(\text{TPU-v3})}}_{f_1\text{: scheduling recovery}} \times \underbrace{\frac{T_{\text{joint}}(\text{TPU-v3})}{T_{\text{joint}}(\text{chip})}}_{f_2\text{: MAC provisioning}}. \tag{17}$$

The first factor f_1 is a *fixed-silicon*, same-area / same-TDP algorithmic gain: it re-schedules the same $65{,}536$-MAC TPU-v3 and buys $\mathbf{1.48}$–$\mathbf{1.78}\times$. The second factor f_2 holds the joint schedule fixed and provisions the PE grid up to the co-designed chip's MAC count, buying $\mathbf{20.0}$–$\mathbf{26.9}\times$. Their product reproduces the per-workload Figure-2 headline (and the blueprint "achieved speedup vs. naive TPU-v3" rows) to within 1.4% (Table 14); the residual is the analytic chip-endpoint solve's utilization roll-off, cross-checked against the blueprints' Timeloop-in-loop objective.

Table 14: Two-factor decomposition of the co-design speedup (Eq. (17)). f_1 is the fixed-silicon scheduling gain on TPU-v3 (same area/TDP), f_2 the MAC-provisioning gain to the co-designed chip. Reconstructed $= f_1 \times f_2$; headline is the blueprint value. Convention: config-0 naive; bf16 cycle basis; HiGHS-Optimal endpoints.

Workload	f_1 (sched., TPU-v3)	f_2 (MAC prov.)	Reconstructed	Headline	Recon./Headline
Llama-1 13B	1.66	20.31	33.75	34.22	0.99
Llama-1 65B	1.73	20.04	34.60	34.60	1.000[‡]
Llama-3 70B	1.48	26.85	39.84	40.16	0.992
DeepSeek-V3	1.78	19.97	35.63	35.68	0.999
Range	**1.48–1.78**	**20.0–26.9**	**34.6–40.2** (prefill-oriented designs)		

[‡]Llama-1 65B's analytic chip-endpoint solve was degenerate under HiGHS time limits, so its f_2 uses the authoritative blueprint objective (exact by construction); the other three use the *independent* analytic chip solve and land within 1.4%.

Is the provisioning factor physically usable? A reader is entitled to ask whether f_2 is real parallelism or an artifact of a cost model in which compute time falls as $1/\text{MACs}$ without bound. It is a fair question: an array can only speed an operation up as far as that operation supplies independent tiles to fill it, and a per-token decode GEMV supplies few. We measured the answer

directly rather than argue it. For every event of DeepSeek-V3 we computed the *saturating* MAC count $M_{\text{sat}} = (\text{independent output tiles}) \times (\text{lanes per tile})$ under the same tile geometry the mapper uses, where independent tiles are row tiles times column tiles (k-tiles accumulate into one output and are not independent) times the number of GEMMs carrying work (expected active experts for routed FFNs). Beyond M_{sat} a chip has no more concurrent tiles to run for that event, and its compute time cannot keep falling. Table 15 gives the distribution in multiples of the TPU-v3 array.

Table 15: Saturating MAC count per event, DeepSeek-V3, as a multiple of TPU-v3's 65,536 MACs. Prefill events saturate three to four orders of magnitude above the largest chip in this paper ($24.4\times$), so the provisioning factor f_2 is physically usable parallelism at prefill. Decode events saturate at 9–$14\times$ at the low end, which is where a bound on usable parallelism would bind, and only for chips beyond it.

Regime	Phase	min	p10	median
B=8, ctx 4096	prefill	3,584×	3,584×	16,384×
	decode	9×	14×	16×
B=32, ctx 4096	prefill	14,336×	14,336×	65,536×
	decode	14×	14×	32×
B=32, ctx 512	prefill	1,792×	1,792×	8,192×
	decode	14×	14×	32×

At the original prefill operating point, the table's tile geometry supplies many independent matrix tiles, so useful parallelism is less restrictive than it is for small decode matrices. The bound becomes relevant when installed arrays exceed the independent work available to an event. These counts describe the mapper's geometry, not placed-array timing, and do not account for growing non-matrix costs. The native full-inference model instead charges padded matrix service and explicit vector, local, and external service for each implementation (Section 6.4); its results must be read under that contract.

Use only the two-factor form; do not multiply the full decision ladder. It is tempting to read the software-ablation ladder (tile, placement, remat, datapath, datatype) as a chain of multiplicative factors and multiply them all. That *overshoots* the headline by 1.8–$2.1\times$ (a mechanical product of 70.7–$73.2\times$ versus the 34–$40\times$ headline), for two convention reasons, neither of them physics:

1. **The INT8 throughput label double-counts.** The blueprint headline T_{joint} is a *bf16 cycle count* at the chip's MAC count; INT8 (W8A8) is carried as a throughput / power label (roughly half the area and power per MAC), *not* as a separate cycle factor. Multiplying an extra $2\times$ for INT8 into a cycle product is a double-count. Dropping it lands the mechanical ladder at 35–$37\times$, within 3–12% of the headline.
2. **The software rows use a $\sim 1.3\times$ smaller naive baseline** (the scale-$T_{\max}$ naive) than the config-0 naive against which the headline is measured, so the two families of factors are not on the same numerator.

Equation (17) is the form that reconciles *exactly* because f_1 and f_2 share a single naive numerator and a single cycle basis. As a naming caution: the same fixed-silicon scheduling recovery is 1.48–$1.78\times$ against the config-0 naive (used above) and 2.14–$2.51\times$ against the grid's scale-$T_{\max}$ naive (the fixed-hardware grid convention); the two differ only by the naive-baseline definition, and must never be crossed within one ratio.

The provisioning factor is sub-linear (the anti-"just add MACs" result). f_2 is not free scaling. Measured as delivered throughput per provisioned MAC in an equal-area / equal-TDP resource-matched ablation, 20.0–$27.0\times$ more MACs realize only $\approx 16\times$ provisioning throughput, a linearity of $\mathbf{0.59}$–$\mathbf{0.81}$ (Llama-3 70B is worst, $27\times$ MACs $\rightarrow 16.0\times$). Equivalently, the fixed-family MAC leg from 65,536 to 1,048,576 MACs (a $16\times$ increase) yields only 14.0–$15.7\times$ cycle reduction, because a residual memory floor remains. Added silicon buys sub-linearly, which is exactly why capturing the free scheduling factor f_1 first (and provisioning MACs only up to the point of diminishing return) is the efficient order of operations.

F.2 One-Decision-at-a-Time Sensitivity Sweeps

We now hold the design fixed and sweep a single decision, reporting the objective (LP cycles) sensitivity. Two decisions have dedicated measured sweeps: global buffer and MAC count; Table 17 summarizes them.

(a) Global-buffer size C_{GM}: measured, near-inert in the compute-bound regime. Table 16 sweeps C_{GM} from 1 to 64 MiB for DeepSeek-V3 on TPU-v3 (900 GB/s), for the joint (free-y) LP and for a fixed-tile FAST-subset LP. The joint LP is *flat*: its objective moves 0.39% across a $64\times$ buffer sweep ($1.0723 \times 10^{12} \rightarrow 1.0682 \times 10^{12}$ cycles), and is bit-identical from 8 MiB upward. By contrast the fixed-tile schedule *needs* the buffer, improving $1.18\times$ over the same sweep. The joint formulation substitutes tile choice for buffer capacity: when tiling is free, a larger global buffer buys almost nothing. The tile-dependent capacity extension is likewise inert here: it matches the retained joint model to within 0.08% across the full $C_{\mathrm{GM}} \in \{4, \ldots, 64\}$ MiB sweep. Under bandwidth pressure (64–128 GB/s) the buffer regains a modest decision ($\approx 1.19\times$ for Llama-1 13B over $8 \rightarrow 64$ MiB), but *only through placement*: with placement frozen (row A) the objective is constant across C_{GM}, so C_{GM} helps exclusively by enabling time-indexed residency, never on its own. The blueprints adopt $C_{\mathrm{GM}} = 64$ MiB.

Table 16: Global-buffer sweep, DeepSeek-V3 on TPU-v3 (900 GB/s, native cache convention, HiGHS-Optimal). The joint (free-y) LP is essentially C_{GM}-insensitive (0.39% over a $64\times$ sweep); the fixed-tile FAST-subset LP is not ($1.18\times$). Objective in LP cycles.

C_{GM} (MiB)	1	2	4	8	16	32	64	sweep ($1 \rightarrow 64$)
Joint (free-y)	1.0723	1.0699	1.0690	1.0682	1.0682	1.0682	1.0682	$\times 10^{12}$; $1.004\times$
Fixed-tile (FAST-subset)	1.4627	1.4459	1.4327	1.4161	1.3863	1.3364	1.2371	$\times 10^{12}$; $1.18\times$

(b) Peak throughput / MAC-unit count. The total-MAC provisioning sweep is measured: raising the PE grid from 65,536 to 1,048,576 MACs (a $16\times$ increase) reduces the objective by 14.0–$15.7\times$ (Llama-1 13B $15.72\times$, DeepSeek-V3 $14.98\times$, Llama-3 70B $14.04\times$, Llama-1 65B $14.02\times$), and the provisioning factor to the full co-designed chip is $f_2 = 20.0$–$26.9\times$ (Appendix F.1), both sub-linear (§F.1). This is the single largest decision on the objective.

(c) DRAM bandwidth: measured, inert in the compute-bound regime. Sweeping bandwidth from 900 to 3,350 GB/s at the fixed 65,536-MAC point moves the objective by $\leq 0.25\%$ (BW leg $= 1.000$–$1.0025\times$ for all four LLMs): at the deployed prefill-like batch these workloads are compute-bound, so bandwidth beyond the floor buys nothing. Bandwidth becomes the dominant decision only in the bandwidth-starved regime: at 64 GB/s (the compact NPU) the joint-vs-naive fixed-hardware gain is 5.9–$31.4\times$, because there the roofline is memory-bound and residency / placement recover a much larger share. Thus bandwidth sensitivity is regime-dependent: near-zero when compute-bound, first-order when memory-bound.

(d) Interconnect: on-chip NoC and inter-device topology. *On-chip*, the NoC / crossbar radix and link count set the PE-to-SRAM routing capacity; this is the resource bounded by the crossbar / NoC-routing gate of Appendix E.3 (gate 4, $N_{\mathrm{NoC}} = n_{\mathrm{PEs}} \times n_{\mathrm{banks}} \leq 5 \times 10^6$ cells), and its provisioning follows the network-on-package accounting of Simba [Shao et al., 2019]. In the single-chip compute-bound regime the on-chip NoC is provisioned to the compute floor and is not the binding resource. *Across devices*, in the multi-GPU / mixture-of-experts regime the inter-device topology (radix and link count of the NVLink-class fabric) binds only when experts are dispatched *across* devices; when the active-expert set is package-resident, as in every blueprint here, the communication term is zero.

Table 17: Single-decision sensitivity summary (compute-bound LLM regime, TPU-v3 operating point). "Objective sensitivity" is the LP-cycle response to the stated sweep.

Decision	**Sweep**	**Objective sensitivity**
Global buffer C_{GM}	$1 \to 64$ MiB (BW$= 900$)	$1.004\times$ (joint); $1.18\times$ (fixed-tile)
	under BW pressure (≤ 128)	$\approx 1.19\times$, via placement only
MAC count (total)	$65{,}536 \to 1{,}048{,}576$ ($16\times$)	14.0–$15.7\times$ (sub-linear); $f_2 = 20.0$–$26.9\times$
DRAM bandwidth	$900 \to 3{,}350$ GB/s	$\leq 1.003\times$ (compute-bound)
	64 GB/s (NPU)	5.9–$31.4\times$ (memory-bound)

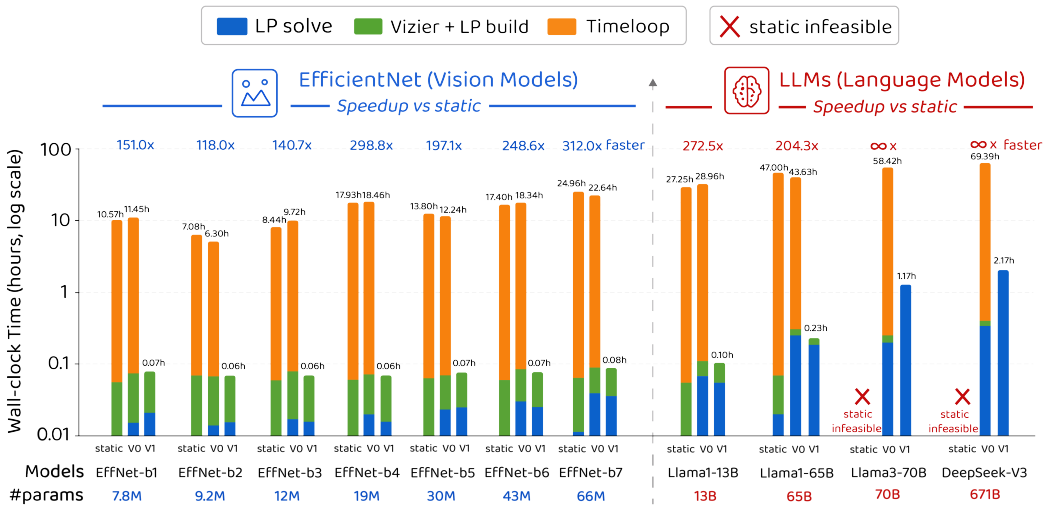

Figure 4: Wall-clock time decomposition and performance acceleration across formulations: CHEETAH-V1's internalized tiling eliminates the Timeloop bottleneck, achieving up to 100–300× faster search than the classical static method. Upper bars show wall-clock hours (log scale); annotations show speedup vs. static.

G Hardware Search and Search Cost

Wall-clock time decomposition and search acceleration results are presented in Figure 4 and Section 6.

G.1 Hardware and Vizier Search

The prefill co-design search used 16 NVIDIA H100 GPUs, each with 80 GB of VRAM, and Vizier for hardware search following the experimental conventions of Zhang et al. [2022]. Table 18 records that campaign's parameter groups; Figure 5 depicts its accelerator template. The native full-inference extension is solved with OCS, whose discovery kernel and certified closing stage are described in Section 5 and Appendix M.1, with a separate finite hardware-search path; it is not an all-H100/Vizier experiment.

A Processing Element (PE) is a repeated compute tile in an accelerator template. The datapath can be modeled as a grid of PEs connected by a mesh on-chip network. PE systolic arrays perform a matrix-vector multiply each cycle. Vector and scalar MACs are modeled by setting systolic array X and Y dimensions to 1. PEs also contain a VPU for non-MAC vector ops. L2 and Global Memory structures are optional.

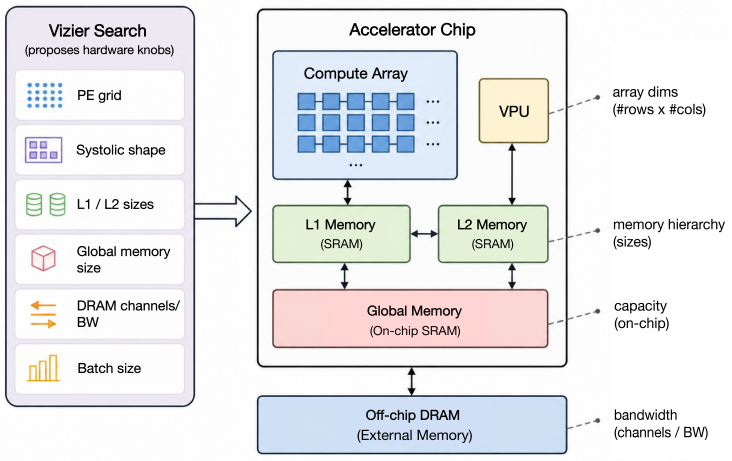

Figure 5: The accelerator datapath configuration. Processing elements (PEs) are arranged in a grid connected by a mesh network. Each PE contains a systolic array for matrix operations, a vector processing unit (VPU) for non-MAC operations, and private or shared L1/L2 memory. A Global Memory serves as the on-chip scratchpad shared across all PEs.

Table 18: Vizier search parameter groups and ranges: initial small search vs. expanded joint-tile-wide search.

Group	Parameter	Small (initial)	LLM-joint-tile-wide
A: Hardware datapath (16 params)	`PEs_x_dim`	pow2 in **4** … **32**	pow2 in **1** … **256**
	`PEs_y_dim`	pow2 in **4** … **32**	pow2 in **1** … **256**
	`Systolic_array_x`	pow2 in **4** … **32**	pow2 in **4** … **128**
	`Systolic_array_y`	pow2 in **4** … **32**	pow2 in **4** … **128**
	`Vector_unit_multiplier`	pow2 1 … 16	pow2 1 … 16
	`L1_buffer_config`	{Private, Shared}	{Private, Shared}
	`L1_input_buffer_size`	pow2 **1** KB … **4** MB	pow2 **1** KB … **1** MB
	`L1_weight_buffer_size`	pow2 **1** KB … **4** MB	pow2 **1** KB … **1** MB
	`L1_output_buffer_size`	pow2 **1** KB … **4** MB	pow2 **1** KB … **1** MB
	`L2_buffer_config`	{Disabled, Private, Shared}	{Disabled, Private, Shared}
	`L2_input_buffer_mult.`	pow2 1 … 128	pow2 1 … 128
	`L2_weight_buffer_mult.`	pow2 1 … 128	pow2 1 … 128
	`L2_output_buffer_mult.`	pow2 1 … 128	pow2 1 … 128
	`L3_global_buffer` (CGM)	{0}∪ pow2 **1** … **64** MB	{0}∪ pow2 **1** … **256** MB
	`GDDR6_channels`	pow2 **1** … **16** (→ 64–1024 GB/s)	pow2 **1** … **32** (→ 64–2048 GB/s)
	`Native_batch_size`	pow2 **1** … **1024**	pow2 **1** … **256**
B: Compiler passes (3 params)	`two_pass_softmax`	{disabled, enabled}	{disabled, enabled}
	`tensor_padding`	{disabled, enabled}	{disabled, enabled}
	`datatype`	{bfloat16, int8}	{bfloat16, int8}
C: fixed-tile rematerialisation (2 params)	`remat_enabled`	{disabled, enabled}	{disabled, enabled}
	`remat_cost_scale`	{0.1, 0.25, 0.5, 1.0, 2.0, 4.0}	{0.1, 0.25, 0.5, 1.0, 2.0, 4.0}
D: joint-tile joint-tiling (4 params)	`pareto_config_count_C`	{10, 50, 100, 250, 500, 1000}	{10, 50, 100, 250, 500, 1000}
	`loop_ordering_strategy`	{weight_stationary, output_stationary, both}	{weight_stationary, output_stationary, both}
	`pareto_pruning_strategy`	{tmax_workingset, tmax_tmin, tmin_workingset}	{tmax_workingset, tmax_tmin, tmin_workingset}
	`tile_workingset_modeling`	{fixed, config_dep}	{fixed, config_dep}
Post-Vizier gates (filter, not search)	`MIN_TOTAL_MACS`	32,768 (4×4×4×4=256 lower physical, 32K gate)	
	`MAX_TOTAL_MACS` cap	none: implicit ≤ 32×32×32×32 ≈ 1 M	1,048,576 explicit cap
	Implied total_MACs reach	≈ 256 … 1 M	256 … 1,048,576 (post-cap)
	Implied DRAM BW reach	64 … 1,024 GB/s	64 … 2,048 GB/s

H LLM-Architect Outer Loop Details

The LLM-Architect outer loop proceeds through the following steps for each campaign round:

1. **Extraction.** Deterministic scripts summarize recent trials into a *Causal Ledger*. Each record includes hardware parameters, validity, Perf/TDP, LP cycles, capacity duals/slacks, bandwidth sensitivity, MFU, memory utilization, roofline gap, failure class, and LP solver metrics.
2. **Context building.** The last N trials and the Causal Ledger are compressed into a compact JSON context for the Strategist. The context emphasizes empirical feasibility, observed invalid regions, best known valid designs, and physical audit signals.
3. **Architect.** The online Architect, implemented as Qwen2.5-7B-Instruct with a LoRA adapter trained on past trial logs, diagnoses the dominant bottleneck and emits a campaign patch. The patch constrains macro-architectural search decisions such as Global Memory, DRAM bandwidth, total MACs, systolic shape, PE grid, and batch size. A larger teacher model, DeepSeek-V3.2, was used offline to generate or critique campaign labels, but is not in the live loop.
4. **Campaign validation.** A deterministic validator clips or rejects the LLM patch to enforce legal hardware ranges, TDP and area limits, MAC caps, CGM feasibility floors, compute/bandwidth coupling, and roofline constraints. This prevents the LLM from proposing physically implausible regions such as maximum-MAC/minimum-bandwidth designs.
5. **Local optimization.** Vizier is initialized inside the validated patch and warm-started with prior trials that lie in the same region. Vizier then samples concrete hardware candidates within the approved campaign. The LLM chooses the high-level architectural neighborhood, while Vizier performs local adaptive optimization.
6. **Unified joint-tile Formulation.** The large-scale LP solver evaluates each candidate by solving the joint-tile joint tiling-fusion-placement LP using saved Pareto $\mathcal{C}$-sets. It returns the objective, cycles, selected tiling configurations, placement/rematerialization decisions, dual/slack summaries, and solver-quality diagnostics.
7. **Roofline audit.** A deterministic Roofline Auditor checks whether the predicted latency is physically plausible. For each candidate, we compute

$$t_{\text{compute}} = \frac{\text{useful FLOPs}}{\text{peak FLOPs/s}}, \qquad t_{\text{memory}} = \frac{\text{post-placement DRAM bytes}}{\text{peak bandwidth}},$$
$$t_{\text{ideal}} = \max(t_{\text{compute}},\, t_{\text{memory}})\,. \tag{18}$$

   A candidate is rejected if the solver-reported latency falls below this lower bound, or if MFU or memory utilization exceeds 1.0 within tolerance.
8. **Ledger update.** The loop records the campaign hypothesis, intervention, observed invalid-rate change, MFU change, roofline validity, and best Perf/TDP change. The ledger also records forbidden regions when a campaign produces excessive invalid trials.
9. **Shortlist Pareto refinement.** At the end of a campaign or study, only the top-ranked candidates are passed to a Pareto refinement run. This reruns targeted Timeloop refinement for selected configurations and re-solves joint-tile LP, providing a final trust check. This is an additional and optional sanity check step.

The Architect receives workload summaries, invalid-rate history, MFU, roofline signals, and prior valid regions, normalized LP duals and bandwidth sensitivities. The system starts from a safe empirically valid patch and allows duals only to make bounded edits, such as raising the CGM floor, increasing bandwidth, reducing MAC count, or changing systolic shapes. The validator rejects any dual-guided patch that excludes known high-performing valid regions or predicts a substantially worse feasible rate.

H.1 Profile-Conditioned LLM-Architect Application

We evaluate if the Architect can condition hardware search on deployment intent, since speech models may prioritize latency while data analytics models may prioritize bandwidth. For each workload,

we run the same Qwen2.5-LoRA Strategist, validator, random-in-patch sampler, and joint-tile LP evaluator while changing only a two-paragraph profile block describing the user's objective and constraints. The resulting best chips differ systematically by profile: overnight batch profiles select higher-bandwidth and larger-CGM regions, compact-edge profiles select smaller feasible regions with the lowest invalid rate, and interactive profiles favor small-CGM, extreme-aspect datapaths. Since the inner joint-tile LP and validator are identical across profiles, these differences isolate the effect of the LLM's profile-conditioned search patch. The absolute Perf/TDP often remains within a narrow band because the joint-tile evaluator reaches a similar structural ceiling, but the hardware families used to reach that ceiling differ substantially. We simulate two user profiles: (1) interactive speech is a spoken dialogue model where latency is more important than bandwidth, and (2) overnight finance batch where the user requires a large amount of financial data to be processed, but might leave it overnight to complete hence bandwidth is more important than latency.

The LLM-Architect clearly learned the profile distinction. Different profiles select meaningfully different chips for the same workload. The signal flows through the LLM's patch choice (steered by the profile prompt), and random-in-patch sampling finds the best chip within each profile-narrowed region. Table 19 gives examples of best designs per profile.

Table 19: Best hardware configuration discovered per (deployment profile, workload) cell. PE SA = PE grid $\times$ systolic array dimensions. MACs = total multiply-accumulate units. BW = DRAM bandwidth (GB/s). CGM = Global Buffer size (MB). Interactive speech = small low-latency-style chip, overnight finance batch = high-throughput, high-bandwidth, compact edge = compact constrained chip, balanced cloud = moderate/general design.

Workload	Profile	PE grid	Systolic	MACs	BW (GB/s)	CGM (MB)	Best Perf/TDP $\times$
Llama-1 13B	`interactive_speech`	1×128	16×16	32K	512	2	2.06
	`overnight_finance_batch`	16×8	128×32	524K	2048	256	2.73
	`compact_edge`	8×2	64×32	32K	512	4	2.06
	`balanced_cloud`	2×256	8×16	65K	64	128	2.97
DeepSeek-V3	`interactive_speech`	1×32	8×128	32K	2048	16	1.07
	`overnight_finance_batch`	8×256	16×8	262K	2048	1	2.55
	`compact_edge`	16×64	16×4	65K	128	2	2.99
	`balanced_cloud`	256×2	32×16	262K	1024	32	2.83

- `interactive_speech` repeatedly selects extreme-aspect PE arrays (1×32, 1×128, 1×256, 2×128); these favour narrow systolic streaming for low single-batch latency. CGM is consistently small (2–4 MB), trading cache capacity for compute density. Matches the profile intent: "minimize end-to-end latency, batch ≤ 8".

- `overnight_finance_batch` consistently selects high bandwidth (1024–2048 GB/s) and often large CGM (16–256 MB). Matches the profile intent: "maximize throughput, large batch, sustained bandwidth".

- `compact_edge` consistently selects the smallest chips (32–65K MACs is common vs. 524K for other profiles) with tiny CGM (1–4 MB). Matches the profile hard constraints: TDP $\leq 0.5\times$, area $\leq 0.5\times$, mobile deployment target.

- `balanced_cloud` selects medium-everything (middle-of-the-road MACs, bandwidth, and CGM), consistent with its unconstrained balanced objective.

Invalid-Rate Signal The absolute Perf/TDP $\times$ values are similar within a workload across profiles (e.g. Llama-3 70B: 3.38–3.53 across all four profiles, within $\approx 5\%$). This is not a failure: it reflects the LP's structural ceiling at fixed compute. *What differs is the chip family used to reach that ceiling.* This is precisely the design-space-narrowing benefit of profile conditioning: the same headline Perf/TDP number is achieved by hardware families with qualitatively different deployment characteristics (narrow streaming chips for interactive speech, large-bandwidth chips for overnight batch), demonstrating that profile conditioning the LLM-Architect steers the search toward different hardware trade-offs without sacrificing overall efficiency.

Table 20: Average invalid rate per deployment profile. The LLM responds to profile constraints: tighter profiles produce more LP-rejected proposals, while compact profiles are more feasible despite strict TDP/area limits.

Profile	Avg. invalid rate	Interpretation
`interactive_speech`	62%	Tightest profile (TDP $\leq 1\times$, batch ≤ 8): LLM proposes patches the LP frequently rejects.
`overnight_finance_batch`	53%	Loose TDP ($4\times$) but `min_batch` ≥ 32 forces large-batch hardware.
`compact_edge`	8%	Despite strict TDP/area constraints, the LLM finds feasible compact regions easily: best feasibility of all profiles.
`balanced_cloud`	36%	Middle-ground constraints produce intermediate invalid rate.

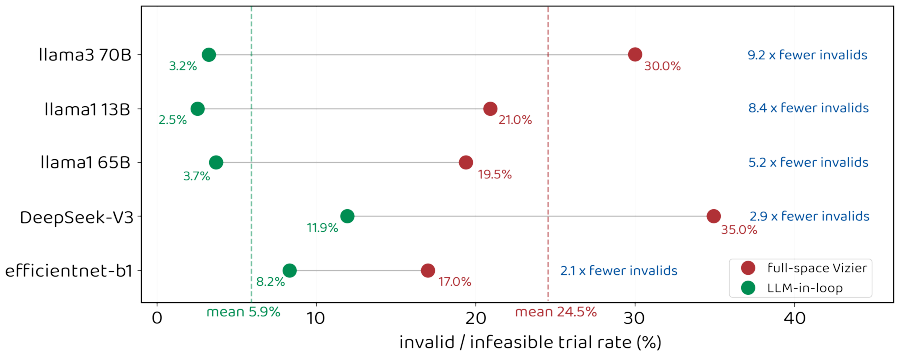

Figure 6: Invalid/infeasible trial rate (%) for full-space Vizier vs. LLM-in-loop across five workloads. The LLM Architect reduces the mean invalid rate from 24.5% to 5.9% ($4.1\times$ reduction), with the largest gains on LLM workloads where the hardware search space is most prone to infeasible configurations.

H.2 LLM in the Loop Prompts

The Strategist System Prompt

You are the Strategist agent in a hardware/software co-design loop.

Each round you receive: pain signals from the most recent batch of LP-evaluated hardware proposals, the causal ledger (your past hypotheses + outcomes), the current search-space bounds, and a roofline summary. You return **ONE JSON object** describing a campaign that narrows the search space for the next round.

You are a *campaign-level search-space editor*, not a performance predictor. Your job is to point Vizier at a small, physically-plausible neighborhood; the deterministic Validator + roofline auditor enforce hard constraints.

Important facts about the evaluator (CORGIsolve joint-tile). For each fixed hardware candidate, joint-tile already *jointly* optimizes:

- per-layer tiling selection $y_{i,c}$ over precomputed Pareto tiling sets $\mathcal{C}_i$
- time-indexed activation and weight placement
- rematerialization decisions
- tiling $\times$ placement interactions via McCormick variables
- fusion / DRAM-traffic reduction under the candidate's memory capacity

Therefore: do **not** assume low Perf/TDP is caused by a missing tiling search. The inner LP is already choosing the best available tiling/fusion/placement plan for the candidate. Your job is **only** to edit the *outer* hardware search region.

Pain signal keys.

- `invalid_rate`: fraction of round's trials not OK
- `mfu_med`, `memory_util_med`: median compute / memory utilization (≤ 1)
- `gap_to_floor_med`: median $t_{\text{solver}}/t_{\text{ideal}}$ (≥ 1; near-1 = at floor)
- `op_intensity_med`: median FLOPs/byte for OK trials
- `ridgepoint_med`: median chip ridgepoint (FLOPs/byte)
- `capacity_dual_p95`, `bandwidth_dual_p95`, `macfloor_dual_p95`: LP duals
- `best_perf_tdp_so_far`: running max Perf/TDP across the search
- `recent_roofline`: `{violation_rate, dominant_violation, ...}`

Causal reading.

1. `capacity_dual` high *or* failures are `global_memory_infeasible` $\Rightarrow$ increase CGM or restrict to configs with enough global memory. Do **not** blindly increase MACs.
2. `memory_util` high, `gap_to_floor` low, or `op_intensity_med` $<$ `ridgepoint_med` $\Rightarrow$ increase bandwidth, *reduce* MAC count, or avoid max-MAC/min-BW regions.
3. MFU low *and* `memory_util` also low $\Rightarrow$ compute shape / systolic mismatch. Prefer different-shaped systolic arrays, **not** more total MACs.
4. Perf/TDP plateaued $\Rightarrow$ propose an `efficiency_shrink` campaign that keeps $\geq 95\%$ of best Perf/TDP while reducing TDP / area / invalid rate.
5. Previous campaign had `invalid_rate` $> 50\%$ $\Rightarrow$ do **not** repeat or further narrow into that region. Shift to the nearest empirically valid region.
6. *Empirical feasibility beats unverified dual reasoning.* Never choose a patch that excludes the best known valid configurations unless explicitly running a small `explore` campaign.

Duals are advisory, not absolute. If dual-based reasoning conflicts with empirical feasibility, prioritise empirical feasibility.

Anti-gaming policy (Validator enforces; obeying saves a wasted round):

- Do **not** propose the entire global search space; every patch must be a strict subset ($\leq 10\%$ volume by default; $\leq 20\%$ if `campaign_type` is `explore`).
- Do **not** pair max `total_macs` with min `GDDR6_channels`: ridgepoint cap 1000 FLOPs/byte.
- Do **not** propose tiny CGM (≤ 1 MB) for LLM workloads. Long-context workloads need ≥ 16 MB CGM (≥ 32 MB for 65B+).
- Do **not** re-propose regions marked forbidden in the ledger.

Patchable parameters (each must remain a non-empty subset of original powers-of-two): `PEs_x_dim`, `PEs_y_dim`, `Systolic_array_x`, `Systolic_array_y`, `Vector_unit_multiplier`, `L1_{input,weight,output}_buffer_size`, `L2_{input,weight,output}_buffer_multiplier`, `L3_global_buffer_size` (CGM in MB), `GDDR6_channels`, `Native_batch_size`.

Output: one JSON object on one line. No prose, no markdown fence.

`campaign_type` **semantics:**

- `exploit`: narrow region around best valid designs so far
- `repair`: targeted intervention for the dominant bottleneck
- `explore`: diverse region not covered by prior failed campaigns (validator allows up to 20% of global space)

- `efficiency_shrink`: Perf/TDP saturated; find a *smaller* chip that keeps $\geq 95\%$ of best Perf/TDP while reducing TDP / area / invalid rate

If no actionable hypothesis exists: return `campaign_name = "explore_current"`, `campaign_type = "explore"`, empty `search_patch`, `dominant_bottleneck = "invalidity"`, and reasonable prediction fields.

Teacher/Critic System Prompt

You are the **teacher/critic** for a hardware/software co-design search loop.

You are given:

- a *Trial Digest* (pain signals from a campaign round)
- a *deterministic oracle patch* (search-space narrowing that empirically worked in the next round of a previous run)
- the student-style template

Your task is to produce a **cleaned training example**. The patch and prediction **must** stay grounded in the deterministic oracle: do **not** invent new bounds. You may:

- rewrite `causal_hypothesis` to be precise and cite the dominant signal
- tighten `dominant_bottleneck` to the right category
- quantify `prediction` deltas (e.g. "drop below 0.25", "increase by 0.10")
- normalize `campaign_name` to `snake_case`
- reject the example by returning `{"keep": false, "reason": "..."}` if the deterministic oracle is internally inconsistent or unsafe

Return **one JSON object on one line**. Schema:

Accept:

```
{"keep": true, "campaign_name": "...", "causal_hypothesis":
"...", "dominant_bottleneck": "capacity|bandwidth|compute_shape|
tdp|invalidity|roofline_violation", "search_patch": <unchanged
from oracle>, "prediction": {...quantitative...}}
```

Reject:

```
{"keep": false, "reason": "..."}
```

Deployment Profile: `interactive_speech`

```
DEPLOYMENT PROFILE (active for this campaign)
  natural-language goal: Minimize end-to-end latency for streaming
                         speech inference. Throughput is secondary.
                         Power should stay moderate. Small batch sizes
                         only; single-request latency is the metric.
  objective_mode:        min_latency
  soft preferences:      prefer_batch_size=small,
                         prefer_latency=very_high,
                         prefer_throughput=medium,
                         prefer_bandwidth=medium,
                         prefer_area=medium
  hard constraints:      max_tdp_relative=1.0,
                         max_area_relative=1.0,
                         max_batch_size=8
  search hint (use as prior_centroid neighborhood):
```

```
    native_batch_size:       [1, 2, 4, 8]
    L3_global_buffer_size:   [16, 32, 64, 128]

Tailor ‘search_patch‘ to this profile. Do NOT always maximize
Perf/TDP; respect the profile’s objective_mode. CORGIsolve V1
still optimizes inner-loop tiling/fusion/remat for each chip;
your job is to point it at the chip region whose trade-offs
match this user need.
```

Deployment Profile: `overnight_finance_batch`

```
DEPLOYMENT PROFILE (active for this campaign)
  natural-language goal: Maximize throughput for large overnight
                         historical-data analysis. Latency is not
                         important. Sustained throughput and memory
                         bandwidth matter most. TDP/power can be
                         high as long as the work completes by
                         deadline.
  objective_mode:        max_throughput_per_tdp
  soft preferences:      prefer_batch_size=large,
                         prefer_latency=low,
                         prefer_throughput=very_high,
                         prefer_bandwidth=very_high,
                         prefer_area=medium
  hard constraints:      max_tdp_relative=4.0,
                         min_batch_size=32
  search hint (use as prior_centroid neighborhood):
    native_batch_size:       [64, 128, 256]
    GDDR6_channels:          [16, 32]
    L3_global_buffer_size:   [128, 256]

Tailor ‘search_patch‘ to this profile. Do NOT always maximize
Perf/TDP; respect the profile’s objective_mode. CORGIsolve V1
still optimizes inner-loop tiling/fusion/remat for each chip;
your job is to point it at the chip region whose trade-offs
match this user need.
```

H.3 Resource Prices and Proposal Validation

Resource prices summarize the current identified program. Proposals can target expensive service, workspace, or retention constraints, but must be checked against their complete implementation contract. A proposal list is not an exhaustive pricing oracle: exhausting that list cannot establish full-master optimality. The native experiments use separately admitted schedules and complete-domain bound arguments as described in Appendix M.

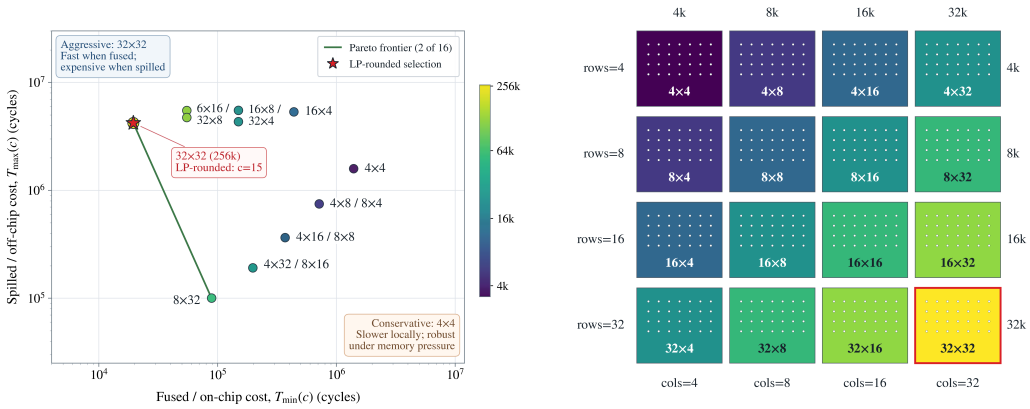

Figure 7: Example per-layer tiling cache for EfficientNet-B7 layer L4. Left: Timeloop-evaluated candidates plotted by fused/on-chip cost $T_{\min}(c)$ and spilled/off-chip cost $T_{\max}(c)$; the frontier shows non-dominated choices and the star is the LP-rounded selection. Right: the 16 tiling choices; labels such as 4×4, 8×8, and 32×32 denote active tile rows/columns inside the fixed accelerator, white dots indicate active PE/MAC sites. These are schedule choices, not different chips.

I Two Optimal Tiling Examples: joint-tile LP Solution at a Glance

This section shows examples of optimal tiling, where accepting a locally slower/smaller tiling to enable better fusion or placement. This showcases the strength of the CHEETAH formulation, which discovers nontrivial tiling-placement-fusion trade-offs that less expressive methods would miss. The first example is Llama-3 70B on a simulated H100; the second is DeepSeek-V3 on a simulated RTX 5090.

By fixing the architecture, the inner loop derives optimal tiling and scheduling strategies that can act as custom speedups for targeted inference tasks. Therefore this section briefly provides motivation for future work on expanded hardware variants. This is motivated and similar in spirit to the speedups seen be FlashAttention [Dao et al., 2022] due to tiling optimization.

I.1 joint-tile LP Solution: Llama-3 70B on simulated H100

Problem dimensions: $L = 720$ layers, $C = 20$ tile configs, $T = 720$ timesteps, $E = 703$ activation edges. Total LP variables: 1,601,280. Objective: 542,790,817 cycles. `CONVERGED_STRICT` in 1,800 PDLP iterations.

Tile-Config Selection ($y_{i,c}$): The joint-tile LP heavily favours small/fast configurations. Table 21 groups tiles into three size bands, and Table 22 gives the per-config histogram.

Table 21: Tile-config band distribution across $L = 720$ layers.

Tile size band	Layers	% of model
Small ($c = 0 \ldots 4$): fast / large working set	427	59%
Medium ($c = 5 \ldots 14$)	275	38%
Large ($c = 15 \ldots 19$): slow / small working set	18	2%

Config histogram (picks per config index):

Table 22: Number of layers assigned to each tile config c. Configs with zero picks are omitted.

c	0	1	3	4	5	8	12	16	17
Layers	285	1	1	140	1	148	125	15	3

Config $c = 0$ (largest tile) was selected for $\approx 40\%$ of layers, with $c \in \{4, 8, 12\}$ covering most of the remainder. The LP decisively favours large tiles for this 70B model on H100: with 1.05M MACs and a 50 MB global buffer, reducing tile size to fit memory is unnecessary and incurs iteration overhead.

Per-Layer Execution Time T_i:

$$T_i^{\min} = 1{,}204 \text{ cycles}, \quad T_i^{\text{med}} = 524{,}288, \quad T_i^{\max} = 1{,}835{,}008, \quad \sum T_i = 542{,}790{,}817.$$

The ten longest layers (Table 23) all hit the same 1,835,008-cycle bound: these are the large FFN gate/up/down and Q/K/V/output projection sublayers in the transformer middle blocks:

Table 23: Ten longest layers. All hit the maximum $T_i = 1{,}835{,}008$ cycles and correspond to large FFN/attention projection sublayers.

Layer idx	T_i **(cycles)**	**Selected config** c
359	1,835,008	4
376	1,835,008	4
375	1,835,008	4
368	1,835,008	4
0	1,835,008	4
367	1,835,008	4
718	1,835,008	0
358	1,835,008	8
357	1,835,008	8
377	1,835,008	12

Placement (p, p^W): Global Buffer Residency

- Activation edges pinned at $t = 0$: $0 \;/\; 703$
- Weight layers pinned at $t = 0$: $0 \;/\; 720$
- Total p-cells set across all t: $0 \;/\; 506{,}160$ (0.00%)
- Total p^W-cells set across all t: $1 \;/\; 518{,}400$ (0.00%)

The joint-tile LP makes almost no use of the global buffer for long-lived tensor residency and nearly everything streams through. With the L1/L2 buffer multipliers in this hardware configuration, **tiling alone is sufficient to keep the chip fed**; persistent pins are not needed.

Rematerialization Events $r_{i,t}$

$$\text{Layers with} \geq 1 \text{ remat event} : 0 \;/\; 720. \qquad \text{Total remat events} : 0.$$

Zero Rematerialization. joint-tile has the freedom to recompute layers to free memory pressure, but on this cell determined it was not worth the additional cycles. Activation streaming combined with tile-config selection is sufficient.

Eviction / Restoration Timeline

- Activation evictions / restorations: $0 \;/\; 0$
- Weight evictions / restorations: $1 \;/\; 1$ (one weight churn across all 720 timesteps)

One Concrete Layer: Layer 360 (Deep Transformer Block)

$$\text{Selected config } c = 0, \qquad T_i = 524{,}288 \text{ cycles}.$$

$$\mathbf{y}_{\text{bin}} \text{ nonzero indices} : \{0,\ 16\}$$

The LP relaxation assigned nonzero weight to two equally-good configs

What This Reveals About joint-tile on Llama-3 70B / H100

1. **Tile choice does most of the work, not placement or remat.** Of 1.6M decision variables, only ≈720 (the $y_{i,c}$ selections) are doing real work; placement (p, p^W) and rematerialisation (r) variables are essentially all zero.
2. **Large tiles win because H100 is large.** With 1.05M MACs and 3.35 TB/s DRAM bandwidth, there is no memory slack pressure forcing small working sets. The LP exploits this by selecting the largest physically permissible tiles.
3. **The $1.53\times$ lift over the naive baseline comes entirely from per-layer tile selection.** Ten different configs are active across the 720 layers, versus the naive one-size-fits-all heuristic. On smaller chips (e.g. TPU-v3) this same mechanism yields a $2.11\times$ lift, because the memory slack disappears and per-layer tile choice becomes correspondingly more impactful.

I.2 joint-tile LP Solution: DeepSeek-V3 on simulated RTX 5090

This example shows the LP makes real per-sublayer scheduling decisions that are directly readable from layer names.

As a qualitative example of joint-tile's solution structure, we inspect the rounded joint-tile LP solution for DeepSeek-V3 on an RTX-5090-like hardware point. The model contains L=903 layers, including 174 MoE sublayers, and the joint-tile LP has 2.5M variables. Although the Pareto cache exposes 20 tile configurations per layer, the solution uses only five configs (Table 24), 0,4,8,12,16, suggesting that joint-tile compresses the available tiling space into a small workload-specific alphabet. The most revealing behavior occurs in the FFN sublayers: ffn_up, ffn_gate, and ffn_down have identical compute time, yet joint-tile assigns them different tile configs in a rotating pattern (Table 25). This indicates that the LP is not merely selecting the fastest per-layer tile; it is interleaving large- and small-working-set choices across adjacent sublayers to manage memory pressure. joint-tile also automatically identifies structural outliers: the only seven layers assigned the smallest tile c=16 correspond to unusual sublayers such as rotary-key projections, low-rank Q projections, shared/expert gates, and expert down-projections (Table 26). Finally, weight churn is highly localized: only four attention-path weights in the first block are evicted/refetched (Table 27), while the MoE sublayers stream without churn. This case illustrates how joint-tile exposes interpretable schedule structure that is unavailable in a sequential Timeloop→fusion pipeline.

Problem Dimensions

- $L = 903$ layers (67-block DeepSeek-V3 with router/expert/shared sublayers per MoE block)
- $C = 20$ tile configs, $T = 903$ timesteps, $E = 886$ activation edges
- 2,504,019 LP variables, objective $= 517{,}356{,}013$ cycles
- Hardware (RTX 5090): 419K MACs, 1,792 GB/s DRAM, 98 MB CGM

Tile-Config Selection: Only 5 of 20 Configs Ever Picked Configs are spaced exactly every 4 indices. joint-tile does not sweep the full continuum but locks onto a discrete alphabet of five shapes (Table 24).

Table 24: Tile-config distribution for DeepSeek-V3 on RTX 5090. Only 5 of 20 candidate configs appear in the rounded solution.

Config c	Layers	% of model	
0	363	40.2%	(largest tile)
4	192	21.3%	
8	172	19.0%	
12	169	18.7%	
16	7	0.8%	(smallest tile: tightest working set)
others	0	unused	

Key Finding: joint-tile Staggers Tile Sizes Across the FFN Trio Every transformer block contains three FFN sublayers (up, gate, down) with identical compute time ($T_i = 2{,}580{,}482$ cycles each). joint-tile assigns *different* tile configs to the three so their working sets do not peak simultaneously in the global buffer. Table 25 shows the pattern across the ten hottest layers.

Table 25: Top-10 longest layers. Despite identical T_i, joint-tile assigns different tile configs to the FFN up/gate/down trio within each block, staggering working-set pressure across timesteps.

Layer idx	Name	T_i (cycles)	Config c
8	`layer0_ffn_up`	2,580,482	0
9	`layer0_ffn_gate`	2,580,482	12
10	`layer0_ffn_down`	2,580,482	8
19	`layer1_ffn_up`	2,580,482	4
20	`layer1_ffn_gate`	2,580,482	0
21	`layer1_ffn_down`	2,580,482	12
30	`layer2_ffn_up`	2,580,482	8
31	`layer2_ffn_gate`	2,580,482	4
32	`layer2_ffn_down`	2,580,482	0

This is the scheduling intelligence in joint-tile: same compute cost, different tiles, rotated so that any window of timesteps contains at most one large-working-set tile and two smaller ones. A naive mapper assigns one tile size to all FFN layers; joint-tile detects the working-set conflict and breaks the symmetry automatically.

Structural Outliers: The 7 Layers That Pick Config $c = 16$ The LP assigned the tightest working-set tile ($c = 16$) to exactly the sublayers with unusual shapes (rotary embeddings, shared-expert gates, and expert down-projections) without any hand-labeled outlier feature (Table 26).

Table 26: The seven layers assigned config $c = 16$ (smallest tile / tightest working set). All correspond to structurally atypical sublayers in the DeepSeek-V3 MoE architecture.

Layer idx	Name	T_i (cycles)	Reason
202	`layer14_K_rope`	1,146,881	Rotary-embedded K: small inner dim
238	`layer16_shared_gate`	286,720	Shared-expert gate: narrow
322	`layer22_K_rope`	1,146,881	Same shape as layer 202
394	`layer27_Q_b`	737,281	Q lower-rank decomposition
573	`layer39_Q_a`	215,040	Q upper-rank decomposition
706	`layer47_expert_gate`	286,720	Per-expert gate
812	`layer54_expert_down`	286,720	Per-expert down-projection

Weight Evictions: Localized to Transformer Block 1 joint-tile evicts weights for only four layers, all in the first transformer block's attention path (Table 27).

Table 27: Weight eviction/restoration events. Only transformer block 1 attention weights are evicted; all 174 MoE blocks have zero churn.

Layer	Name	Evict	Restore	T_i (cycles)	Config c
12	`layer1_Q_b`	1	0	737,281	0
13	`layer1_KV_a`	1	1	71,680	12
14	`layer1_KV_b`	1	1	327,680	8
15	`layer1_K_rope`	1	1	1,146,881	4

The LP determined that block 0/1 attention weights are cheaper to refetch than to evict the FFN trio they compete with in the global buffer.

Relative Layer Costs Table 28 summarizes the execution time of key sublayer types.

Table 28: Relative execution time of key sublayer types. MoE routers are ≈72× cheaper than FFN sublayers yet still receive varied tile assignments because 67 of them serialize in the dependency chain.

Sublayer type	T_i **(cycles)**	**Ratio vs. FFN**
FFN up / gate / down	2,580,482	1.00× (long pole)
Q/K/V projections	700K–1,200K	≈27–47%
MoE routers	35,840	1.4% (72× smaller)

What This Demonstrates About joint-tile

1. **joint-tile performs scheduling, not just per-layer tile picking.** The FFN trio tile rotation is a genuine scheduling decision: no naive per-layer mapper can produce this.
2. **joint-tile discovers structural outliers automatically.** The seven $c = 16$ picks land exactly on `K_rope`, `Q_a`, `Q_b`, `expert_gate`, and `expert_down` sublayers without any hand-labeled outlier feature in the input.
3. **joint-tile commits to a 5-config alphabet, not a 20-config sweep.** This is actionable for hardware designers: only 5 of 20 candidate tile shapes ever matter for this workload (Figure 8).
4. **Memory churn is highly localized.** Only 4 evictions occur, all in transformer block 1, all in the attention path. The 174 MoE blocks have zero churn: joint-tile streams them entirely.

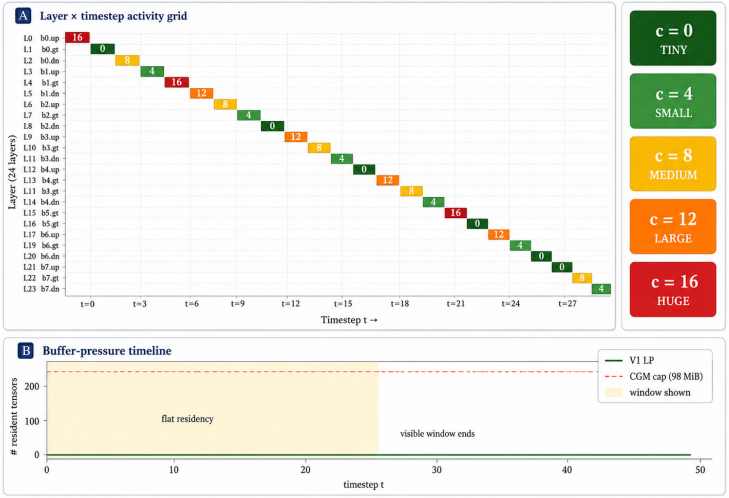

Figure 8: Visualization or rotating tiles selected by joint-tile LP. There exists 903 FFN sublayers in this workload, tile config C=20. The LP only picks 5 of them (since the rest are Pareto-dominated).

J Examples of CHEETAH Design Blueprints

We provide detailed blueprints for eight CHEETAH designs below (Variant 1: Tables 29, 30, 31 and 32; Variant 2: Tables 33, 34, 35 and 36). For each workload we give two variants: **Variant 1** (TSMC N7 + GDDR/LPDDR commodity memory) and **Variant 2** (TSMC N5 + HBM3 upgraded memory). Within a workload, the two variants share the same PE grid, so both inherit the same compute floor under the tile-dependent joint-tile LP, and achieve the same speedups. The variants differ only in area, TDP, and memory-subsystem cost. Every blueprint passes the roofline check and all six pre-RTL manufacturability gates of Appendix E.

Reading the utilization rows. Three rows below — compute utilization, fusion efficiency and ρ_{comp} — are the *same* recorded ratio $t_{\text{floor}}/T_{\text{LP}}$ under their recorded names for it, and differ only in rounding and percentage formatting. They measure how closely the LP objective approaches the mapper-derived compute floor of Eq. (13), not what fraction of the datapath's arithmetic the model occupies. In particular they are not comparable to FAST's compute utilization, which is a fraction of peak throughput and is reported there as 0.14 for simulated TPU-v3 [Zhang et al., 2022]; Appendix E.2 sets out the distinction and the FLOP-counting conventions a utilization comparison would require.

Compute boundedness and the bandwidth choice. The binding floor is compute, not bandwidth, at the target workloads examined. At the chip's peak DRAM bandwidth (range 544 GB/s LPDDR5X (N7) to 2,457 GB/s HBM3 (N5)), we measure $T_{\text{DRAM}}/T_{\text{compute}} \leq 3.3 \times 10^{-3}$ on every blueprint. Total per-forward DRAM bytes are (13B / 65B / 70B / DS-V3) = (30.5 / 121.8 / 126.2 / 95.7) GB, and production inference serving amortizes these reads across many forward passes per weight load (typical batches-per-load ≥ 100 in vLLM and TGI). Therefore the steady-state DRAM traffic the LP charges per forward is far below the full weight tensor. The two TSMC N7 and TSMC N5 variants presented in Subsections J.1 and J.2 aims to make the latency–cost–yield trade-off explicit. TSMC N7's commodity memory subsystem still meets the bandwidth floor with $\geq 200\times$ headroom (the LP consumes $< 0.4\%$ of peak BW), so paying for HBM3 (TSMC N5) buys lower TDP per delivered TFLOP, not higher compute throughput.

J.1 Variant 1: TSMC N7 + GDDR/LPDDR

Table 29: Workload: `llama1_13b` · TSMC N7 · $V_{\text{dd}} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.46\times$ (= 364 W / 250 W)
Normalized Area	$0.61\times$ (= 427 mm^2 / 700 mm^2)
Peak Compute	2,703 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	1,344 GB/s (GDDR6X $\times$16)
Memory capacity	32 GB
Batch size	256
Num PEs	1320 (33×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	2,010 FLOP/byte
Fused model OpInt	2.05×10^6 FLOP/byte
Performance (QPS)	11.08 QPS @ batch=256
Inference latency	90.22 ms / sample
Normalized Perf/TDP	$23.51\times$
Achieved speedup vs. naive TPU-v3	$34.22\times$
ρ_{comp} (Eq 12)	0.998
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 1,680 / 4096
reticle_area	PASS: 427 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,351,680 cells / 5,000,000
power_density	PASS: 0.63 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,351,680
DRAM bandwidth	1,344 GB/s
DRAM channels	16
PE array	33×40 (1,320 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	2.31×10^{10}
Compute floor cycles	2.31×10^{10}
Memory floor cycles	2.27×10^7 (at 1,344 GB/s)
TPU-v3 naive baseline	7.92×10^{11} cycles
Tile alphabet C	16
Layers L	360
LP solver	`custom JAX PDHG`
LP solve time	1.7 s

Table 30: Workload: `llama1_65b` · TSMC N7 · $V_{\text{dd}} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.20\times$ (= 300 W / 250 W)
Normalized Area	$0.51\times$ (= 357 mm^2 / 700 mm^2)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB
Batch size	256
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	4,818 FLOP/byte
Fused model OpInt	4.02×10^6 FLOP/byte
Performance (QPS)	1.37 QPS @ batch=256
Inference latency	729.94 ms / sample
Normalized Perf/TDP	$28.83\times$
Achieved speedup vs. naive TPU-v3	$34.60\times$
ρ_{comp} (Eq 12)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 357 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.74 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	1.87×10^{11}
Compute floor cycles	1.87×10^{11}
Memory floor cycles	2.24×10^8 (at 544 GB/s)
TPU-v3 naive baseline	6.42×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	6.7 s

Table 31: Workload: `llama3_70b` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.57\times$ (= 392 W / 250 W)
Normalized Area	$0.61\times$ (= 425 mm^2 / 700 mm^2)
Peak Compute	3,539 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB
Batch size	256
Num PEs	1728 (48×36)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.5%
Fusion efficiency	100.0%
OpInt ridgepoint	6,505 FLOP/byte
Fused model OpInt	2.07×10^6 FLOP/byte
Performance (QPS)	3.46 QPS @ batch=256
Inference latency	288.77 ms / sample
Normalized Perf/TDP	$25.61\times$
Achieved speedup vs. naive TPU-v3	$40.16\times$
Match to target	100.2%
ρ_{comp} (Eq 12)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 425 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,769,472 cells / 5,000,000
power_density	PASS: 0.83 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,769,472
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	48×36 (1,728 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	7.39×10^{10}
Compute floor cycles	7.40×10^{10}
Memory floor cycles	2.32×10^8 (at 544 GB/s)
TPU-v3 naive baseline	2.97×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	

Table 32: Workload: `deepseek_v3` · TSMC N7 · $V_{\text{dd}} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.20\times$ (= 300 W / 250 W)
Normalized Area	$0.51\times$ (= 357 mm^2 / 700 mm^2)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB (active-expert resident)
Batch size	128
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.2%
Fusion efficiency	100.0%
OpInt ridgepoint	4,818 FLOP/byte
Fused model OpInt	1.46×10^6 FLOP/byte
Performance (QPS)	2.40 QPS @ batch=128
Inference latency	417.32 ms / sample
Normalized Perf/TDP	$29.73\times$
Achieved speedup vs. naive TPU-v3	$35.68\times$
ρ_{comp} (Eq 12)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 357 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.74 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	5.34×10^{10}
Compute floor cycles	5.37×10^{10}
Memory floor cycles	1.76×10^8 (at 544 GB/s)
TPU-v3 naive baseline	1.91×10^{12} cycles
Tile alphabet C	16
Layers L	903
LP solver	`custom JAX PDHG`
LP solve time	

J.2 Variant 2: TSMC N5 + HBM3

Table 33: Workload: `llama1_13b` · TSMC N5 · $V_{dd} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.31\times$ (= 328 W / 250 W)
Normalized Area	$0.52\times$ (= 363 mm^2 / 700 mm^2)
Peak Compute	2,703 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	819 GB/s (HBM3 $\times 1$)
Memory capacity	24 GB
Batch size	256
Num PEs	1320 (33×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	3,300 FLOP/byte
Fused model OpInt	2.05×10^6 FLOP/byte
Performance (QPS)	11.08 QPS @ batch=256
Inference latency	90.22 ms / sample
Normalized Perf/TDP	$26.08\times$
Achieved speedup vs. naive TPU-v3	$34.22\times$
ρ_{comp} (Eq 12)	0.998
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 1,024 / 4096
reticle_area	PASS: 363 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,351,680 cells / 5,000,000
power_density	PASS: 0.75 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,351,680
DRAM bandwidth	819 GB/s
DRAM channels	8
PE array	33×40 (1,320 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	2.31×10^{10}
Compute floor cycles	2.31×10^{10}
Memory floor cycles	3.73×10^7 (at 819 GB/s)
TPU-v3 naive baseline	7.92×10^{11} cycles
Tile alphabet C	16
Layers L	360
LP solver	`custom JAX PDHG`
LP solve time	1.7 s

Table 34: Workload: `llama1_65b` · TSMC N5 · $V_{\text{dd}} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.74\times$ ($= 434\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.69\times$ ($= 485\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB
Batch size	256
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	1,067 FLOP/byte
Fused model OpInt	4.02×10^6 FLOP/byte
Performance (QPS)	1.37 QPS @ batch=256
Inference latency	729.94 ms / sample
Normalized Perf/TDP	$19.93\times$
Achieved speedup vs. naive TPU-v3	$34.60\times$
ρ_{comp} (Eq 12)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: $485\,\text{mm}^2 / 600\,\text{mm}^2$
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: $0.54\,\text{W/mm}^2 / 2.0$
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	1.87×10^{11}
Compute floor cycles	1.87×10^{11}
Memory floor cycles	4.96×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	6.42×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	6.7 s

Table 35: Workload: `llama3_70b` · TSMC N5 · $V_{\text{dd}} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$2.10\times$ ($= 526\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.79\times$ ($= 553\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	3,539 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB
Batch size	256
Num PEs	1728 (48×36)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.5%
Fusion efficiency	100.0%
OpInt ridgepoint	1,440 FLOP/byte
Fused model OpInt	2.07×10^6 FLOP/byte
Performance (QPS)	3.46 QPS @ batch=256
Inference latency	288.77 ms / sample
Normalized Perf/TDP	$19.09\times$
Achieved speedup vs. naive TPU-v3	$40.16\times$
ρ_{comp} (Eq 12)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: $553\,\text{mm}^2 / 600\,\text{mm}^2$
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,769,472 cells / 5,000,000
power_density	PASS: $0.64\,\text{W/mm}^2 / 2.0$
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,769,472
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	48×36 (1,728 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	7.39×10^{10}
Compute floor cycles	7.40×10^{10}
Memory floor cycles	5.14×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	2.97×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	

Table 36: Workload: `deepseek_v3` · TSMC N5 · $V_{\text{dd}} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.74\times$ (= 434 W / 250 W)
Normalized Area	$0.69\times$ (= 485 mm^2 / 700 mm^2)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times$3)
Memory capacity	72 GB (active-expert resident)
Batch size	128
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.2%
Fusion efficiency	100.0%
OpInt ridgepoint	1,067 FLOP/byte
Fused model OpInt	1.46×10^6 FLOP/byte
Performance (QPS)	2.40 QPS @ batch=128
Inference latency	417.32 ms / sample
Normalized Perf/TDP	$20.55\times$
Achieved speedup vs. naive TPU-v3	$35.68\times$
ρ_{comp} (Eq 12)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: 485 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.54 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	5.34×10^{10}
Compute floor cycles	5.37×10^{10}
Memory floor cycles	3.89×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	1.91×10^{12} cycles
Tile alphabet C	16
Layers L	903
LP solver	`custom JAX PDHG`
LP solve time	

K Limitations, Future Work, and Broader Impact

The model optimizes a declared workload, finite implementation library, and analytical hardware contract. The frozen 1,557-branch expanded master is certified to a 0.067154% LP objective gap. This completes that continuous instance; a larger implementation universe requires new bounds. OCS's closing stage uses manually selected supports and proof directions. General automatic support discovery and global discrete optimality remain open; neither LLM guidance nor a favorable numerical objective supplies a global optimality proof. Continuous relaxation bounds, feasible schedules, numerical inference equivalence, and physical timing are distinct claims. The full-input DeepSeek result uses captured routes and expanded attention under its recorded precision policy; its workload is reproducible, not statistically representative of every deployment. The large speedup depends on the reference and custom memory-interface assumptions. Same-chip controls isolate software effects but are not globally reoptimized ablations, and they must be read at their own hardware and workload: retention savings shrink as a link widens, batching can remove array padding without changing an external load count, and nested KV placement controls are not independent contributions.

The full-inference execution credits compatible batching, KV retention/placement, and SwiGLU storage fusion. Larger native fusion groups, unrestricted overlap, quality-changing menus, and general rematerialization gains need additional experiments. RTL and synthesis studies cover selected schedules and datapaths, not complete-chip calibration. Extending the framework to multiple chips, robust workload distributions, or training introduces further communication and numerical constraints. More efficient inference designs may reduce energy per task; deployment scale and manufacturing costs determine their broader resource impact.

L Complete Rows of the Master Blocks

Equation (CHEETAH master LP) is a perspective formulation of a declared finite family. If branch b has equality matrix E_b, inequality matrix G_b, right-hand sides e_b, g_b, and coordinate box $[\ell_b, u_b]$, its complete block form is

$$\begin{aligned} \min_{\lambda,x} \quad & \sum_{b\in\mathcal{B}_C} \kappa_b \sum_{e\in\mathcal{E}_b} d_{be} + \sum_{b\in\mathcal{B}_P} \alpha_b \left(\sum_i T_{bi} + \sum_{i,t} c_{bi}^{\mathrm{remat}} r_{bit} \right) \\ \text{s.t.} \quad & E_b x_b = e_b \lambda_b, \quad G_b x_b \le g_b \lambda_b, \\ & \ell_{b\nu} \lambda_b \le x_{b\nu} \le u_{b\nu} \lambda_b, \quad \lambda_b \ge 0, \quad \sum_b \lambda_b = 1. \end{aligned} \tag{19}$$

This is the explicit coefficient expansion of $\sum_b c_b^T x_b$: duration coordinates carry cost κ_b in causal branches; operation latency and rematerialization carry their original costs in retained schedule branches. All other coordinates in these blocks have zero objective cost. Branch weights are not multiplied into this expression a second time. A native-only instance has $\mathcal{B}_P = \varnothing$. The declared row matrices include the direct physical constraints below precisely when that versioned branch includes them. Only finite endpoints generate box rows. A branch at weight one retains its original constraints and objective; setting every other branch to zero gives an explicit inclusion map. This does not transfer a lower bound from one branch to a larger family. The retained schedule block below explains inclusion of the supplied joint LP when it is configured as a branch. The certified 21-branch native instance uses the causal blocks and their identified extensions; its bound is not asserted for a union with additional schedule models or a larger implementation library.

L.1 The Retained Joint Tiling–Placement Block

For original operation i, configuration c, edge e, and fixed time label t, let $P_{ik} = p_{\mathrm{assoc}(i,k),o(i)}$ for an associated activation, $P_{iW} = p^W_{i,o(i)}$ for weights, and $P_{ik} = 0$ when no placement association exists. The block's contribution to the master objective is $\alpha_b(\sum_i T_i + \sum_{i,t} c_i^{\mathrm{remat}} r_{it})$; α_b converts

its cost units. Its explicit rows, in the original joint-LP notation, are

$$\sum_c y_{ic} = \lambda_b, \qquad h_b = h_b^P \lambda_b,$$
$$T_i \geq \sum_c T_i^{\min}(c) y_{ic}, \qquad T_i \geq \sum_c T_i^{\max}(c) y_{ic} - \sum_{c,k} t_i^k(c) w_{ick},$$
$$0 \leq w_{ick} \leq y_{ic}, \quad w_{ick} \leq P_{ik}, \quad w_{ick} \geq y_{ic} + P_{ik} - \lambda_b,$$
$$\sum_e d_e p_{et} + \sum_i d_i^W p_{it}^W \leq C_{GM,b} \lambda_b,$$
$$p_{et} \leq p_{e,t-1} + r_{\mathrm{src}(e),t} \quad (t > o(\mathrm{src}(e))),$$
$$p_{it}^W \leq p_{i,t-1}^W \quad (t > o(i)), \qquad r_{it} = 0 \quad (t \leq o(i)),$$
$$r_{it} \leq p_{et}, \quad r_{it} \leq p_{it}^W \quad (e \in \mathrm{in}(i),\ t > o(i)),$$
$$T_i \geq 0, \qquad 0 \leq y, w, p, p^W, r \leq \lambda_b.$$

These rows preserve the original fractional savings convention. Extra cold-start, copy, or exact-product constraints are not appended to this block. Causal execution uses its own complete contract instead.

L.2 Native Cover, Lifecycle and Storage

All following equations use scaled coordinates in one native branch. The event range is $0 \leq e < E$, with storage/completion states at $0 \leq e \leq E$. A column covers its original nodes S_k, including any conversion nodes; all cover, work-mass, active-state, typed $w = f$, control and operand-readiness rows are those in Eq. (CHEETAH master LP). In particular $\sum_k a_{ke} = A_e \leq \lambda_b$ is an aggregate active-mass restriction. Distinct invocations retain distinct coverage constraints even when coefficients are reused.

Let O_k be implementation outputs and τ_k their declared destination. For initial indicators $I_j, I_{j\tau}$, the birth and tier recurrences are

$$\beta_{j0} = I_j \lambda_b, \qquad p_{j\tau 0} = I_{j\tau} \lambda_b,$$
$$\beta_{j,e+1} = \beta_{je} + \sum_{k: j \in O_k} f_{ke},$$
$$p_{j\tau,e+1} = p_{j\tau e} - v_{j\tau e} + \sum_{\sigma:\sigma\to\tau} q_{j,\sigma\to\tau,e} + \sum_{k: j\in O_k, \tau_k = \tau} f_{ke} + \mathbf{1}_{\{\tau = D,\ j \in \mathcal{J}_{\mathrm{stage}}\}} \delta_{je}. \tag{20}$$

H, D, S denote host, DRAM and SRAM. The drain term occurs only in staging branches. Ordinary copy links are $H \leftrightarrow D \leftrightarrow S$. A copy retains its source and adds destination state at the next event; it is not a destructive move or instantaneous readiness. State, eviction, copy, choice and lifecycle masses have boxes $[0, \lambda_b]$.

For a nonterminal object's original users $\mathcal{U}_j$, backing requires

$$\begin{aligned} &\beta_{je} \leq p_{jHe} + p_{jDe} + p_{jSe} + D_{ie} && (i \in \mathcal{U}_j,\ 0 \leq e \leq E), \\ &\beta_{je} \leq p_{jHe} + p_{jDe} + p_{jSe} && (j \text{ terminal},\ 0 \leq e \leq E), \\ &\beta_{jE} \leq p_{jHE} + p_{jDE} && (j \text{ terminal}). \end{aligned} \tag{21}$$

Terminal backing implies the corresponding user rows because $D_{ie} \geq 0$; their exact elimination changes no feasible point. KV appends use original object identities, complete prefix operands and these same persistence rules. The native certified family adds no recomputation columns. A separate recomputation extension must charge repeat work, gate it on prior production, restore tier state without a second birth/completion, and explicitly identify which backing obligations can be discharged by recomputation.

Selected numerical representation. A raw FP8 parameter and its converted BF16 parameter are different objects. For converted object j, raw owner $r(j)$, and consumer i, define $R_{ij} = \sum_{k \ni i:\, k \text{ reads } r(j)} y_k$. Selected-reader branches replace the corresponding nonterminal user row by

$$\beta_{je} \leq \sum_\tau p_{j\tau e} + D_{ie} + R_{ij}, \qquad R_{ij} \leq \sum_\tau p_{r(j),\tau e} + D_{ie}. \tag{22}$$

The actual selected implementation still requires readiness in its declared input tier. Original conversion nodes remain covered. Earlier branches retain their original backing rows; Eq. (22) is an identified alternative contract, not an equivalence transformation of them.

L.3 Capacities, Staging and Installed Hardware

Define the transient ordinary occupancy

$$o_{j\tau e} = p_{j\tau e} - v_{j\tau e} + \sum_{\sigma:\sigma\to\tau} q_{j,\sigma\to\tau,e}.$$

Let $N = h_{b,\text{arrays}}$, $S = h_{b,\text{SRAM}}$ in MiB, and $V = h_{b,\text{vector}}$. The source introduces $\chi_e \approx NA_e$ with the four linear inequalities

$$n_- A_e \le \chi_e \le n_+ A_e, \qquad N + n_+ A_e - n_+\lambda_b \le \chi_e \le N + n_- A_e - n_-\lambda_b. \tag{23}$$

Let W_N, W_V be the original per-array and active-vector workspace constants, P_k private panel SRAM, D_k private DRAM workspace, and $O_{k\tau} = \sum_{j\in O_k:\tau_k=\tau} m_j$ output reservation. The newest native branch uses

$$\begin{aligned}
&\sum_j m_j o_{jSe} + W_N\chi_e + W_V A_e + \sum_k (P_k + O_{kS})a_{ke} + (C_{\text{stage}} + C_{\text{receiver}})\lambda_b \le 2^{20}S,\\
&\sum_j m_j o_{jDe} + \sum_k (D_k + O_{kD})a_{ke} + \sum_{j\in\mathcal{J}_{\text{stage}}} m_j\delta_{je} \le C_D\lambda_b,\\
&\sum_j m_j o_{jHe} + \sum_k O_{kH}a_{ke} \le C_H\lambda_b.
\end{aligned} \tag{24}$$

Branch-specific absent terms are zero. Entry and exit capacity rows count the resident state and installed dedicated reservations, without active work or in-flight transfers. Installed bank, register, controller and receiver requirements also give affine hardware rows, alongside area and power proxies. Host/local channel counts and receiver variants are whole-program choices; arrays, vector lanes, SRAM and DRAM bandwidth remain shared continuous coordinates inside each branch.

Explicit parameter staging. For each stageable owner j, the additional coordinates obey

$$\begin{aligned}
&\eta_{j0} = \eta_{jE} = 0, \quad \eta_{j,e+1} = \eta_{je} + \rho_{je} - \delta_{je},\\
&0 \le \delta_{je} \le \eta_{je} \le \beta_{je}, \qquad 0 \le \rho_{je} \le p_{jHe} - v_{jHe},\\
&\sum_j m_j\eta_{je} \le C_{\text{stage}}\lambda_b,\\
&\sum_j m_j(\eta_{je} - \delta_{je} + \rho_{je}) + \sum_{j,\ H\in\{\tau,\tau'\}} \min(m_j, C_{\text{stage}})q_{j,\tau\to\tau',e} + \sum_k R_k^{\text{stage}}a_{ke} \le C_{\text{stage}}\lambda_b.
\end{aligned} \tag{25}$$

The state bound applies through exit; the transition row applies to action events. R_k^{stage} is the original implementation's reservation: ordinary host service uses $\min(B_k^H, C_{\text{stage}})$, while full-K and receiver implementations use their explicit panel/buffer reservation. The drain enters DRAM state and transient capacity in Eqs. (20)–(24). Dedicated staging and receiver SRAM cannot simultaneously hold ordinary data.

L.4 Native Resource Demands and Linear Service Envelopes

For original stream bytes B_k, MAC demand M_k, and vector work V_k, the native variable-rate demands are

$$\begin{aligned}
Q_e^B &= \sum_{j,\tau\to\tau'} m_j q_{j,\tau\to\tau',e} + \sum_k B_k w_{ke} + \sum_{j\in\mathcal{J}_{\text{stage}}} m_j\delta_{je},\\
Q_e^N &= \sum_k (M_k/256)w_{ke}, \qquad Q_e^V = \sum_k V_k w_{ke}.
\end{aligned} \tag{26}$$

Here one native array has 16×16 MACs. All traffic on the same DRAM service pool enters Q_e^B before applying its epigraph. Different resource services each satisfy $Z_e^r \le d_e$. For a rate $r \in [\ell, h]$, $u_- = 1/h$, $u_+ = 1/\ell$, the ideal homogenized reciprocal and product rows are

$$\begin{aligned}
&\ell\lambda_b \le r \le h\lambda_b, \quad u_-\lambda_b \le u \le u_+\lambda_b,\\
&u \ge 2\lambda_b/a - r/a^2 \quad (a \in \mathcal{A}), \qquad u \le ((\ell+h)\lambda_b - r)/(\ell h),\\
&0 \le Q \le \bar{Q}\lambda_b, \quad Z \ge 0, \quad Z \ge u_- Q,\\
&Z \ge \bar{Q}u + u_+ Q - \bar{Q}u_+\lambda_b, \quad Z \le u_+ Q, \quad Z \le \bar{Q}u + u_- Q - \bar{Q}u_-\lambda_b.
\end{aligned} \tag{27}$$

The implemented LP uses outward-rounded reciprocal endpoints and row coefficients derived from exact rational interpretations of supplied binary64 values. Each coefficient error is paid against its original coordinate box in the row RHS, rounded upward; errors on unbounded coordinates are refused. Thus stored rows contain the physical graph without asserting equality to it. The source demand bounds $\bar{Q}$ cover the full declared generator domain, not just activated working columns.

Fixed-rate ports use integer-scaled service rows. With B_k^H, B_k^L the original host/local byte coefficients, t_k^{setup} host setup ticks, B_k^{stage} the staging service bytes and t_k^{recv} receiver ticks,

$$\begin{aligned}
&\sum_{j,\ H\in\{\tau,\tau'\}} m_j q_{j,\tau\to\tau',e} + \sum_j m_j\rho_{je} + \sum_k (B_k^H + r_H t_k^{\mathrm{setup}}) w_{ke} \le r_H d_e,\\
&\sum_{j,\ S\in\{\tau,\tau'\}} m_j q_{j,\tau\to\tau',e} + \sum_k B_k^L w_{ke} \le r_L d_e,\\
&\sum_j m_j(\rho_{je} + \delta_{je}) + 2\sum_{j,\ H\in\{\tau,\tau'\}} m_j q_{j,\tau\to\tau',e} + \sum_k B_k^{\mathrm{stage}} w_{ke} \le r_{\mathrm{stage}} d_e,\\
&\sum_k t_k^{\mathrm{recv}} w_{ke} \le d_e, \qquad \sum_k t_k^{\min} w_{ke} \le d_e.
\end{aligned} \tag{28}$$

Stage-owner sums are restricted to $\mathcal{J}_{\mathrm{stage}}$. Ordinary kernels pay $B_k^{\mathrm{stage}} = 2B_k^H$; streamed implementations substitute their paid staging recipe. Absent receiver/setup/floor terms are zero. The current source retains the floor row when its implementation contract requires it. The objective is $\kappa \sum_e d_e$, with κ the exact stored binary64 conversion from ns to ms. Integral replay adds its declared clock, descriptor and dispatch accounting; LP event sharing alone does not establish realizable kernel overlap.

L.5 Direct Roofline and Physical Constraints

The strengthened fixed-chip implementation uses one installed chip $\bar{h}_b$ per whole-program branch. Its scaled coordinates satisfy $h_b = \lambda_b \bar{h}_b$; no operation chooses a separate chip. Let $\bar{C}_b$ be installed BF16 matrix throughput in FLOP/ns, and $\bar{B}_{br}$ the delivered byte/ns rate of interface r. With original useful work $F_k^{\mathrm{useful}} \le F_k^{\mathrm{issued}}$ and linear traffic incidence B_{bre}, the source adds

$$\begin{aligned}
&\sum_k F_k^{\mathrm{issued}} w_{ke} \le \bar{C}_b d_e, && \mathsf{B}_{bre}(q,w,\rho,\delta) \le \bar{B}_{br} d_e,\\
&\sum_k V_k w_{ke} \le \bar{V}_b d_e, && b \in \mathcal{B}_{\mathrm{phys}},\ e \in \mathcal{E}_b.
\end{aligned} \tag{29}$$

For the suite's 16×16 arrays at 1 GHz, $\bar{C}_b = 512\bar{N}_b$ and $F_k^{\mathrm{issued}} = 512\,\mathrm{jobs}_k\,\mathrm{bundle}_k\,\mathrm{reductionCycles}_k$; one MAC is two FLOPs. The demand sums include every concurrently active implementation and transfer sharing a resource. HBM, host, local SRAM, DMA staging and aggregate bank ports have separate incidences and limits. The suite's external-copy staging row charges at least $64\lceil m/32\rceil$ bytes for a transfer of m bytes, accounting for its 32-byte-word write and read. This is an alignment-independent necessary bound; replay pays actual alignment, commands and queue costs. Kernel local traffic, SRAM-copy endpoints and DMA staging enter one aggregate bank-port row.

For a selected branch ($\lambda_b = 1$), summing over the same event interval with total duration $T = \sum_e d_e > 0$ gives

$$\mathrm{MFU} = \frac{\sum_{e,k} F_k^{\mathrm{useful}} w_{ke}}{\bar{C}_b T} \le \frac{\sum_{e,k} F_k^{\mathrm{issued}} w_{ke}}{\bar{C}_b T} \le 1, \qquad \mathrm{MBU}_r = \frac{\sum_e \mathsf{B}_{bre}}{\bar{B}_{br} T} \le 1. \tag{30}$$

Traffic includes the represented reads, writes, repeated loads and transfers; vector arithmetic is not counted as BF16 matrix work. No tolerance declares a positive resource residual zero at exact admission. These inequalities remain linear because the installed rates are branch constants. In a variable-hardware relaxation, $C(h)d_e$ is bilinear; its McCormick envelope does not provide this exact fixed-chip guarantee. Fractional branch mixtures and intermediate solver iterates are not reported as fabricated or executable chips.

Write installed-component area and all-components-active peak power as $\mathsf{A}_b^\lambda = a_{b0}\lambda_b + a_b^T h_b$ and $\mathsf{P}_b^\lambda = p_{b0}\lambda_b + p_b^T h_b$. The shared constraint compiler adds the following exact affine rows:

$$\begin{aligned}
&\mathsf{A}_b^\lambda \le 600\lambda_b, \quad \mathsf{P}_b^\lambda \le 300\lambda_b, \quad \mathsf{P}_b^\lambda \le 2\mathsf{A}_b^\lambda, \\
&\frac{h_{b,B}}{\eta_H \beta_H} \le n_H^{\max}\lambda_b, \quad \frac{h_{b,B}}{\eta_H \beta_H} + \frac{\bar{B}_{b,\text{host}}}{\eta_{\text{host}}\beta_{\text{host}}}\lambda_b \le n_{\text{pkg}}^{\max}\lambda_b, \\
&h_{b,B} \le \bar{B}_{b,\text{controller}}\lambda_b, \quad 2^{20}h_{b,S} = 65536\, n_b\lambda_b, \quad n_b h_{b,N} \le 5{,}000{,}000\lambda_b.
\end{aligned} \tag{31}$$

Here area is in mm^2, power in W, β is byte/ns per signal pin, η is delivered efficiency, n_b is the fixed installed bank count, and $h_{b,B}, h_{b,S}, h_{b,N}$ denote bandwidth, SRAM MiB and arrays. The declared BF16 scenario uses at most 4,096 combined host/HBM signal pins, whole 64-KiB banks with $n_b \le 4{,}096$, bank-array aspect between $1/4$ and 4, and a logic-depth proxy at most 35. Each bank supplies a combined 32-byte/cycle port at 1 GHz. Installed-buffer lower bounds prevent hardware arrays, vector registers or DMA storage from borrowing memory already assigned to live data. Bank organization, pipeline depth and interface choices are fixed template constants; an invalid template cannot become an admitted positive-weight branch.

Peak power uses 1.2 mW per BF16 MAC in the revised analytical scenario, with separately priced SRAM, vector, controller and interface components. It is not multiplied by MFU. Inherited area coefficients and their sensitivity factors remain declared assumptions. The NoC-size and chip-average density rows are proxies, not detailed network congestion or hotspot models; arbitrary bank/link overlap additionally needs its actual resource incidence. LP construction and recovery use the same identified coefficient set and gates.

Implementation and certificate scope. These rows are compiled by `native_physical_contract.py` and `native_suite_physical_rows.py`; the fixed-rate native wrapper also adds direct compute, bandwidth and vector ceilings. The complete saved Llama-13B execution passes 520,216,367 added scalar rows and four hardware boxes. Integration into every other master construction path is unfinished. The 1,557-branch expanded family has a complete 0.067154% certificate (Appendix M.3), retaining variable hardware, its declared physical rows and its original product/bank relaxation. Its fractional point does not inherit the exact installed-rate MFU guarantee of the fixed-chip wrapper merely by satisfying a relaxed product envelope.

L.6 Feature Scope

Table 37 lists the encoded families. The generic formulation can express further clock, memory technology or reuse contracts, but they are not thereby members of the certified finite native pool. Adding a branch, implementation, or hardware range requires a new instance identity and new bounds. Quality-changing alternatives additionally need their own numerical contract.

Table 37: Master encoding and the current native performance result.

Family	Encoding	Native result credited here
Hardware sizing	Shared resources, installed banks, area/power rows	One chip; explicit interface assumptions
Tiling and batching	Independent cover and work coefficients	Compatible shared-weight GEMMs and per-use tiles
Fusion/registers	Original ports, outputs and private workspace	SwiGLU and elementwise storage choices
Temporal placement	Tier states, paid copies, transient capacity	246 KV chains vs. 193 in a fixed partition
KV persistence	Append identities, prefix readiness, terminal backing	Full history and paid restoration
Parameter supply	Staging, panel buffers, raw/converted backing	Full-K panels and paid FP8-to-BF16 receiver
Rematerialization	Retained schedule or explicit repeat extension	No native repeat-column gain claimed
Precision/sparsity	Separately qualified implementation contracts	Fixed capture precision; no pruning/quality gain
Overlap/reordering	Event placement and resource epigraphs	Serial inter-kernel replay; internal panel pipelines

Table 38: Encoded decision families and actual recovery coverage. The master formulates broader freedoms than the decoder implementation menus. "Fixed" means an active inherited policy, not a free selector.

Feature	DeepSeek decode	Llama decode
Hardware resources	Selectable finite whole-chip branches	Selectable finite whole-chip branches
Implementation / tiling	Independent compatible full-K choices	Full-K choices; 13B menu grows from 25 to 31
Activation placement	Resident arena fixed; rounded producer layout selectable	Original DRAM arenas and boundaries fixed
Weight residency	Original resident backing fixed; loads paid	Eligible whole-owner retention selectable; loads paid
Fusion / storage	Rounded attention producer-to-AV layout selected; inherited gate/up and SwiGLU	Inherited boundaries; no new fusion recipe
Tiling–placement coupling	Footprint, ports and shared resources checked; general lifetime exchange absent	Same restriction
KV retention	Original owners, appends and entry fixed and charged	Original owners/appends charged
MoE residency / offload	Original routing, groups and resident parameters fixed	Not applicable
Rematerialization	Formulated; new decoder implementation absent	Formulated; new decoder implementation absent
Scheduling / concurrency	Cross-N prefetch and two contexts selectable; arbitrary overlap absent	Ordinary serial DRAM kernels
Serving / phase choice	Decode-only; original request order and starting KV fixed	Same restriction

M Native Full-Inference Provenance and Reproduction

M.1 End-to-End Solution and Recovery Workflow

The completed continuous solve uses a CHEETAH-specific certificate-producing procedure. Compact working problems discover witnesses; the complete original LP determines acceptance. The method retains an explicit record of manually chosen supports and proof directions. It is not a demonstrated automatic branch-and-price or full-master PDHG algorithm.

1. Freeze the mathematical instance. The expanded family contains 1,557 whole-program branches, retaining all 61,985,244 independent native invocations and 62,030,334 logical nodes including conversions. The admitted branch has 15,044,386,833 implementation choices and 5,809,955,157 original event labels. Hardware ranges, physical rows, source coefficients, original boxes and the declared bank/product relaxation remain fixed throughout closing. Generated coefficient classes share arithmetic, not invocation decisions. Implementations outside this identified library are outside the certificate.

2. Fit coverage prices against actual source menus. The compiler enumerates every real ancestral implementation on its original membership, saved tile, representation and eligibility conditions. Coincident memberships retain all genuine implementation options. Original exact-cover equalities have free multipliers; inequality multipliers keep their required signs. The final closing stage uses 45,085 finite CPU HiGHS fits over the nonempty families, with eight workers; six empty families require no solve. Complete checks cover 31,706,528 weighted invocations and 39,223,145 group inequalities. Analytic KV, conversion and prefill prices complete the source coverage construction under one shared resource budget.

3. Reconstruct and lift a stateful primal. Numerical cover marginals are reconstructed on a 2^{32} rational grid; integer incidence arithmetic verifies exact coverage of every original member. Literal source compilation binds 12,264,122 positive original coordinates and 255 actual coefficient templates. A causal prefix supplies mass $f = 1/41{,}969{,}600$ of the inherited execution. The suffix supplies the remaining mass through the new cover, preserving independent coordinates and original event labels. The selector is $fy^{\text{prefix}} + (1-f)y^{\text{suffix}}$; coverage alone is insufficient to establish feasibility.

The lift therefore reconstructs owner states, readiness and all required transfers. It mirrors 25,607,232 nonterminal owners between their existing SRAM/DRAM tiers, pays initialization and suffix copies, and preserves original initial inventory, KV history and terminal backing. One further CPU HiGHS program jointly adjusts original KV alternatives, shared hardware, reciprocal variables and service epigraphs on this candidate construction. Its 10,245 variables, 1,039 equalities, 104 inequalities and 112,633 nonzeros cover 1,022 KV classes. Reusing a coefficient-class mixture restricts this candidate search, not the frozen master; its optimizer objective supplies no unrestricted lower.

4. Check the original domain and compose the certificate. For $\min c^T x$ subject to $Ex = e$, $Gx \le g$, $x \in [\ell, u]$, the Lagrangian convention used by the checker gives, for $\mu \ge 0$ and free ν,

$$L(\nu, \mu) = -e^T\nu - g^T\mu + \inf_{\ell \le x \le u} (c + E^T\nu + G^T\mu)^T x. \tag{32}$$

All original coordinate bounds, residual signs and recession directions enter this calculation. In particular, a tiny negative residual on an unbounded nonnegative coordinate cannot be rounded to zero. Restricted multiplier search is permitted because each returned lower is evaluated over the complete original domain. An omitted-column working dual alone is insufficient.

The primal checker verifies the complete source lift, using exact rational fields and affine temporal identities rather than materializing all events. Saved-point reconstruction and separate replay reproduce the objective and proof. The final adopted receipt also includes the explicit staging-reservation supplement. Complete branch proofs, including zero-weight perspective cases, give $L = \min_b L_b$; the best admitted branch supplies U. We stop at the first checked $(U - L)/U \le 10^{-3}$. The final composition checks committed proof receipts and identities; it does not rerun every inherited checker. Shared checker algorithms and manual support selection are recorded limitations of automation and independent implementation, not missing terms in the certificate. Appendix M.3 gives the accepted endpoints.

5. Search discrete hardware and recover software. The hardware branch-and-bound tree is separate from the whole-program branch index b in the LP. A tree node is a rectangle in a declared finite menu of array counts, vector lanes, DRAM rates, host channels and local-service units. Minimum-resource budget violations prune infeasible rectangles. Optimistic resource values, relaxed private-capacity constraints and repriced local implementations provide latency lower estimates. In a joint variant, each parameter family has a two-state retained/streamed dynamic program. Nonnegative prices on shared capacity rows yield the conditional Lagrangian bound

$$L(h,\mu) = \sum_f \min_{\pi \in \mathcal{P}_f} \left[t_f(h,\pi) + \sum_t \mu_t B_{ft}(\pi) \right] - \sum_t \mu_t C_t$$

plus costs outside those paths. Here f identifies an original parameter family, π an allowed retention path, and B_{ft} its storage use. GPU scans evaluate the independent paths; shared prices preserve the capacity coupling in the bound. A path mixture or optimistic rectangle is not itself an executable schedule.

The demonstrated menu contains 536,380,416 hardware combinations: 64–1,024 arrays in steps of two, 1,024–16,384 vector lanes in steps of 128, 512–2,560 GB/s DRAM in steps of 128 plus 2,608 GB/s, 1–64 host channels, and 1–8 local-service units. An earlier fixed-software tree exhausted that menu; the joint capacity-path tree made 1,190 GPU oracle calls and retained 142 unresolved schedule leaves, reduced to 78 by its subsequent replayed incumbent. Its bounds were numerical and conditional on the then-current cover, progress and implementation menu. They are not the final LP certificate, and subsequent menu expansions require new tree bounds.

Only a replayed feasible design supplies a pruning incumbent; an unverified candidate does not. Surviving hardware choices undergo integral cover/path selection, capacity repair and transfer scheduling. Replay then computes the predicted phase latencies using the selected chip's actual modeled rates, clocks, formats and buffer reservations. This is how an integral hardware/software result is obtained without claiming a completed tree over every master selector.

What an exhaustive discrete proof would additionally require. One would need complete lower bounds at every remaining hardware/schedule node, pricing over every allowed implementation under each branching decision, and a fully explored or pruned frontier. Making tile or branch choices integral alone does not make continuous rate/product envelopes exact. Spatial branching to enforce physical products addresses a stronger nonlinear design problem; it is not necessary to solve the frozen LP relaxation and is not claimed here. Analytical replay supports the reported design latency, while fabrication and global discrete optimality remain separate questions.

M.2 The OCS Solver

Activated columns and temporal fields define a restricted working LP with a checked lift to original coordinates. Let $f(x) = c^T x + I_{[\ell,u]}(x)$, let Kx stack equality and inequality row activities, and let $g(z) = I_{\{z_{\mathrm{eq}}=b_{\mathrm{eq}},\ z_{\mathrm{ineq}} \le b_{\mathrm{ineq}}\}}(z)$. The selected fixed-step OCS kernel performs

$$\begin{aligned}
\widehat{v}^{k+1} &= \mathrm{prox}_{\Sigma g^*}(v^k + \Sigma K \bar{x}^k), \\
\widehat{x}^{k+1} &= \mathrm{prox}_{Tf}(x^k - TK^T \widehat{v}^{k+1}), \\
(x^{k+1}, v^{k+1}) &= \alpha_k (x^{\mathrm{anchor}}, v^{\mathrm{anchor}}) + (1-\alpha_k)(\widehat{x}^{k+1}, \widehat{v}^{k+1}), \qquad \alpha_k = \frac{1}{k+2}, \\
\bar{x}^{k+1} &= 2x^{k+1} - x^k.
\end{aligned} \tag{33}$$

Each block restarts from its average. Centered arithmetic computes differences from the anchor to avoid repeatedly summing a large baseline. The primal proximal map is $\Pi_{[\ell,u]}(x - Tc)$; equality dual coordinates receive the RHS shift, while $\le$-row multipliers additionally project onto the nonnegative orthant. Reversible scaling and diagonal steps are verified against the original operator. The tested paged adapter moves bounded vector pages between host storage and GPU; it reduces GPU memory but still stores the working iteration vectors somewhere.

Variable metric. The diagonal metric rescales the primal and dual step sizes by the smoothed square root of the dual-to-primal residual ratio, which keeps primal and dual progress balanced on a

system whose coefficients span ten orders of magnitude. Reversible scaling and JAX operator kernels [Chambolle and Pock, 2011, Halpern, 1967, Applegate et al., 2022, Bradbury et al., 2018] apply K and K^T without forming the event-expanded matrix. On a 36,508-column, 313,061-row residency support the kernel reaches a checked 0.0774% interval in 313.25 seconds on one A100.

Closing stage. The kernel supplies prices and working supports; the closing stage certifies them. Signed coverage prices are fitted against each family's actual implementation menus, including coincident ancestral alternatives, with 45,085 nonempty-family compact LP fits across eight CPU workers (six empty families need no call), their cover reconstructed on a 2^{32} rational grid. A stateful lift preserves a causal prefix and pays for the copies, evictions, workspace and reservations required by the changed suffix choices; one further fit with 10,245 variables and 112,633 nonzeros adjusts shared hardware, reciprocal/service fields and original KV alternatives, and its restricted support generates the candidate upper. Rational temporal fields and affine interval identities then represent the primal and verify residuals over every original event, and dual multipliers are checked for their original signs and valid Lagrangian box support over all coordinates, so no event-expanded matrix or dense temporal witness is ever allocated. Support and proof directions were selected manually; automatic discovery for arbitrary instances is not established.

M.3 Certified Solution of the Frozen Expanded Master

We solve the complete frozen 1,557-branch DeepSeek co-design LP to a certified **0.067154% relative objective gap**. The result combines an original-source-admitted continuous primal with complete branchwise lower certificates, retaining all original implementations and temporal events. It covers the declared variable-hardware family, not only a working support or the earlier fixed-chip LP. Its target identity is

`40897836035c66a132467ca7254a54e4de505d0e42a0551f2a7177281c82ff61`.

Exact result. The checked endpoints, in milliseconds, are

$$L = \frac{1830773506473914770133641322 5125}{193428131138340667952988 16},$$
$$U = \frac{1948570268098944085881269008519433323761143612595330 8591}{2057355529685916849955078447444831462062494318592 0}.$$

Their decimal previews are $L = 946{,}487.719081842$ ms and $U = 947{,}123.742096447$ ms. The exact relative interval is

$$\frac{U-L}{U} = \frac{1385455227641099641285831440160526 5879}{2063129021399938564020791190701430542 5879} = 0.000671531064354\ldots < 10^{-3}.$$

The approximately 636.023015-ms absolute interval is below the 947.123742-ms allowance. All 1,557 branches remain accounted for: 1,095 have lower bounds at least $0.999U$, 462 have checked fixed-cost infeasibility, and zero remain unresolved at this accuracy. Branch 212 supplies the upper. The composition joins 1,787 committed endpoint receipts and checks the complete original family, including zero-weight perspective obligations.

Full-domain lower from compact price fits. The successful proof assigns free prices π_i to original exact-cover equalities. For each admissible implementation k with members S_k, its member prices must be dominated by its actual priced resource cost. The source compiler includes every genuine ancestral implementation and eligibility condition; equal shapes do not imply equal availability. Disabled choices retain their exact zero boxes. In the final resource construction on the 1,536 added branches, let H_b be the stored host rate, B_b^{cap} the stored bandwidth cap, D_k^B stream bytes, H_k host bytes, s_k paid host-setup ticks, and $\mathcal{I}_k$ the original member set:

$$a_b = \max\{1, 256/H_b\}, \qquad N_b = B_b^{\mathrm{cap}} + a_b H_b,$$
$$C_{bk} = D_k^B + a_b(H_k + H_b s_k), \qquad \sum_{i\in\mathcal{I}_k} \pi_i \le C_{bk}.$$

The bandwidth-cap and host-service rows share the duration budget κ. Their demand/work contributions cancel using actual implementation costs; only cover-equality prices change. Each

choice residual is $\kappa(C_{bk} - \sum_{i \in \mathcal{I}_k} \pi_i)/N_b \geq 0$. Transfer residuals are nonnegative, unused unbounded-coordinate residuals vanish, and the original box-support calculation supplies the complete lower. All algebra uses the actual stored rational coefficients, including $\kappa = 4722366482869645/472236648286964521369 6$. Branch-specific normalizers and original-row multipliers are checked explicitly; scalar gains from separate resource bounds are not simply added. The earlier branches retain their own complete proofs. Zero-weight cases have zero cover RHS, and the perspective weight residuals are $L_b - L \geq 0$ for $L = \min_b L_b$, certifying the entire family.

The final closing stage raises the complete lower by 698.017580 ms using previously unpriced KV operands and conversion host traffic, by 164.928562 ms using exact ancestral menus, and by 292.970650 ms using a disjoint prefill matrix partition. These are sequential improvements of one valid certificate, not independent unavoidable latencies or feature speedup credits. Negative prices belong to equality rows; inequality signs remain unchanged.

Source-admitted primal and staging supplement. Integer cover reconstruction preserves all 31,706,528 weighted invocations. The stateful lift binds 12,264,122 positive original columns, pays SRAM/DRAM mirror traffic and checks 25,607,232 nonterminal owners. Initial inventory, KV growth and reads, terminal backing, shared capacity and the original physical/service envelopes remain represented. Complete temporal proofs cover all original labels. The checker evaluates 20,131 prefix coefficient classes and 1,147,467 original Forms and coordinate boxes, as well as the changed implementation and owner conditions.

The final admission also incorporates a required staging supplement: the first checker included staging service Forms but lacked an explicit check of changed reservations. For 1,278 coefficient templates and 90,485 staged owners, the supplement verifies $r_k \leq C_{\text{stage}}$ and $\sum_k a_{ke} \leq 1$, implying $\sum_k r_k a_{ke} \leq C_{\text{stage}}$. Staging state/receive/drain fields remain identically zero in this inherited construction; new mirror copies use only SRAM and DRAM. The adopted wrapper binds this supplement and the original proof/replay. The point and its objective do not change.

Saved-rational reconstruction and a separate checker invocation reproduce U. The literal objective is the prefix duration, paid initialization and suffix resource maxima, using the frozen conversion coefficient. The prefix is unchanged; initialization costs about 1.0565 ns. The net upper improvement is $2{,}189.759860$ ms. Priced transfer bytes are not independently added as latency where the source uses shared epigraph maxima.

Numerical provenance and stopping. The final closing computation uses **CPU HiGHS**: 45,085 finite coverage fits across eight workers and one 10,245-variable resource/KV candidate fit. Analytic price constructions, rational reconstruction, verification and branch composition use no optimizer. These are closing-stage costs; inherited campaigns also contributed witnesses and proofs. Full primal admission takes 268.51 s and saved-point replay 196.26 s; the slowest coverage worker takes 49.80 s. Concurrent times are not an additive end-to-end clock. Discovery stops at the first checked acceptance. No full-input OCS/PDHG update or convergence supplies this result. Manually selected support and proof directions remain part of the discovery record; no general-purpose automatic solver is claimed.

Execution remains a separate result. The retained later chip and supported schedule have predicted latency $2{,}718{,}970.508466$ ms: $804{,}507.786532$ ms prefill, $1{,}871{,}489.872783$ ms decode and $42{,}972.849152$ ms conversion. The chip uses 320 16×16 BF16 arrays, 4,736 vector lanes, 1,024 conversion lanes, 78.25 MiB SRAM and 1,981 GB/s DRAM service; its declared analytical area and peak power are 323.206858 mm^2 and 299.9295 W. It is distinct from the other chips and matched controls retained elsewhere in this paper. The new continuous certificate does not change any execution timing or baseline ratio. Original-source admission of this returned execution establishes its membership, not an embedding theorem for every possible execution. The LP upper is fractional and uses the frozen reciprocal/product relaxation; neither it nor its small objective interval proves near-optimality of the recovered chip.

Reproduction and verification scope. The sealed package `native_menu_state_20261027_v1` contains 13,902 files and 3,518,764,197 bytes, with verified Elements, Mazeppa and A100 readbacks. Its external content-manifest SHA-256 is

`d92c11134f59a4a7c4a7b4a7834418417df61c71db08bddb586094be6864b4a0`.

The authoritative artifacts are `composition_stateful_v1`, `stateful_adoption_v2`, `stateful_cycle_v2`, `stateful_staging_check_v1` and `discovery_accounting_v2`. The final composition validates committed receipts and identities rather than rerunning every inherited source checker. Saved-point replay shares checker algorithms; separate mathematical/source reviews supplement it. Preceding input/runtime dependencies remain in their recorded verified roots, so the new package is not a self-contained replacement for all preceding evidence. Replay instructions reconstruct the saved point without new optimization.

N Coefficient Audit Trail: Where Every Constant Comes From

Tables 39 (cost model), 40 (gate thresholds), 41 (menus), 42 (workload quantities) and 43 (solver grids) give the inventory, which records the prefill study's coefficients and menu assumptions. It is not the numerical contract for the native full-input experiment, which is specified in Appendix M. R denotes a reported measurement or silicon anchor, A an assumption, and D a derived value; these labels describe provenance, not validation of every proposed chip or precision configuration. Timeloop-derived costs are modeled estimates. Quantization and prefix-cache menu entries below receive no native speedup credit. Source paths refer to the original released builders, whose legacy `socp` filenames do not change the linear form of the exported program.

The full inventory, including per-coefficient file and line provenance, is maintained in `systems/deepseek_socp_formulation.md` in the released code; constants marked A above are the explicit modeling assumptions a reviewer can vary, and every one enters the program linearly or through a one-hot menu, so sensitivity to any single assumption is a re-solve, not a re-derivation.

Table 39: Silicon cost model: area and power coefficients. Sources: `feasibility_checks.py` lines 32–48 and the LP builders (`deepseek_socp_codesign.py` lines 341–350, `deepseek_socp_serving_codesign.py` lines 147–152).

Constant	Value (units)	St.	Anchor
`RETICLE_AREA_MM2` (A_{cap})	600 (hard limit 858) mm^2	A	ASML/TSMC 26×33 mm reticle field is 858 mm^2 (the standard lithography limit); 600 leaves yield headroom, the same cap style as FAST [Zhang et al., 2022]
`MM2_PER_MAC` (k_{MAC})	1.5×10^{-4} mm^2/MAC (INT8, 7 nm)	R	**Post-synthesis extracted** (Appendix D.3): OpenROAD/ASAP7 7 nm placement of an INT8 systolic tile at 256/1024/4096 MACs gives 8.21×10^{-5} mm^2/MAC of cell area ($R^2 = 0.999999$ linear, 0.46% spread), i.e. this value at 54.7% placement utilization; also calibrated so TPU-v3 inputs (65,536 MACs) reproduce its core area, and consistent with Simba 8-bit PE area at 16 nm scaled to 7 nm [Shao et al., 2019]
`MM2_PER_MB_SRAM` (k_{SRAM})	1.5 mm^2/MB (7 nm)	R	TSMC N7 SRAM bitcell 0.027 μm^2/bit (ISSCC 2017 N7 test chip) plus macro overhead gives 1.4–1.6 mm^2/MB
`MM2_PER_DRAM_CHANNEL`	8.0 mm^2/channel	R	HBM PHY plus controller floorplan area on TPU/GPU die shots (TPU-v4i die analysis) [Jouppi et al., 2021]
`MIN_CHIP_AREA_MM2`	50.0 mm^2	A	IO ring, control, and power-grid floor observed on small accelerator dies (Simba chiplet package overheads) [Shao et al., 2019]
`W_PER_MAC_AT_FULL_UTIL`	0.4×10^{-3} W/MAC at 1 GHz (7 nm INT8)	R	Calibrated to TPU-v3 250 W TDP at 65,536 MACs × 700 MHz, then roughly 5× 7 nm INT8 efficiency (Horowitz energy scaling) [Jouppi et al., 2021]
`DEFAULT_UTILIZATION`	0.5 (fraction)	A	Measured LLM-serving MFU of 30–50% on production accelerators (vLLM and TPU serving reports) [Kwon et al., 2023]
$k_{\text{HBM,byte}}$ (HBM-resident capacity area)	$k_{\text{SRAM}}/10 = 1.43 \times 10^{-7}$ mm^2/byte	A	DRAM roughly 10× denser than SRAM per bit (standard memory-technology density gap)
k_{BW} (bandwidth area)	0.08 mm^2/(GB/s)	D	`MM2_PER_DRAM_CHANNEL` (Real) divided by roughly 100 GB/s per HBM3 channel (819 GB/s per stack over 8 channels, JEDEC JESD238)
k_{LD} (load-engine area)	$= k_{\text{BW}} = 0.08$ mm^2/(GB/s)	A	Load/DMA engine area assumed comparable to PHY area at matched bandwidth
π_{MAC} (power per MAC at operating util.)	2.0×10^{-4} W/MAC	D	Product of `W_PER_MAC_AT_FULL_UTIL` (TPU-v3 calibrated, Real) and `DEFAULT_UTILIZATION` [Jouppi et al., 2021]. **Silicon-anchored**: 0.4×10^{-3} W/MAC at 1 GHz is 0.40 pJ/MAC-op, bracketed by whole-chip shipped INT8 silicon (H100 0.35, A100 0.64, TPU-v4i 1.27 pJ/op), which an array-only coefficient must sit at or below (Appendix D.3)
P_{cap} (TDP cap)	700 W	A	NVIDIA H100 SXM5 TDP is 700 W (vendor datasheet)
π_{bit} (DRAM access energy)	4.0×10^{-12} J/bit	A	HBM2 access energy roughly 3.9 pJ/bit (O'Connor et al., Fine-Grained DRAM, MICRO 2017)

Table 40: Pre-RTL manufacturability gate thresholds. Sources: `feasibility_checks.py` lines 57–64 and 174.

Constant	Value (units)	St.	Anchor
`COOLING_W_PER_MM2` (power-density gate)	2.0 W/mm^2	A	Envelope over real silicon: TPU-v3 0.36, TPU-v4 0.55, H100 0.7, MI300X 0.6, WSE-3 1.5 W/mm^2; 2.0 corresponds to aggressive AIO/liquid cooling
`MAX_LOGIC_DEPTH` (logic-depth gate)	35 gate levels at 1 GHz	A	FO4-per-cycle timing literature: roughly 25–35 logic levels per 1 ns cycle at 7 nm (Harris/Horowitz FO4 method)
`SRAM_BANK_KB` (sram-macro gate)	64 KB per macro	R	Standard foundry SRAM compiler macro size; Simba uses 64 KB-class buffers per PE cluster [Shao et al., 2019]
`MAX_BANKS` (sram-macro gate)	4096 banks	A	Practical routing limit for a banked global buffer; conservative floorplanning rule
`MAX_BANK_ASPECT` (sram-macro gate)	4.0 (aspect in [1:4, 4:1])	A	Standard physical-design floorplanning guideline for macro arrays
`HBM3_GBPS_PER_PIN` (pin gate)	0.8 GB/s per pin (6.4 Gbps / 8)	R	JEDEC HBM3 standard JESD238: 6.4 Gb/s per pin
`MAX_PIN_COUNT` (pin gate)	4096 pins (4 stacks $\times$ 1024 data pins, roughly 3.3 TB/s)	R	JEDEC HBM3 1024-bit interface per stack; 4-stack package standard (TPU-v3 uses 4 HBM stacks) [Jouppi et al., 2021]
Crossbar routing-cell cap (crossbar gate)	5×10^6 cells ($n_{\text{pes}}\times n_{\text{banks}}$; flat-crossbar bound 10^6, relaxed 5$\times$ for a 2-level hierarchical NoC)	A	Industry hierarchical-NoC precedent: TPU-v4 mesh plus crossbar, H100 GPC/L2 partitions, MI300X chiplet XGMI, WSE-3 tile fabric [Jouppi et al., 2021]

Table 41: Precision, datatype, and decision menus. Sources: `feasibility_checks.py` lines 65–73 and `deepseek_socp_codesign.py` lines 228–280.

Constant	Value (units)	St.	Anchor
`MAC_WIDTH_BY_DTYPE`	int8 = 8, bf16/fp16 = 16, fp32 = 32 bits	R	Datatype bit-widths by definition (IEEE 754 / bfloat16)
Precision-menu byte fractions (b_W, b_A, b_{KV})	W8A8 = (1,1,1); W4A8 = (0.5,1,1); W4A8-KV4 = (0.5,1,0.5); W4A16 = (0.5,2,1)	A	Illustrative settings motivated by AWQ (W4), GPTQ (W4), and KVQuant (KV4); not quality-qualified here [Lin et al., 2024, Frantar et al., 2023, Hooper et al., 2024]
Array-aspect menu dims	sq16x16 = (16,16); wgmma8x32 = (8,32); tcgen05_32x8 = (32,8) rows $\times$ cols	R	NVIDIA Hopper WGMMA and Blackwell tcgen05 native MMA tile shapes (PTX ISA); ThunderKittens documents the H100 tile constraint [Spector et al., 2024]
Utilization $\eta(\text{spec}, a_r, a_c)$	$\eta = (\text{rows}/\lceil\cdot\rceil_r)(\text{cols}/\lceil\cdot\rceil_c)$, clipped to $[10^{-3}, 1]$	A	Analytic ceil-quantization model of array underfill; reproduces the Timeloop mapper low-GEMV-utilization artifact [Parashar et al., 2019]
DVFS menu γ (clock scale)	[0.7, 1.0, 1.3] for lo/nom/hi	A	Standard CMOS DVFS operating points ($\pm$30% around nominal); f proportional to V
DVFS menu p_f (dynamic-power factor)	[0.5, 1.0, 1.9] for lo/nom/hi	A	P proportional to fV^2 (roughly cubic in f), the CMOS dynamic-power law
Radix-cache menu sizes	[0, 64, 256, 1024] MB	A	Cache-size sweep grid; an on-chip-SRAM-scale prefix cache [Zheng et al., 2024]
Radix-cache hit-rate profile η_s	[0.0, 0.30, 0.50, 0.65] (saturating), fraction of prefill reuse	A	SGLang RadixAttention measured prefix-cache hit rates (saturating in cache size) [Zheng et al., 2024]

Table 42: Workload-derived quantities (DeepSeek-V3). Sources: `deepseek_socp_codesign.py` lines 180–183 and 283–363, `timeloop_full.py` lines 137–156.

Constant	Value (units)	St.	Anchor
Timeloop Pareto tile menu $T_{\min}(c)$, $T_{\max}(c)$, $t_{I/O/W}(c)$, $d_{I/O/W}(c)$	per-op per-tile arrays (cycles; bytes)	R	Measured Timeloop mapper Pareto frontier for DeepSeek-V3 layers [Parashar et al., 2019]
f_e routing frequency (balls-in-bins base)	$f_{\text{unif}} = 1-(1-8/256)^B$ (0.983 at B=128; 0.638 at B=32)	D	Distinct-expert balls-in-bins derivation from the DeepSeek-V3 top-k=8 of n=256 routed experts [DeepSeek-AI, 2024]
Zipf routing-skew exponent	0.6 (mean-normalized tilt on f_{unif})	A	Mild expert-load imbalance consistent with the MoE load-balancing literature (DeepSeek-V3 aux-loss-free routing reports residual skew) [DeepSeek-AI, 2024]
E_{distinct} (expected distinct experts per decode step)	$256(1-(1-8/256)^{32}) \approx 163$ at batch 32	D	Balls-in-bins expectation at the decode operating batch; f_e renormalized to this mean [DeepSeek-AI, 2024]
W_e (per-expert weight bytes, 8-bit)	$3 \times 7168 \times 2048 =$ 44.0 MB (up, gate, down)	D	DeepSeek-V3 config: hidden 7168, MoE intermediate 2048 [DeepSeek-AI, 2024]
d_{KV} (MLA KV bytes per token, 8-bit)	$61 \times (512+64) =$ 35,136 B/token	D	DeepSeek-V3 MLA: $n_{\text{layers}} = 61$ times (kv_lora_rank 512 plus qk_rope_head_dim 64) cached per token [DeepSeek-AI, 2024]
`DEEPSEEK_V3_CONFIG`	n_{layers}=61, hidden 7168, 256 routed experts, k=8 active, kv_lora_rank 512, qk_rope_head_dim 64, MoE intermediate 2048	R	DeepSeek-V3 technical report architecture table [DeepSeek-AI, 2024]

Table 43: Solver grids and operating-point references. Sources: `deepseek_socp_codesign.py` lines 220 and 326–334, `deepseek_socp_serving_codesign.py` lines 72–129.

Constant	Value (units)	St.	Anchor
M_{ref} (reference MAC count)	65,536 MACs	R	Declared TPU-v3 reference MAC count [Jouppi et al., 2021]
BW_{ref} (reference bandwidth)	900 GB/s	A	TPU-v3 4× HBM2 headline bandwidth [Jouppi et al., 2021]
MAC design envelope $[M_{\min}, M_{\max}]$	[16,384, 4,194,304] MACs	A	Spans 1/4 TPU-v3 to 64× TPU-v3 within reticle; a design-space envelope (FAST-style search bounds) [Zhang et al., 2022]
BW design envelope $[\text{BW}_{\min}, \text{BW}_{\max}]$	[100, 8000] GB/s	A	Low end GDDR-class single channel; high end HBM3E multi-stack (roughly 8 TB/s, B200-class)
`cgm_grid_mb` (CGM one-hot menu)	[1, 2, 4, 8, 16, 32, 64, 128, 256] MB	A	Power-of-two sweep bracketing TPU-v3's 32 MB on-chip global memory [Jouppi et al., 2021]
`BW_GRID` (bandwidth one-hot grid)	[225, 450, 675, 900, 1350, 1800, 2400, 3200, 4200, 5400, 6700, 8000] GB/s	A	Enumerated grid from quarter-TPU-v3 HBM2 to HBM3E multi-stack; one-hot branches are solved integrally (exact per branch), and the pin gate prunes entries above 3276.8 GB/s at the 4096-pin cap